upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

E.T.S. de Ingeniería Industrial, Informática y de Telecomunicación

Grado en Ingeniería Informática

Generando rostros humanos etiquetados mediante GANs para reconocimiento de emociones

Autor

Adrian Mikel Maiza Coupin

Director

Mikel Galar Idoate

Trabajo Fin de Grado

Pamplona, 09/2022

"A menudo les digo a mis alumnos que no se dejen engañar por el término "inteligencia artificial": no tiene nada de artificial. La IA está hecha por seres humanos, destinada a comportarse por seres humanos y, en definitiva, a influir en la vida de los seres humanos y en la sociedad humana."

- Fei-fei Li

AGRADECIMIENTOS

A mi tutor por el conocimiento que me ha brindado y los recursos que me ha proporcionado.

A mis amigos, por acompañarme en todos estos años de carrera.

A mis padres por toda la ayuda que me han proporcionado y a mi querida abuela (†) *por todos los ánimos que me ha dado.*

RESUMEN

Dada la creciente oleada de investigaciones en el ámbito de las redes generativas adversarias (GAN) en estos últimos años y viendo que se pueden conseguir resultados de alta calidad con modelos como StyleGAN2 [1], se abre la posibilidad de generar imágenes sintéticas como nunca antes. Esto permitiría a pequeños proyectos generar sus propios *datasets* de imágenes acorde a las necesidades de éstos, sin tener que depender de terceros ni de tener que recopilar los datos manualmente, lo que llevaría un tiempo y coste extra.

Las ventajas de la generación de *datasets* mediante GANs abre un nuevo abanico de posibilidades, ya que podrían generarse bases de datos de rostros a la carta que podrían utilizarse para entrenar modelos de reconocimiento de expresiones faciales. Además, estos modelos permitirían generar datos de manera más balanceada, mitigando así los posibles sesgos en los datos originales. También, las imágenes generadas no representarían a ningún ser humano real [2]. Por lo que se podría obtener más soltura en el manejo de los datos, facilitando la investigación.

El propósito de este proyecto es estudiar la viabilidad de utilizar datos sintéticos para el entrenamiento de modelos de reconocimiento de expresiones faciales mediante estas redes generativas adversarias. Para ello se proponen dos métodos de generación de datos controlada, ya que en este caso es necesario que los rostros generados expresen diferentes emociones. Por tanto, ambos métodos tratan de generar imágenes de rostros con una entrada aleatoria y ciertos datos de entrada adicionales como el género, la raza, la edad y el sentimiento que debe expresar dicho rostro.

En este proyecto se estudia la viabilidad de cada uno de los modelos generativos comparando sus tiempos de entrenamiento, así como la calidad de los resultados de manera cualitativa y de manera cuantitativa, utilizando métricas acordes a las GAN. Posteriormente, se entrenan modelos de reconocimiento de expresiones para comparar el rendimiento según el conjunto con el que han sido entrenados (datos reales o sintéticos).

PALABRAS CLAVE

- *Reconocimiento de expresiones faciales*
- *Modelos generativos*
- *Redes Generativas Adversarias*
- *Generación de rostros condicionados*
- *Aprendizaje profundo*

ÍNDICE

RESUMEN v

ÍNDICE DE FIGURAS xii

ÍNDICE DE TABLAS xiii

1. INTRODUCCIÓN 1

1.1 Objetivos 4

2. ANTECEDENTES 5

2.1 Aprendizaje adicional 5

2.1.1 Especialización en Aprendizaje Profundo 5

2.1.2 Especialización en Redes Generativas Adversarias 5

2.2 Reglamento General de Protección de Datos (RGPD) 5

2.3 Historia de las Redes Neuronales 6

2.4 Elementos básicos de una red neuronal y su funcionamiento 7

2.4.1 La neurona 7

2.4.2 Función de activación 8

2.4.3 Función de coste 8

2.4.4 Algoritmo de retro-propagación 9

2.5 Aprendizaje Profundo 10

2.5.1 Entrenamiento, validación y testeo 10

2.5.2 Batch Normalization 11

2.5.3 Dropout 11

2.5.4 Media Móvil Ponderada Exponencial (MMPE) 12

2.5.5 Optimizador Adam 13

2.5.6 Optimizador DiffGrad 14

2.6 Métricas de clasificación 15

2.7 Redes Neuronales Convolucionales 16

2.7.1 Convolución 17

2.7.2 Pooling 17

2.7.3 Redes Neuronales Deconvolucionales 18

3. REDES GENERATIVAS ADVERSARIAS 19

3.1 W-GAN . . . 21
3.2 Penalización de Gradiente . . . 22
3.3 Distancia de Incepción de Fréchet (FID) . . . 22
3.4 Precision y recall . . . 23
3.5 Conditional-GAN . . . 24
3.6 Normalización de Instancias Adaptativa (AdaIN) . . . 25
3.7 Pro-GAN . . . 26
3.8 StyleGAN . . . 27
3.9 StyleGAN2 . . . 29

4. BÚSQUEDA DE BASES DE DATOS 32

4.1 TinyFace . . . 32
4.2 CelebA . . . 33
4.3 UTKFace+ . . . 33
4.4 Fer2013 . . . 35
4.5 FairFace . . . 35
4.6 AffectNet . . . 36

5. ENTORNO DE DESARROLLO 39

5.1 Lenguaje de programación y herramientas . . . 39
5.1.1 Librerías . . . 39
5.2 Hardware utilizado . . . 40

6. EXPERIMENTOS PREVIOS 41

6.1 Experimentación con redes GAN . . . 41
6.2 Experimentación con redes GAN condicionales . . . 45

7. PROPUESTA 47

7.1 Método 1: Generación de rostros mediante StyleGAN2 condicional . . . 47
7.2 Método 2: Generación de rostros mediante StyleGAN2 y aprendizaje auto-supervisado contrastivo . . . 50

8. EXPERIMENTACIÓN Y RESULTADOS 52

8.1 Experimento 1 . . . 52

8.2 Experimento 2 53
8.3 Experimento 3 55
8.4 Experimento 4 56
8.5 Extracción de métricas 58
8.6 Comparación de resultados 60
8.7 Resultados teóricos del Método 2 62

9. COMPARATIVA DE DATOS Y DISCUSIÓN **63**

10. CONCLUSIÓN Y LÍNEAS FUTURAS **66**

REFERENCIAS **67**

ANEXOS **72**

A Primer anexo 72

ÍNDICE DE FIGURAS

1.1 Competición ILSVRC. . . . 1
1.2 Número de artículos sobre IA en arXiv 2010-2019 [5]. . . . 2
1.3 *Supersampling* con modelo TecoGAN [7]. . . . 2
1.4 Recoloración con DeOldify [8]. . . . 2
1.5 Evolución en las imágenes generadas por la IA (Brundage, et al., 2018) [14]. . . . 3
1.6 Rostros generados por StyleGAN2 [1]. . . . 3
2.1 Perceptrón. . . . 6
2.2 Ejemplos de redes neuronales. . . . 6
2.3 Estructura y componentes de una red neuronal. . . . 7
2.5 Diagrama de red neuronal con las derivadas para *back-propagation*. . . . 9
2.6 Características jerárquicas aprendidas de un modelo de aprendizaje profundo. . . . 10
2.7 Diagramas de redes neuronales con y sin *dropout*. . . . 12
2.8 Diferencias entre los optimizadores Adam y DiffGrad [23]. . . . 14
2.9 Matriz de confusión. . . . 15
2.10 Diagrama de características de la jerarquía visual del cerebro humano. . . . 16
2.11 Diagrama de una Red Neuronal Convolucional VGG-16 [25]. . . . 16
2.12 Ejemplo de convolución sobre una imagen. . . . 17
2.13 Distintas técnicas de *pooling*. . . . 17
2.14 Ejemplo de convolución transpuesta sobre una imagen. . . . 18
2.15 Arquitectura U-Net para segmentación de imagen. . . . 18
3.1 Arquitectura de una red GAN. . . . 19
3.2 Diferencias entre los VAE y las redes GAN. . . . 20
3.3 Problemas comunes en el entrenamiento de redes GAN. . . . 21
3.4 Dos distribuciones de datos con su representación gráfica de *precision* y *recall*. . . . 23
3.5 Caras generadas por modelo Conditional-GAN en base a la clase “Raza”. . . . 24
3.6 Arquitectura de red GAN condicionada. . . . 25
3.7 Transferencia de estilos. . . . 25
3.8 Entrenamiento progresivo de una red Pro-GAN. . . . 26
3.9 Método progresivo de inclusión de nuevo escalado. . . . 26
3.10 Diferencias entre el generador de Pro-GAN y StyleGAN. . . . 27
3.11 Ejemplos de variación estocástica. . . . 28
3.12 Artefactos en imágenes generadas por StyleGAN [31]. . . . 28
3.13 Estructura detallada de StyleGAN [1]. . . . 29
3.14 Nueva arquitectura y de-modulación de pesos. . . . 30
3.15 Artefactos de “fase” al cambiar la postura de un rostro en StyleGAN. . . . 31
3.16 Tres arquitecturas de generadores y discriminadores estudiados en [1]. . . . 31
4.1 Ejemplos del *dataset* TinyFace [37]. . . . 32
4.2 Ejemplos del *dataset* CelebA [38]. . . . 33
4.3 Ejemplos del *dataset* UTKFaces [39]. . . . 34
4.4 Sesgo en las clases “Raza” y “Edad” en UTKFaces [39]. . . . 34
4.5 Ejemplos del *dataset* FER2013 [40]. . . . 35
4.6 Ejemplos del *dataset* FairFace [41]. . . . 36
4.7 Instancias erróneas del *dataset* AffectNet. . . . 37
4.8 Sesgo en las clases de AffectNet [42]. . . . 37
4.9 Ejemplos del *dataset* AffectNet [42]. . . . 38
6.1 Resultados obtenidos con red GAN profunda. . . . 41
6.2 Arquitectura de GAN para *dataset* CelebA. . . . 42
6.3 Resultados obtenidos con red W-GAN profunda. . . . 42
6.4 Problemas con implementaciones de redes DC-GAN en Tensorflow. . . . 43

6.5 Desvanecimiento de gradiente con modelos implementados en Tensorflow. 43
6.6 Resultados de red DC-GAN (GAN convolucional profunda con `PyTorch`). 44
6.7 Progreso de entrenamiento de red WDC-GAN (de izquierda a derecha). 44
6.8 Resultados obtenidos mediante cWDC-GAN. 45
6.9 Arquitectura de red cDC-GAN. 45
6.10 Resultados obtenidos mediante cDC-GAN. 46
6.11 Resultados por "Edad" (de izquierda a derecha) y "Raza" (de arriba a abajo). . 46
7.1 Diferentes conjuntos de subvariedades. 48
7.2 Arquitectura de StyleGAN2 condicional. 49
7.3 Diferencias entre el generador tradicional y el generador con MME. 49
7.4 Arquitectura y fases de entrenamiento de GAN-Control (basada en [57]). 50
8.1 Resultados del Experimento 1 con condicionado en estilo y rostros centrados. . . 53
8.2 Imágenes generadas con deformaciones en el área de la boca. 53
8.3 Resultados del Experimento 2 con condicionado en mapeo. 54
8.4 Diferencias en calidad de ejemplos por clases desbalanceadas. 54
8.5 Número de ejemplos antes y después de la reducción de instancias. 55
8.6 Valores de la función de coste adversario durante el entrenamiento. 56
8.7 Resultados del Experimento 3 con claro colapso de moda. 56
8.8 Resultados del Experimento 4 con tamaño 256×256. 57
8.9 *Precision* y *recall* de los *checkpoints* seleccionados en experimentos 2, 3 y 4. . . . 60
8.10 Ejemplos con etiquetas específicas. 61
8.11 Ejemplos conseguidos con modelos pre-entrenados de GAN-Control [57]. 62
9.1 Balance entre el número de clases de "Expresión" de cada *dataset* (en %). 63
9.2 Matrices de confusión de los modelos ResNet50 entrenados. 64

ÍNDICE DE TABLAS

2.1 Métricas obtenidas mediante la matriz de confusión de la Figura 2.9. 15
3.1 Ejemplo de codificación *one-hot*. 24
5.1 Especificaciones técnicas del sistema. 40
6.1 Tiempos de entrenamiento para distintas configuraciones de redes DC-GAN. . . . 44
8.1 Híper-parámetros del Experimento 1. 52
8.2 Híper-parámetros del Experimento 2. 53
8.3 Híper-parámetros del Experimento 4. 57
8.4 Métricas de los mejores *checkpoints* de cada experimento. 61
9.1 Resultados de los modelos ResNet50 de cada configuración. 64

1. INTRODUCCIÓN

El campo de la inteligencia artificial, también conocida bajo las siglas de IA, es extremadamente amplio. Abarca temas desde el procesamiento de datos médicos (e.g. clasificación de células cancerígenas), predicción bursátil, modelos de edición de imagen y vídeo, traducción de idiomas, hasta complejos modelos de generación de maquetas tridimensionales a partir de imágenes de estructuras capturadas desde distintos ángulos.

El aprendizaje automático o *machine learning* es uno de los campos más importantes de la IA hoy en día. Dentro de éste, las redes neuronales se han convertido en la familia de algoritmos más populares de estos últimos años. Especialmente aquellas redes neuronales con muchas capas, a lo que se denomina como aprendizaje profundo (*deep learning*), un subcampo que ha conseguido revolucionar el aprendizaje automático en el ámbito de la visión artificial, el procesamiento del lenguaje natural y la conducción autónoma, entre otros.

Si nos centramos en el área de visión artificial, la gran mayoría de redes neuronales profundas necesitan de bases de datos de imágenes de gran tamaño para poder ser entrenadas. Por ese motivo, la generación de bases de datos para el entrenamiento de estos modelos lleva siendo un tema de gran importancia en el ámbito de la IA desde que Fei-fei Li, profesora de ciencias de la computación en la Universidad de Standford, comenzara a investigar sobre la expansión y la mejora de datos en 2006. Dicha investigación acabaría generando una de las bases de datos de imágenes más grandes de la historia: ImageNet.

Con más de 14 millones de imágenes etiquetadas, ImageNet es una de las bases de datos de imágenes más importantes en el ámbito del Deep Learning. Después de su creación, a partir de 2010, Fei-fei Li empezó a colaborar con el equipo de la ya existente *PASCAL VOC challenge*, una competición creada en 2005 cuyo propósito era permitir a los equipos de investigación competir entre ellos para sacar el mejor resultado en reconocimiento de imágenes. El resultado de dicha colaboración dio lugar al *ImageNet Large Scale Visual Recognition Challenge* (ILSVRC), una nueva versión de la competición *PASCAL VOC challenge* en la que se añadió una versión reducida de ImageNet con 1000 categorías distintas (Figura 1.1).

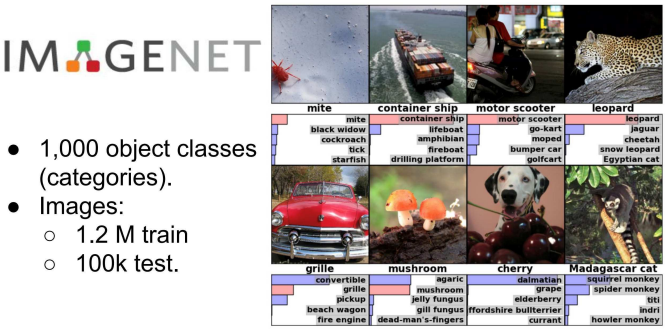


Figura 1.1: Competición ILSVRC.

Al rededor de 2011, el error en la ILSVRC de los 5 mejores modelos era del 25 %. Pero en 2012, un nuevo modelo de red convolucional llamado AlexNet consiguió alcanzar un error del 15 %, haciendo uso de unidades de procesado gráfico (GPU) durante el entrenamiento [3]. Esto dio paso a una nueva revolución en la inteligencia artificial que empezó a llamar la atención en el ámbito de la industria [4]. Como se puede ver en la Figura 1.2, el número de artículos relacionados con el aprendizaje automático, visión artificial y reconocimiento de patrones empezó a crecer de manera exponencial después de esto.

Gracias a competiciones de este tipo y a sus bases de datos, a lo largo de los años han ido surgiendo nuevos modelos de aprendizaje profundo como las Redes Neuronales Convolucionales, las Redes Neuronales Recurrentes, los Autocodificadores, los modelos de difusión o las famosas redes generativas adversarias conocidas como GAN (del inglés *Generative Adversarial Network*),

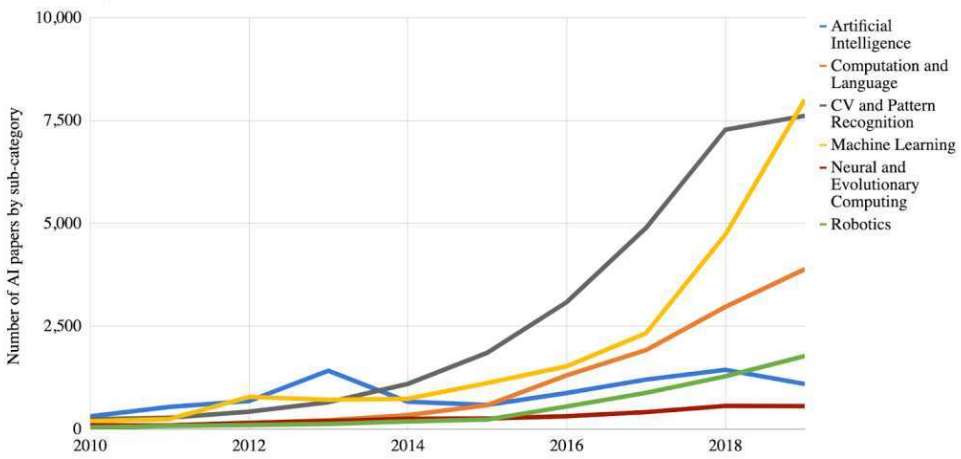


Figura 1.2: Número de artículos sobre IA en arXiv 2010-2019 [5].

un tipo de modelo generativo diseñado por Ian Goodfellow y sus compañeros de laboratorio en junio de 2014 [6]. Éste tipo de redes tienen la capacidad de generar datos sintéticos de todo tipo: imágenes de paisajes, imágenes de rostros, datos clínicos de pacientes no existentes, voces humanas para *Text To Speach* o TTS (conversión de texto a voz), arreglos musicales, etc. Aparte de generar nuevos datos, las GAN también pueden utilizarse para realizar otras tareas: aumentar la resolución de imágenes y vídeos (véase la Figura 1.3), generar *frames* de un vídeo y aumentar los *frames* por segundo de éste; restaurar una foto en blanco y negro pasándola a color (véase Figura 1.4), generar modelos tridimensionales de una escena a partir de imágenes tomadas desde distintos ángulos, etc. Como se puede observar, las posibilidades son múltiples.



Figura 1.3: *Supersampling* con modelo TecoGAN [7].



Figura 1.4: Recoloración con DeOldify [8].

Uno de los temas con más avances en los últimos años con respecto a las redes GAN es la generación de rostros humanos. El nivel de calidad de las imágenes generadas por modelos como

W-GAN [9], DC-GAN [10], Info-GAN [11], PRO-GAN [12] y otros, han ido mejorando a lo largo de los años. Se puede observar como, desde 2014 hasta 2017, en tan solo tres años se consiguieron grandes avances en este ámbito (véase la Figura 1.5). A día de hoy, las imágenes generadas por modelos como StyleGAN2 [1] y AliasFree-GAN [13] tienen una calidad sin precedentes (Figura 1.6), a costa de grandes tiempos de entrenamiento y una mayor necesidad de recursos que sus predecesores.



Figura 1.5: Evolución en las imágenes generadas por la IA (Brundage, et al., 2018) [14].



Figura 1.6: Rostros generados por StyleGAN2 [1].

Debido al gran potencial que estos modelos tienen a la hora de generar grandes cantidades de datos artificiales de alta fidelidad, sería interesante estudiar las posibles aplicaciones de modelos generativos como los presentados anteriormente con la intención de generar bases de datos para distintos temas de investigación relacionados con rostros faciales humanos.

Uno de los campos más estudiados en los últimos años es el reconocimiento de emociones en rostros humanos [15]. Como se ha explicado antes, los modelos de aprendizaje profundo que se desarrollan para este tipo de estudios necesitan de una gran cantidad de datos etiquetados para el entrenamiento supervisado de éstos. Obtener dichos datos puede ser un verdadero quebradero de cabeza en algunos casos debido a los posibles costes de recopilación de datos, generalmente elevados en cuanto tiempo y recursos, y a su consecuente manejo según normativas como la Regulación General de Protección de Datos Europea. Estos problemas de coste y protección de datos ralentizan en gran medida el desarrollo y la investigación. Por ese motivo, en este Trabajo Fin de Grado se plantea utilizar modelos generativos para la generación de datos para el reconocimiento facial de expresiones, que consiste en la clasificación y cuantificación de expresiones faciales humanas correspondientes a una emoción (“Enfado”, “Asco”, “Miedo”, “Felicidad”, “Neutral”, “Tristeza” y “Sorpresa”). Además, se considera interesante generar rostros de distintas edades, razas y géneros para tener así una base de datos balanceada e intentar evitar posibles problemas de sesgo en los datos.

1.1. Objetivos

El objetivo principal de este Trabajo Fin de Grado es el de estudiar distintas técnicas de generación de bases de datos de rostros faciales humanos etiquetados para su uso en proyectos de detección de emociones. Para ello se establecen varias fases en el desarrollo del trabajo:

1. Estudiar el arte en el ámbito de las redes generativas adversarias. Se investiga sobre la literatura de redes GAN, desde las primeras versiones hasta las versiones más actuales. Se estudian también distintas técnicas de condicionar la generación de estas.
2. Buscar bases de datos adecuadas para la generación de rostros y modificar éstas para su posterior utilización en los posibles modelos generativos condicionados.
3. Generar rostros mediante GANs. En esta fase se estudia la viabilidad de generar datos con las primeras versiones de redes generativas adversarias. Una vez generados, se evalúan los resultados en cuanto a calidad visual y velocidad de entrenamiento y generación de datos.
4. Generar rostros condicionados por edad, género, raza y expresión facial mediante arquitecturas de modelos generativos condicionados estudiados. Además, se evalúan los resultados mediante técnicas de medición de la calidad de modelos generativos.
5. Estudiar diferentes técnicas de generación de datos controlada mediante técnicas de entrenamiento diferentes a las GANs condicionadas.
6. Entrenar modelos de detección de emociones mediante los datos generados con las distintas técnicas propuestas y comparar los resultados con modelos entrenados con los datos reales, reflejando las ventajas y desventajas de cada método propuesto.

2. ANTECEDENTES

Para poder acometer los objetivos establecidos es necesario ofrecer primero un repaso sobre los temas clave entorno a este Trabajo Fin de Grado. Para ello, en los siguientes apartados se presentan algunos conceptos aprendidos a lo largo del grado así como información sobre nuevos temas sobre aprendizaje profundo o conceptos legales como el Reglamento General de Protección de Datos Europeo (RGPD).

2.1. Aprendizaje adicional

Los dos métodos que se proponen más adelante para la generación de bases de datos hacen uso de las redes GAN y su funcionamiento está fuertemente ligado a las redes neuronales convolucionales y las redes neuronales deconvolucionales. Dado que a lo largo del grado no se han abarcado estos temas, ha sido necesario un aprendizaje adicional a través de la Especialización en Aprendizaje Profundo y en la Especialización en Redes Generativas Adversarias de Coursera. Esto forma parte del Objetivo 1: *Estudio del arte en el ámbito de las redes generativas adversarias.*

2.1.1. Especialización en Aprendizaje Profundo

En esta especialización, se construyen y se entrenan arquitecturas de redes neuronales como las redes neuronales convolucionales, las redes neuronales recurrentes, las LSTM (*Long Short Term Memory*, utilizadas en el procesamiento del lenguaje natural) y los *transformers*. Además, se aprende a mejorar estos modelos con estrategias como el uso de *Dropout*, *Batch Normalization*, la inicialización Xavier, el optimizador Adam, etc. Para programar y entrenar estos modelos se utiliza Python y TensorFlow. A lo largo de las prácticas se abordan casos del mundo real como el reconocimiento del habla, la síntesis musical, los *chatbots*, la traducción automática, el procesamiento del lenguaje natural, la transferencia de estilos en imagen, el reconocimiento de imagen, y mucho más.

2.1.2. Especialización en Redes Generativas Adversarias

Esta especialización ofrece una introducción a la generación de imágenes con redes GAN, desde los conceptos fundamentales hasta las técnicas más avanzadas cubriendo las implicaciones sociales, incluyendo el sesgo en el aprendizaje automático y las formas de detectarlo, la preservación de la privacidad, y más. Se utiliza Python y PyTorch para entrenar los modelos que generan las imágenes y para evaluar algunas de las redes GAN mas avanzadas del mercado, como StyleGAN, con el método de la Distancia de Incepción de Fréchet (FID) que ofrece una cuantificación de la fidelidad y la diversidad de éstas.

2.2. Reglamento General de Protección de Datos (RGPD)

El RGPD es una normativa europea [16] que ofrece privacidad a la ciudadanía pero a su vez limita en gran medida el manejo de datos en proyectos como los de investigación y desarrollo de modelos de IA por tener el nivel más alto de protección. Para más información se pueden consultar los documentos oficiales de la AEPD: Requisitos para Auditorías de Tratamientos que incluyan IA, y Guía RGPD para Responsables de Tratamiento.

2.3. Historia de las Redes Neuronales

El modelo de red neuronal existe desde el año 1943 cuando Warren McCulloch (neurofisiólogo) y Walter Pitts (matemático), motivados por los estudios de Alan Turing sobre el cerebro humano como forma de ver el mundo computacional, desarrollaron la primera red neuronal a base de circuitos eléctricos. En el año 1957, Frank Rosenblatt empezó a desarrollar el primer modelo basado en las neuronas del cerebro humano conocido como Perceptrón [17] (Figura 2.1). Este modelo fue una revolución, ya que después de haber aprendido una serie de patrones podía reconocer otros similares los cuales nunca se habían visto durante el entrenamiento. Aún así, se enfrentaba a ciertas limitaciones, como por ejemplo su incapacidad de resolver problemas de clasificación de clases no separables linealmente, por ejemplo, la puerta XOR. En 1969, Marvin Minskey y Seymour Papert demostraron matemáticamente la debilidad del Perceptrón ante este tipo de problemas, lo que causó el repentino corte de la financiación en los proyectos de inteligencia artificial, dando lugar al conocido "invierno de la IA".

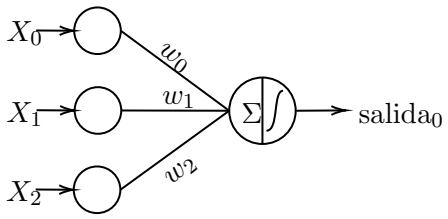
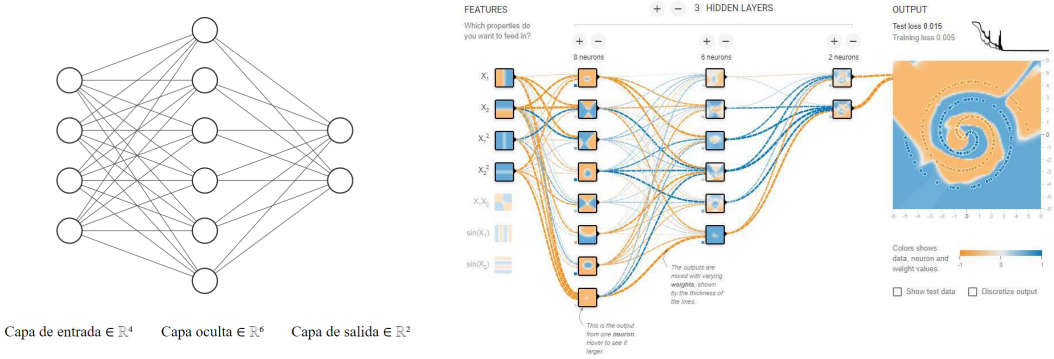


Figura 2.1: Perceptrón.

Para resolver los problemas no lineales, se desarrollaron redes más complejas conocidas como Perceptrón Multicapa, arquitecturas de redes con combinaciones de varias neuronas/perceptrones en distintas capas (véase la Figura 2.2a) que permitía ofrecer un conocimiento jerarquizado. Con estas redes, en teoría, se conseguiría resolver muchos más problemas (como el de la Figura 2.2b). El único inconveniente era el método de entrenamiento de estas, el método de Perturbación Aleatoria o mejor conocido como fuerza bruta, el cual era extremadamente ineficiente y muy costoso para la tecnología de la fecha. Pero todo cambió cuando en el año 1987 David E. Rumelhart, Geoffrey E. Hinton y Ronald J. Williams publicaron el artículo *"Learning representations by back-propagating errors"* [18] en el que utilizaban el algoritmo de *back-propagation* propuesto por Seppo Linnainmaa [19], un nuevo método de entrenamiento que necesitaba recorrer toda la red neuronal hacia atrás una sola vez para reajustar sus parámetros. Este método ayudó en gran medida al "renacer" de la IA (al menos en el campo de las redes neuronales), ya que a partir de ese momento fue posible entrenar redes neuronales multicapa, algo que hasta entonces era inviable.



(a) Diagrama de red neuronal (b) Red neuronal en Tensorflow Playground

Figura 2.2: Ejemplos de redes neuronales.

2.4. Elementos básicos de una red neuronal y su funcionamiento

Con la anterior introducción a la historia de las redes neuronales, se puede empezar a explicar los elementos de dichas redes y su funcionamiento. Para ello se explican los conceptos de neurona, función de activación, función de coste y algoritmo de retro-propagación o *"back-propagation"*.

2.4.1. La neurona

La neurona es la unidad básica de procesamiento de una red neuronal. Como se ha comentado anteriormente, el primer modelo basado en una neurona fue el Perceptrón [17]. Estas neuronas, imitando a las neuronas biológicas, tienen unas conexiones de entrada de datos o "estímulos" con los que calculan un valor de salida. Por lo tanto, una neurona no deja de ser una función matemática. La neurona utiliza todos los valores de entrada para calcular una suma ponderada de la forma $w_0x_0 + w_1x_1 + \cdots + w_nx_n$ que es igual a la función para calcular una regresión lineal (véase la Figura 2.3). Los valores w son llamados pesos y serán estos los que a lo largo del entrenamiento de la red irán modificando sus valores para adecuarse a los datos. Al igual que en modelos de regresión lineal, el valor w_0 actúa como parámetro de sesgo siendo $x_0 = 1$.

Viendo que una neurona no es más que un modelo de regresión lineal, el verdadero potencial de estas se encuentra en la combinación por capas de varias neuronas. Como podemos ver en la Figura 2.3, la estructura de una red se compone de una capa de entrada de datos, varias (o ninguna) capas ocultas y una capa de salida. Esta combinación se conoce como Perceptrón Multicapa o Red Neuronal. Uno de los problemas a la hora de jerarquizar dichas neuronas de esta manera es que la combinación de varias regresiones lineales da lugar a una única regresión lineal. Es decir, la estructura de una red neuronal sería equivalente a una sola neurona. Por lo tanto, para poder sacar el máximo potencial de estas redes, es necesario utilizar lo que se conoce como "función de activación" (a en la Figura 2.3). También podemos ver en dicha Figura (en la capa de salida) la función C, que representa a la "función de error" utilizada en el proceso de entrenamiento u optimización de la red, proceso que se explicará más adelante.

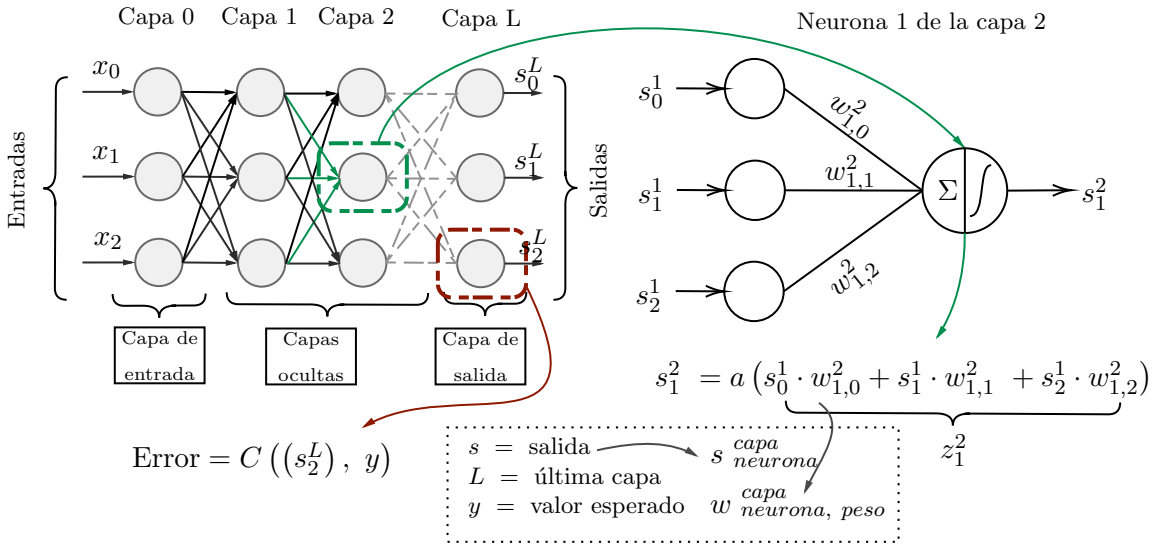


Figura 2.3: Estructura y componentes de una red neuronal.

2.4.2. Función de activación

El propósito de la función de activación es la de distorsionar la salida de la neurona añadiéndole deformaciones no lineales para que la combinación de varias neuronas sea efectiva y no sea equivalente al uso de una única neurona. Existen varias funciones de activación y cada una de ellas es útil para ciertos problemas. Las más conocidas son:

- Función escalonada: $f(x) = \begin{cases} 0 & si \quad x < 0 \\ 1 & si \quad x \geq 0 \end{cases}$
- Función sigmoidea: $f(x) = \sigma(x) = \dfrac{1}{1 + e^{-x}}$
- Función tangente hiperbólica: $f(x) = \dfrac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$
- Función RELU: $f(x) = \begin{cases} 0 & si \quad x < 0 \\ x & si \quad x \geq 0 \end{cases}$
- Función Leaky RELU: $f(x) = \begin{cases} \alpha \cdot x & si \quad x < 0 \\ x & si \quad x \geq 0 \end{cases}$

2.4.3. Función de coste

Al igual que otros modelos de aprendizaje automático, las redes neuronales intentan adecuarse a unos datos de entrada. Para ello utilizan algoritmos de optimización que intentan minimizar o maximizar el valor de una función objetivo. En las redes neuronales se intenta minimizar el error de estas, por lo tanto, se utilizan unas funciones llamadas funciones de error o funciones de coste. Estas funciones tienen un cometido importante, ya que deben condensar fielmente todos los aspectos del modelo en un único valor, de forma que las mejoras en ese valor sean representativos de un modelo mejor. Existen dos tipos principales de funciones de error: de regresión y de clasificación. Las primeras ofrecen un valor de error para modelos que predicen un valor real (precio, edad, ...) y las otras devuelven un valor entero correspondiente a una clase (tipo de vehículo, raza de perro, ...). Entre las más conocidas están:

- **Funciones de regresión:**
  - Error cuadrático medio (Error L2): $MSE = \dfrac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n}$
  - Error absoluto medio (Error L1): $MAE = \dfrac{\sum_{i=1}^{n}|y_i - \hat{y}_i|}{n}$
- **Funciones de clasificación:**
  - Entropía cruzada binaria: $BCE = -(y \cdot log(\hat{y}_i) + (1 - y) \cdot log(1 - \hat{y}_i))$
  - Entropía cruzada categórica: $CCE = -\sum_{i=1}^{C} y_i \cdot log(\hat{y}_i)$

En las anteriores fórmulas tenemos el valor de la salida de la red y_i, el valor esperado o real $\hat{y}_i$, el número de valores a evaluar n y el número de clases C.

2.4.4. Algoritmo de retro-propagación

El algoritmo de retro-propagación o *"back-propagation"* [18] utiliza el principio de cadena de responsabilidad para propagar el error de la salida hacia atrás, dando más importancia a aquellas neuronas que tienen un mayor impacto en la salida de la red. Para ello utiliza métodos como el descenso del gradiente, que utiliza las derivadas parciales de los pesos con respecto al coste para conseguir el gradiente y utilizar ese valor para optimizar la salida mediante un proceso iterativo denominado entrenamiento. Para calcular esas derivadas es necesario calcular las derivadas de las funciones de coste C, la función de activación a, la suma ponderada Z y los pesos w.

Primero se empieza por calcular las derivadas de la ultima capa L:

$$\frac{\partial C}{\partial w^L} = \frac{\partial C}{\partial a^L} \cdot \frac{\partial a^L}{\partial z^L} \cdot \frac{\partial z^L}{\partial w^L} \qquad Z^L = W^L \cdot a^{L-1} + b^L \qquad \text{Error capa } L = C(a(Z^L), y)$$

Podemos resumir las derivadas $\frac{\partial C}{\partial a^L} \cdot \frac{\partial a^L}{\partial z^L}$ como el error imputado en la neurona, es decir δ^L. Dado que la derivada de $\frac{\partial z^L}{\partial w^L}$ es el valor de la activación de la capa anterior a^{L-1}, al final, la derivada de los pesos respecto al coste en la última capa quedaría así:

$$\frac{\partial C}{\partial w^L} = \delta^L \cdot a^{L-1} \tag{2.1}$$

Para el resto de capas, gracias al principio de *back-propagation*, el cálculo de las derivadas parciales se resume en gran medida. Podemos generalizar para todas las capas anteriores calculando los valores de la penúltima capa $L-1$:

$$\frac{\partial C}{\partial w^{L-1}} = \frac{\partial C}{\partial a^L} \cdot \frac{\partial a^L}{\partial z^L} \cdot \frac{\partial z^L}{\partial a^{L-1}} \cdot \frac{\partial a^{L-1}}{\partial z^{L-1}} \cdot \frac{\partial z^{L-1}}{\partial w^{L-1}} = \delta^L \cdot W^L \cdot \frac{\partial a^{L-1}}{\partial z^{L-1}} \cdot a^{L-2} \tag{2.2}$$

Al igual que en la capa L, podemos resumir el error de la neurona como $\frac{\partial C}{\partial z^{L-1}} = \delta^{L-1}$. Con esto podemos generalizarlo a todas las capas de la siguiente manera:

$$\frac{\partial C}{\partial w^{l-1}} = \delta^l \cdot W^l \cdot \frac{\partial a^{l-1}}{\partial z^{l-1}} \cdot a^{l-2} = \delta^{l-1} \cdot a^{l-2} \tag{2.3}$$

Con estas derivadas calculadas, en cada iteración del entrenamiento de la red se propaga el error hacia atrás (véase Figura 2.5) actualizando así todos los pesos.

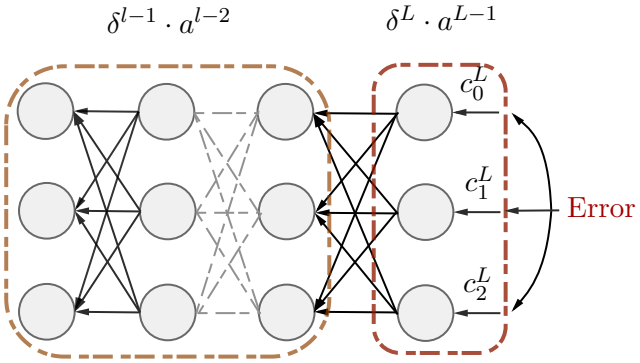


Figura 2.5: Diagrama de red neuronal con las derivadas para *back-propagation*.

2.5. Aprendizaje Profundo

El aprendizaje profundo, del inglés *Deep Learning*, es un subconjunto del aprendizaje automático, concretamente de las redes neuronales, que consiste esencialmente en una red neuronal con tres o más capas ocultas. Estos modelos intentan simular el comportamiento del cerebro humano (aunque están lejos de igualar su capacidad) permitiéndoles "aprender" a través de grandes cantidades de datos. Aunque una red neuronal con una sola capa puede hacer predicciones aproximadas, las capas ocultas adicionales pueden ayudar a optimizar y refinar la precisión ayudando a su vez a eliminar parte del preprocesamiento de los datos que suele implicar el aprendizaje automático. Estos algoritmos pueden procesar datos no estructurados, como texto e imágenes, y automatizan la extracción de características como puede verse en la Figura 2.6, eliminando parte de la dependencia de expertos humanos a la hora de preprocesar los datos.

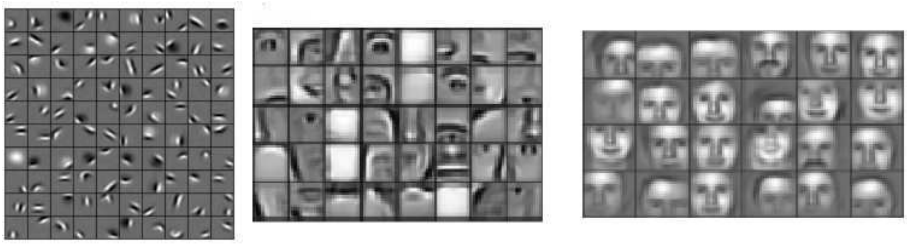


Figura 2.6: Características jerárquicas aprendidas de un modelo de aprendizaje profundo.

El aprendizaje profundo impulsa muchas aplicaciones y servicios basados en IA que mejoran la automatización, realizando tareas analíticas y físicas sin intervención humana. Los modelos de aprendizaje profundo se utilizan en productos y servicios del día a día (como los asistentes digitales, los mandos de televisión con voz y la detección de fraudes bancarios), así como en tecnologías más avanzadas (como los vehículos de conducción autónoma).

En este área se utilizan varias técnicas de optimización para mejorar los modelos. Entre ellos se encuentran la normalización por lotes (*Batch Normalization*), la técnica del abandono (*Dropout*) o el optimizador Adam. Pero antes de explicar estas técnicas, se debe dejar claro qué diferencia hay entre el entrenamiento, la validación y el testeo de un modelo.

2.5.1. Entrenamiento, validación y testeo

En el entrenamiento de modelos como las redes neuronales, es importante establecer tres conjuntos de datos: el de entrenamiento, el de validación y el de testeo. En el entrenamiento se utiliza una partición de los datos de entrada (normalmente de un 70-80 % del tamaño original) utilizando el algoritmo de *"back-propagation"* explicado en el Apartado 2.4.4. Varias de las técnicas que intentan mejorar el entrenamiento de las redes neuronales (como las que se explican más adelante) tienen varios atributos denominados híper-parámetros que, dependiendo de sus valores, pueden mejorar su rendimiento. Para poder comparar entre los distintos valores de dichos híper-parámetros, se utiliza el conjunto de validación, el cual utiliza una pequeña cantidad de datos no utilizados durante el entrenamiento (normalmente de un 10-15 % del tamaño original) para evaluar el modelo con cada configuración. Una vez se ha elegido la configuración de híper-parámetros que mejor resultado ofrece, se da paso a la fase de testeo. En este última fase se

evalúa otra partición de datos ajenos al entrenamiento (normalmente de un 10-15 % del tamaño original) para obtener el rendimiento real de la red.

2.5.2. Batch Normalization

La normalización por lotes (también conocida como *Batch Normalization* o *BatchNorm*) es un método utilizado para que el entrenamiento de las redes neuronales artificiales sea más rápido y estable mediante la normalización de las entradas de las capas por medio del recentrado y el reescalado. Fue propuesto por Sergey Ioffe y Christian Szegedy en 2015 [20].

Aunque todavía se discute sobre las razones de su efectividad, algunos piensan que ayuda mitigar el problema del desplazamiento de la covarianza interna, un fenómeno observado en experimentos que produce cambios a las capas internas en las medias y varianzas de las entradas durante el entrenamiento. De lo que no se discute es de sus ventajas, ya que permite que el modelo sea menos sensible a la inicialización de pesos y reduce el problema del desvanecimiento del gradiente o *gradient vanishing*, un problema en el que el entrenamiento de la red se estanca y no aprende más pues el gradiente que se utiliza para actualizar los pesos es muy pequeño. Se podría decir también que este tipo de normalización actúa como una especie de regularización ya que, al reescalar cada pequeño lote con su media y su varianza, se introduce algo de ruido.

El proceso de normalización se utiliza únicamente en el entrenamiento de la red y se describe en el Algoritmo 1 (basado en [20]):

Algoritmo 1 Normalización por lotes

Entrada: Valores de x en un pequeño lote $B = \{x_{1...m}\}$;
Parámetros a aprender: γ, β

Salida: $\{y = BN_{\gamma,\beta}(x_i)\}$

$\mu_B \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i$ ▷ media del mini-lote

$\sigma_B^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$ ▷ varianza del mini-lote

$\widehat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$ ▷ normalizar

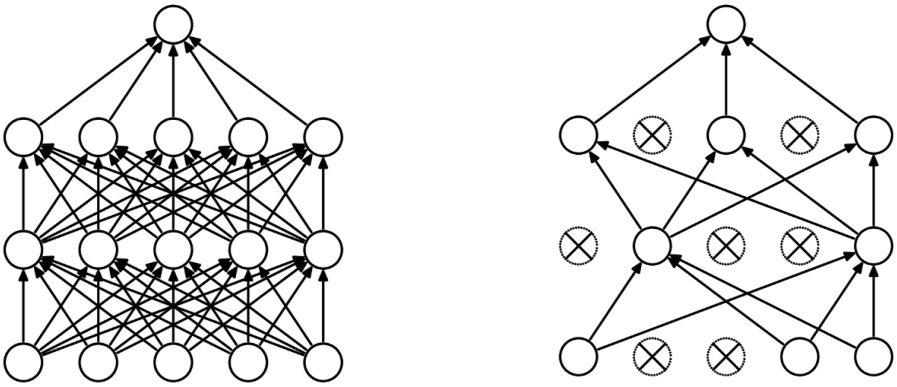
$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv BN_{\gamma,\beta}(x_i)$ ▷ escalar y centrar

2.5.3. Dropout

Las ventajas de las redes neuronales profundas a la hora de obtener una mejora en la exactitud de las predicciones, se vieron fuertemente afectadas por uno de los problemas más comunes del aprendizaje automático, el sobreajuste. Cuando un modelo se sobreajusta, decimos que se adecúa demasiado a los datos de entrenamiento, perdiendo parte de la generalización que le permite obtener buenos resultados en las conjuntos de datos que no se han usado en el entrenamiento. Por ese motivo, en el año 2012 Geoffrey E. Hinton y sus compañeros de laboratorio tuvieron la idea de excluir aleatoriamente subconjuntos de características durante el entrenamiento [21] para crear un efecto de regularización. A esta técnica la llamaron "abandono" o en inglés *"dropout"*.

El funcionamiento de esta técnica es bastante sencillo: cada una de las neuronas de la red tiene

una probabilidad p (normalmente entre el 20-50 %) de ser seleccionadas para "apagarse" (véase la Figura 2.7b). Es decir, si una neurona es seleccionada, su salida no se tiene en cuenta en la propagación hacia delante y tampoco se actualizan sus pesos en la retro-propagación de esa iteración.



(a) Diagrama de red neuronal (b) Después de aplicar *dropout*

Figura 2.7: Diagramas de redes neuronales con y sin *dropout*.

Como bien se explica en el Apartado 2.5.1, este tipo de técnicas únicamente se utilizan durante el entrenamiento del modelo. Por lo tanto, a la hora de evaluar el modelo en el proceso de testeo, todas las neuronas están activadas. En cuanto al proceso de validación, se evalúa el modelo al igual que en el proceso de testeo. Es decir, todas las neuronas están activadas.

A pesar de que el *dropout* ha terminado siendo una técnica excepcionalmente exitosa, las razones de su éxito aún no se comprenden bien a nivel teórico.

2.5.4. Media Móvil Ponderada Exponencial (MMPE)

En términos sencillos, una "media móvil" es una herramienta estadística para determinar la dirección de una tendencia y, para ello, agrega un subconjunto de puntos de datos, normalmente durante un periodo de tiempo específico, y divide el total por el número de puntos de datos del subconjunto para obtener una media. Se denomina media móvil porque este cálculo se realiza de forma recursiva a medida que los puntos de datos crecen con el tiempo.

La media móvil exponencial es un tipo de media móvil que aplica más peso a los puntos de datos más recientes que a los ocurridos en el pasado. En otras palabras, es como dar más importancia a la última experiencia o recuerdos que a los más antiguos, suponiendo que éstos estén representados por puntos de datos.

La siguiente ecuación representa la fórmula para evaluar la media móvil ponderada exponencial:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \theta_t \tag{2.4}$$

En la ecuación anterior, el valor de θ_t representa el valor observado en la instancia t. Podemos decir entonces que v_t corresponde al valor aproximado de la media de $\frac{1}{1-\beta}$ instancias. Por ejemplo, dando un valor de $\beta = 0{,}98$, v_t tendría la media aproximada de 50 instancias. Si reducimos el valor a $\beta = 0{,}5$, v_t pasaría a tener la media aproximada de 2 instancias, haciendo que la media se adapte mucho más rápido a los cambios en los datos.

2.5.5. Optimizador Adam

Adam es un algoritmo de optimización que puede utilizarse en lugar del clásico descenso de gradiente estocástico para actualizar los pesos de la red de forma iterativa basándose en los datos de entrenamiento. Fue presentado por Diederik Kingma, de OpenAI, y Jimmy Ba, de la Universidad de Toronto en 2014 [22].

Adam es diferente al descenso de gradiente estocástico clásico ya que este último mantiene un único ratio de aprendizaje α para todas las actualizaciones de pesos y este no cambia durante el entrenamiento. Adam también utiliza un principio similar al momento, técnica que utiliza la información de gradientes pasados para calcular el trayecto a seguir. Los autores describen a Adam como una combinación de otros dos optimizadores basados en el descenso de gradiente estocástico:

- Algoritmo de Gradiente Adaptativo (AdaGrad) que mantiene un ratio de aprendizaje por parámetro que mejora el rendimiento en problemas con gradientes dispersos (por ejemplo, problemas de lenguaje natural y visión artificial).
- Propagación de la raíz cuadrada (RMSProp) que también mantiene ratios de aprendizaje por parámetro que se adaptan en función de la media de las magnitudes recientes de los gradientes para cada peso (e.g. la rapidez con la que cambia). Esto significa que el algoritmo funciona bien en problemas no estacionarios.

Podemos ver como funciona el optimizador en el Algoritmo 2, basado en [22]:

Algoritmo 2 Optimizador Adam

Entrada: α : Ratio de aprendizaje
$\beta_1, \beta_2 \in [0, 1)$: Índices de decrecimiento exponencial del momento
$f(\theta)$: Función estocástica objetivo con parámetros θ
θ_0 : Vector de parámetros inicial

Salida: θ_t : Parámetros actualizados

$m_0 \leftarrow 0$ ▷ Inicializar primer vector de momento

$v_0 \leftarrow 0$ ▷ Inicializar segundo vector de momento

$t \leftarrow 0$ ▷ Inicializar iterador

mientras θ_t no converja **hacer**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ ▷ Obtener gradientes de la iteración t

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta 1) \cdot g_t$ ▷ Actualizar primer momento estimado

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta 2) \cdot g_t^2$ ▷ Actualizar segundo momento estimado

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ ▷ Corregir sesgo del primer momento estimado

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ ▷ Corregir sesgo del segundo momento estimado

$\theta_t \leftarrow \theta_{t-1} - \alpha_t \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ ▷ Actualizar parámetros

fin mientras

devolver θ_t ▷ Devolver parámetros actualizados

2.5.6. Optimizador DiffGrad

DiffGrad es un nuevo optimizador introducido en el artículo *"diffGrad: An optimizer for CNN's"* de Dubey en 2019 [23]. Se basa en el optimizador Adam, de probada eficacia, mediante el desarrollo de una "sujeción por fricción adaptativa" y la monitorización del cambio local de los gradientes con el fin de fijar automáticamente los valores óptimos de los parámetros que Adam puede saltarse. DiffGrad monitoriza el cambio inmediato del gradiente actual frente al paso anterior, y aplica esa "sujeción por fricción adaptativa" que puede decelerar rápidamente cuando el cambio de gradiente es pequeño, y por tanto implica que una solución óptima puede estar cerca.

En el trabajo propuesto [23] se introduce un coeficiente de fricción *diffGrad* (DFC) para controlar la tasa de aprendizaje utilizando información del comportamiento del gradiente a corto plazo. El DFC se representa por ξ y se define como:

$$\xi_{t,i} = AbsSig(\Delta g_{t,i}) \tag{2.5}$$

donde $AbsSig$ es una función sigmoidea no lineal que acota cada valor entre 0.5 y 1, y se define como:

$$AbsSig(x) = \frac{1}{1 + e^{-|x|}} \tag{2.6}$$

mientras que $\Delta g_{t,i}$ es el cambio de gradiente entre las iteraciones inmediatamente anteriores y las actuales, dado como:

$$\Delta g_{t,i} = g_{t-1,i} - g_{t,i} \tag{2.7}$$

donde $g_{t,i}$ es el gradiente calculado para el parámetro i en la iteración t.

El método de optimización de los parámetros es prácticamente el mismo que el de Adam con la diferencia de que los parámetros se actualizan de la siguiente manera:

$$\theta_{t+1,i} = \theta_{t,i} - \alpha_t \cdot \frac{\xi_{t,i} \cdot \hat{m}_{t,i}}{\sqrt{\hat{v}_{t,i}} + \epsilon} \tag{2.8}$$

En la Figura 2.8 podemos observar las diferencias en los valores obtenidos con Adam y con DiffGrad.

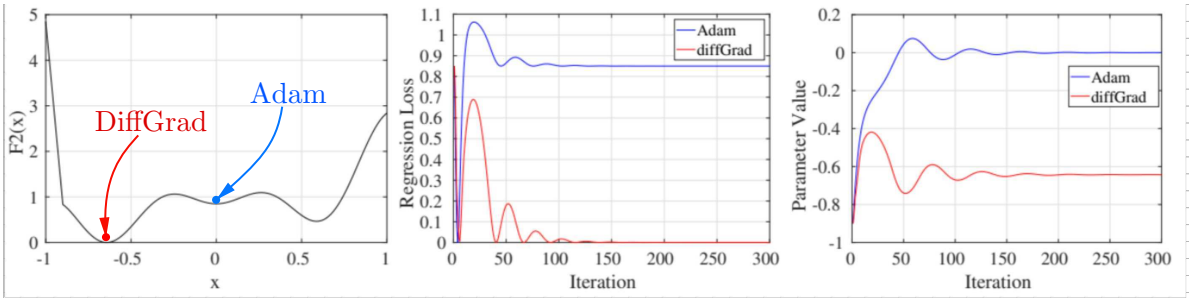
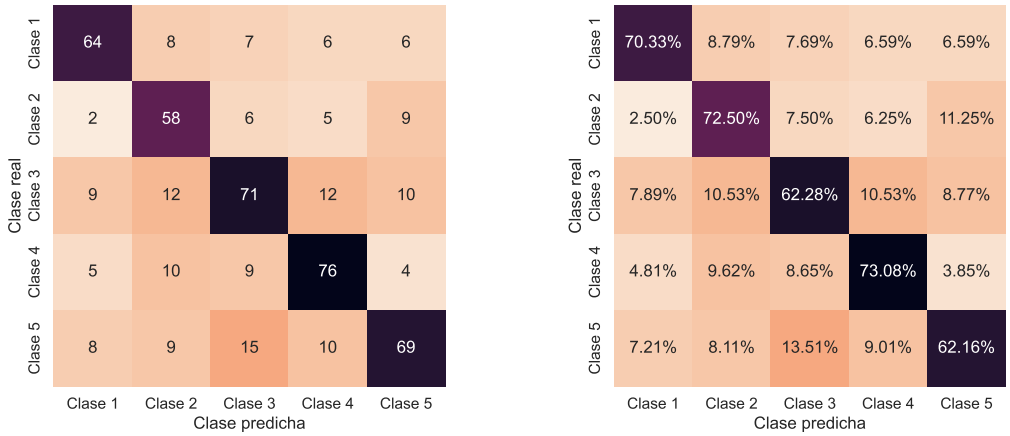


Figura 2.8: Diferencias entre los optimizadores Adam y DiffGrad [23].

2.6. Métricas de clasificación

Habiendo entrenado modelos de clasificación, y durante el propio entrenamiento, se utilizan ciertas métricas para monitorizar la eficiencia de éstos. Todas ellas están basadas en números de la matriz de confusión del modelo. Esta matriz es un resumen de la clasificación realizada por el modelo para cada ejemplo, en base a la clase a la que pertenece. En la Figura 2.9 se puede observar un ejemplo de esta matriz (en 2.9a las ocurrencias en números enteros y en 2.9b los porcentajes por número de ejemplos por clase real). Cada fila y columna está asociada a una clase, donde cada fila hace referencia a la clase real de la instancia y cada columna a la clase predicha por el modelo para esa instancia.



(a) Números enteros

(b) Porcentajes por fila

Figura 2.9: Matriz de confusión.

Con estos valores de la matriz de confusión se pueden calcular las métricas de *accuracy*, *precision*, *recall* y F_1 *score*. La primera calcula el número de instancias clasificadas correctamente entre el número total de instancias. La *precision* devuelve la fracción de cuantas de las instancias predichas como una clase son realmente de esa clase. El *recall* calcula la fracción de cuantas de las instancias que son realmente de una clase son predichas como esa clase. Finalmente, el valor de F_1 *score* devuelve la media armónica de la *precision* y el *recall* para combinar ambos valores en uno solo. Para obtener los valores de dichas métricas para todas las clases, se puede calcular con la media o con la media ponderada por el número de ejemplos por clase. Para la matriz de la Figura 2.9, en la Tabla 2.1 se tienen los valores calculados para cada clase y las medias comentadas antes.

	Precisión	**Recall**	**F_1**	**Ejemplos**
Clase 1	0.7273	0.7033	0.7151	91
Clase 2	0.5979	0.7250	0.6554	80
Clase 3	0.6574	0.6228	0.6396	114
Clase 4	0.6972	0.7308	0.7136	104
Clase 5	0.7041	0.6216	0.6603	111
Media	0.6768	0.6807	0.6768	500
Media ponderada	0.6793	0.6760	0.6759	500
Accuracy				0.6760

Tabla 2.1: Métricas obtenidas mediante la matriz de confusión de la Figura 2.9.

2.7. Redes Neuronales Convolucionales

Una red neuronal convolucional (ConvNet o CNN) es un algoritmo de aprendizaje profundo que puede tomar una imagen de entrada, asignar importancia (pesos y sesgos entrenables) a varios aspectos u objetos de la imagen y ser capaz de diferenciar unos de otros. La arquitectura de una CNN intenta replicar el patrón de conectividad de las neuronas en el cerebro humano y se inspira en la organización de la corteza visual (véase Figura 2.10).

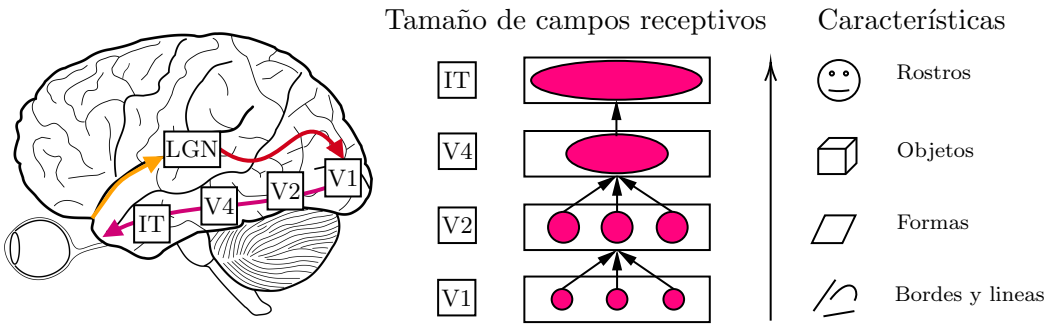


Figura 2.10: Diagrama de características de la jerarquía visual del cerebro humano.

Las CNNs, a diferencia de las redes neuronales convencionales, no dependen del tamaño de entrada de la imagen pues usan filtros de convolución como pesos. Una red neuronal convencional necesitaría, en la primera capa, *filas* × *columnas* × *canales* pesos para cada neurona. Es obvio que cuando hablamos de imágenes de gran tamaño, como imágenes a color de 1024×1024 píxeles, el número de parámetros a optimizar para cada neurona en la primera capa sería 3 145 728. Pero en las CNNs, el número de parámetros en cada neurona vendría asociado al número de filtros y su tamaño: 64 *filtros* × 3 *filas* × 3 *columnas* = 572 *parámetros*. Por lo tanto, dado que mediante los filtros utilizados el modelo CNN es capaz de capturar la información espacial de la imagen con muchos menos parámetros, es evidente que usar este tipo de redes es más eficiente.

Las redes CNN han demostrado ser muy útiles para varios tipos de problemas como clasificación de imágenes (AlexNet [3] es un buen ejemplo de ello), detección de objetos (donde YOLO [24] ha conseguido muy buenos resultados), etc. En la Figura 2.11 se puede ver la arquitectura de una de estas redes CNN, concretamente la arquitectura VGG-16 [25].

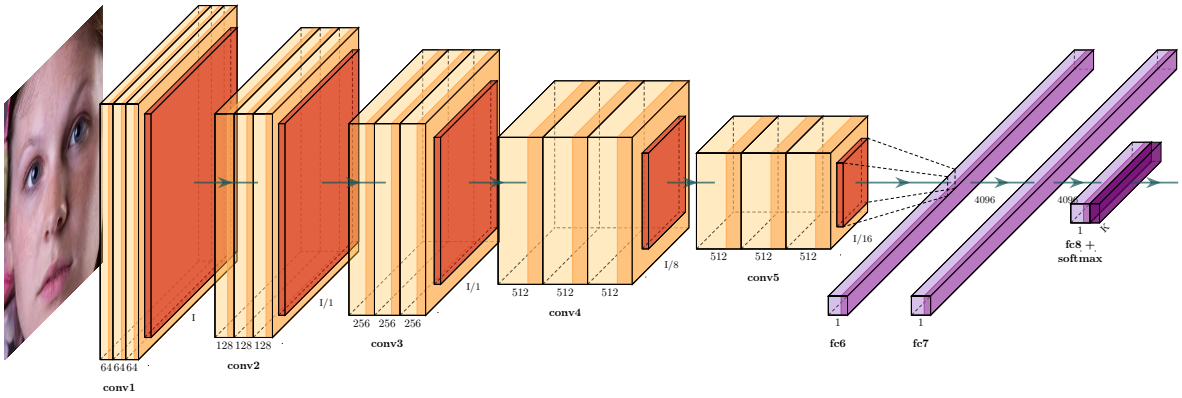


Figura 2.11: Diagrama de una Red Neuronal Convolucional VGG-16 [25].

2.7.1. Convolución

Antes se han mencionado los filtros que las CNN utilizan para procesar la información en cada neurona. Estos filtros son matrices de tamaño $N \times N$ (normalmente de tamaño impar) que se utilizan para aplicar convoluciones a la imagen. Una convolución es una operación matemática entre dos funciones f y g que produce una tercera función $f*g$. En el ámbito de la visión artificial, la operación de convolución se define como el proceso de aplicar un filtro a una imagen mediante unas secuencia de multiplicaciones matriciales con un algoritmo de ventana deslizante utilizando un desplazamiento configurable, que da como resultado otra imagen de menor o mismo tamaño. En la Figura 2.12 se puede ver como se aplican.

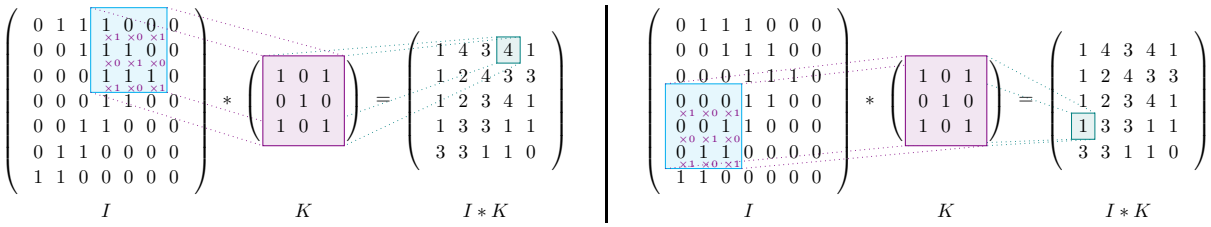
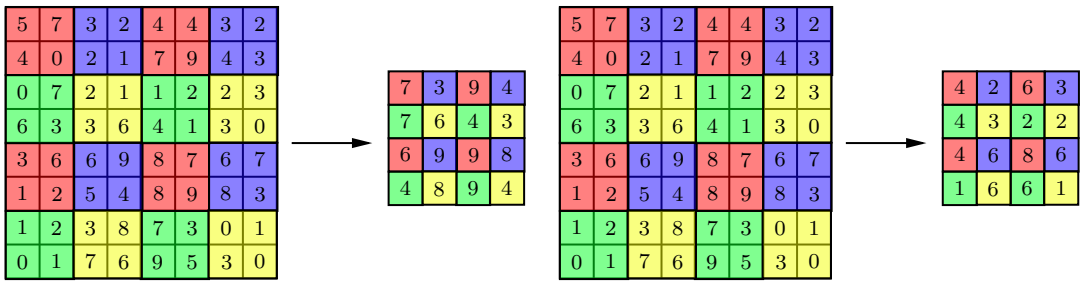


Figura 2.12: Ejemplo de convolución sobre una imagen.

Mediante el uso de estas convoluciones, cada neurona aprende una serie de filtros que combinados con los filtros de otras neuronas consiguen reconocer patrones que pueden servir a las capas posteriores. Es de esta manera, con una estructura jerarquizada, que la red es capaz de reconocer patrones complejos.

2.7.2. Pooling

La operación de agrupación o *pooling* consiste en aplicar un filtro con un desplazamiento del mismo tamaño que el filtro sobre cada canal del mapa de características extraído en una de las capas convolucionales de la red y de cierta manera resumir esas características manteniendo la mayoría de la información. Con esta operación, se reduce la dimensión de las características extraídas, disminuyendo así el número de parámetros a aprender. Existen varios tipos de *pooling*, pero los más conocidos son el *max pooling* y el *average pooling*. Como sus nombres indican, el *max pooling* escoge el valor más alto de entre los píxeles de la imagen (véase Figura 2.13a) y el *average pooling* obtiene la media de todos los píxeles (véase Figura 2.13b).



(a) *Max-Pooling* (b) *Average-Pooling*

Figura 2.13: Distintas técnicas de *pooling*.

2.7.3. Redes Neuronales Deconvolucionales

Las Redes Neuronales Deconvolucionales o Redes Neuronales Convolucionales Transpuestas son una variación de las CNN. En este caso, en vez de partir de una imagen y reducir su tamaño mediante convoluciones, las redes deconvolucionales aumentan el tamaño de la imagen en cada capa. Para hacer esto se usa lo que se conoce como una convolución transpuesta. Las convoluciones transpuestas se definen como el proceso de ampliar una imagen separando sus valores y aplicar un filtro para reducir esa ampliación de la imagen a un tamaño mayor que la que tenía al principio. Por ejemplo, si se tiene una imagen de 3×3, se amplía la imagen a 6×6 rellenando con valores (e.g 0) al rededor del píxel y se aplica una operación de convolución con un filtro de 3×3 (cuyos valores serán modificados a lo largo del entrenamiento de la red) para acabar con una imagen de 4×4. Se puede ver este ejemplo en la Figura 2.14.

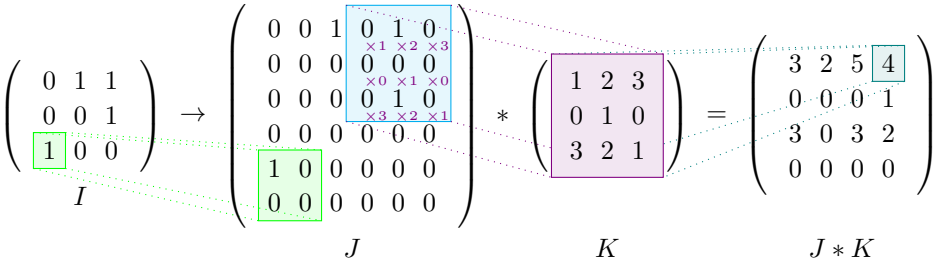


Figura 2.14: Ejemplo de convolución transpuesta sobre una imagen.

Las redes deconvolucionales han ido ganando mucha fama en los últimos años dado que se han descubierto múltiples aplicaciones de estas en arquitecturas como en los Autocodificadores Variacionales [26] o en arquitecturas U-Net [27] con las que se pueden resolver múltiples problemas como segmentación de imágenes, en el que la información de las capas de reducción de dimensionalidad se conectan con las capas de aumento (véase la Figura 2.15), dando así información espacial a las capas encargadas de reestructurar la imagen segmentada. Otro de los ámbitos de uso de estas redes es en el área de las famosas Redes Generativas Adversarias, modelos que en los últimos años han sido de gran interés en la comunidad científica del aprendizaje automático.

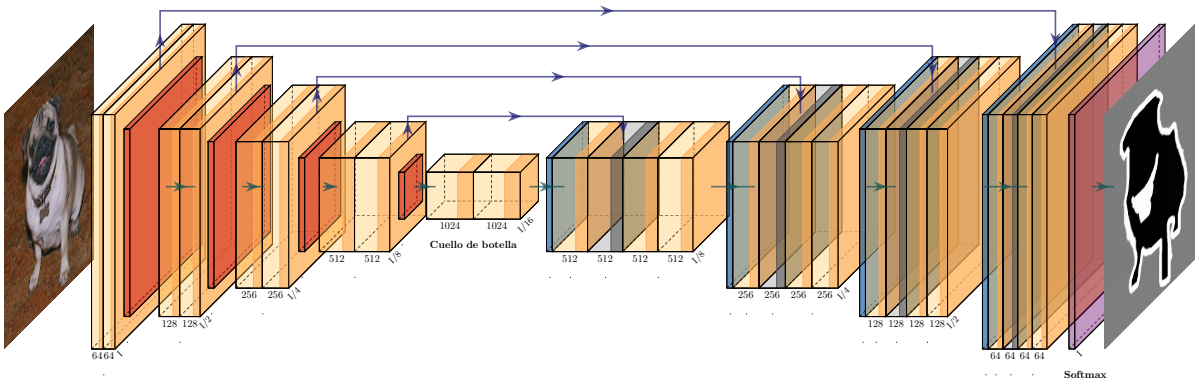


Figura 2.15: Arquitectura U-Net para segmentación de imagen.

3. REDES GENERATIVAS ADVERSARIAS

Como se ha explicado en el Capítulo de Introducción 1, las Redes Generativas Adversarias (GAN) son modelos de aprendizaje profundo capaces de generar datos sintéticos basados en la distribución de datos de entrenamiento. Es decir, pueden generar puntos nuevos en el espacio latente de la distribución original. Fueron presentadas por Ian Goodfellow en 2014 [28] y desde entonces han ido evolucionando de manera muy rápida. Al principio, se usaron redes neuronales profundas para esta arquitectura, pero una vez se vio la viabilidad de utilizar redes CNN con las GPUs [3], se añadieron a la arquitectura de estas redes GAN (véase Figura 3.1) y gracias a ello la calidad de los resultados obtenidos aumentaron considerablemente. Después de eso, la cantidad de nuevos modelos que han ido surgiendo no ha dejado de crecer. Grandes investigadores como Yann Lecun, Investigador jefe de IA en Facebook y profesor en el Instituto Courant de la Universidad de Nueva York, han manifestado que esta ha sido la idea más interesante en el ámbito del aprendizaje automático en los últimos diez años.

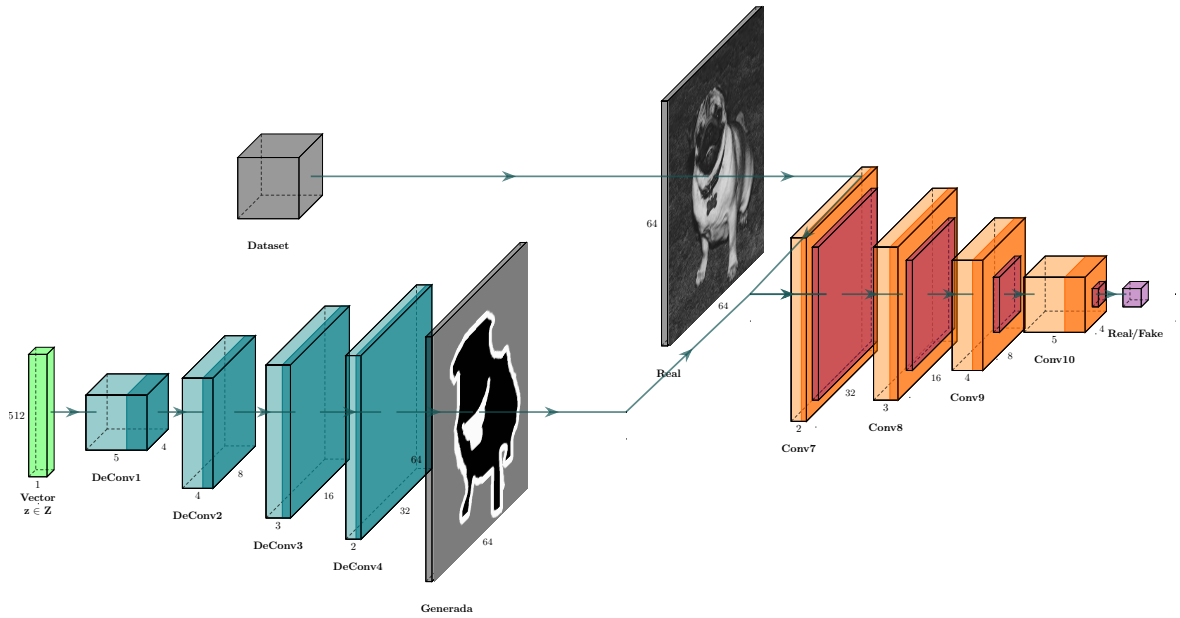


Figura 3.1: Arquitectura de una red GAN.

El principio del entrenamiento de las redes GAN se basa en el entrenamiento dinámico y conjunto de un modelo generador y un modelo discriminador. Explicado de otra manera, imagínese por un momento que se tiene a un falsificador de arte (generador) y a un autentificador encargado de detectar falsificaciones (discriminador). El falsificador será el encargado de generar los cuadros falsos, intentando crear obras que puedan engañar al detector de falsificaciones. Este último tiene acceso a los datos reales de entrenamiento y es con esos datos y los datos generados por el falsificador que podrá ir entrenando y aprender si la entrada que recibe es real o falsa. En cada iteración del entrenamiento, el discriminador toma los datos generados por el generador, los cuales se marcan con la clase "falso", y los datos de entrenamiento, que se marcan con la clase "real", y actualiza sus pesos con el error obtenido. Una vez el discriminador a actualizado sus pesos, el generador vuelve a generar nuevos datos, pero esta vez se marcan con la clase "real". Entonces se calcula el coste en el discriminador y se propaga el error (sin actualizar los pesos del discriminador) al generador. Es con ese error que el generador actualizará sus pesos, mejorando la calidad de los datos que genera para poder engañar al discriminador. Todo este proceso se

puede ver en el Algoritmo 3, basado en [28].

Algoritmo 3 Entrenamiento de redes generativas adversarias (GAN) por mini-lotes mediante descenso de gradiente estocástico. k es un híper-parámetro que representa le número de series de datos a aplicar al discriminador

para número de iteraciones del entrenamiento **hacer**
  para k iteraciones **hacer**
  - Obtener mini-lote de vectores latentes aleatorios $\{z^{(1)}, \ldots, z^{(m)}\} \subseteq p_g(z)$.
  - Obtener mini-lote de ejemplos del *dataset* $\{x^{(1)}, \ldots, x^{(m)}\} \subseteq p_{datos}(x)$.
  - Actualizar el discriminador maximizando el gradiente estocástico:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} [\log d(x^{(i)}) + \log(1 - d(g(z^{(i)})))]. \tag{3.1}$$

  fin para
  - Obtener mini-lote de vectores latentes aleatorios $\{z^{(1)}, \ldots, z^{(m)}\} \subseteq p_g(z)$.
  - Actualizar el generador minimizando el gradiente estocástico:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} [\log(1 - d(g(z^{(i)})))]. \tag{3.2}$$

fin para

Con esta técnica se consiguen resultados que hasta ahora no se habían conseguido. Antes de las redes GAN se utilizaban Autocodificadores Variacionales, basados en las redes U-Net [27]. Pero como se puede observar en la Figura 3.2, los resultados conseguidos con éstas dejaban bastante que desear en cuanto a detalles en las imágenes en comparación con los que se pueden obtener con las redes GAN.



(a) Resultados VAE (b) Resultados GAN

Figura 3.2: Diferencias entre los VAE y las redes GAN.

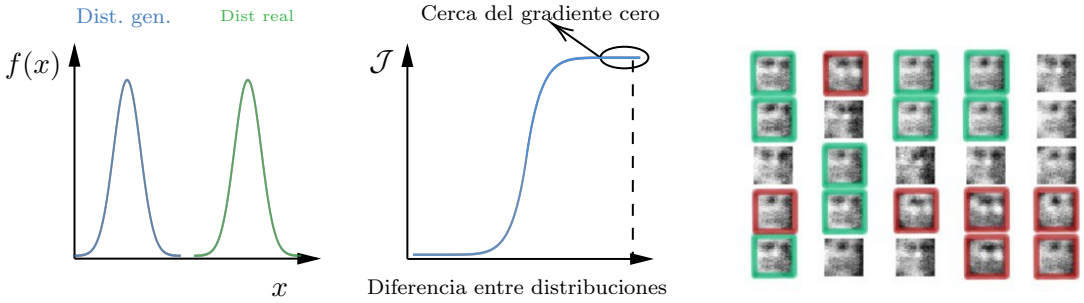
Los siguientes apartados tratan de explicar algunos de los conceptos aprendidos en el estudio del estado del arte de las redes GAN. Se explican conceptos como las redes W-GAN [9], la Penalización del Gradiente y su uso en las redes W-GAN, la Distancia de Incepción de Fréchet [29] para evaluar modelos GAN, la Normalización de Instancias Adaptativa [30] utilizada por modelos como StyleGAN [31], entre otros.

Dado que el objetivo del proyecto es obtener rostros etiquetados con emociones y otras variables demográficas mediante redes GAN condicionales, en los últimos apartados se explica cómo funcionan. Además se ofrece información sobre el modelo StyleGAN2 [1] y sus predecesores (Pro-GAN [12] y StyleGAN [31]), pues se utilizará este modelo de manera condicionada para generar datos de rostros faciales condicionados por “Edad”, “Raza”, “Género” y “Expresión”.

3.1. W-GAN

Uno de los problemas más comunes en los entrenamientos de las redes generativas es que cuando se utiliza el error BCE, la mayoría de las veces se da el conocido "desvanecimiento del gradiente". Esto ocurre ya que el discriminador tiende a superar al generador con bastante diferencia, haciendo que el modelo GAN deje de actualizarse por culpa del "desvanecimiento del gradiente" (véase Figura 3.3a).

Otro de los problemas más comunes es el "colapso de moda". Esto ocurre cuando el generador se queda atascado en uno de los mínimos locales de la distribución que intenta imitar, haciendo que los resultados que genera sean casi siempre los mismos. Aunque la calidad pueda ser muy buena, en cuanto a variación en los datos se refiere ofrece muy poco. Se denomina "colapso de moda" porque la distribución de los datos puede tener varias modas distintas y el generador se queda "atascado" en una de ellas. En la Figura 3.3b se puede ver las similitud entre imágenes con bordes del mismo color.



(a) Desvanecimiento del gradiente

(b) Colapso de moda

Figura 3.3: Problemas comunes en el entrenamiento de redes GAN.

Para solventar estos problemas, en el año 2017, Martin Arjovsky presentó el modelo de red Wasserstein-GAN o W-GAN [9]. Este modelo de red generativa se basa en la distancia de Wasserstein-1 para entrenar a la red en vez del error BCE. La distancia de Wasserstein-1 o *Earth Mover* se puede entender de manera "intuitiva" como la cantidad de "masa" que hay que transportar para transformar una distribución $\mathbb{P}_r$ a otra distribución $\mathbb{P}_g$. En la siguiente fórmula, $\Pi(\mathbb{P}_r, \mathbb{P}_g)$ denota la serie de distribuciones conjuntas $\gamma(x, y)$:

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y)\sim\gamma}[\| x - y \|] \tag{3.3}$$

Como el ínfimo es bastante problemático, en [9] se propone aproximar W dadas unas funciones K-Lipschitz f resolviendo lo siguiente:

$$\min_g \max_c \left[\mathbb{E}(c(x)) - \mathbb{E}(c(g(z)))\right] \tag{3.4}$$

En este modelo, en vez de tener un discriminador que obtenga el resultado de la clase de la imagen ("real" o "falso") se tiene un "crítico" que genera un valor real. Podemos decir que éste

es un valor cuantitativo que representa cuan real es la imagen de entrada. El único requisito es que la función del "crítico" sea 1-Lipschitz continua. Es decir, que para dos valores de entrada cuales quiera, el valor de la norma de ambos sea menor o igual a 1:

$$\| f(x_1) - f(x_2) \|_2^2 \leq 1 \qquad \forall x_1, x_2 \in \mathbb{R} \tag{3.5}$$

Para poder asegurar esa continuidad 1-Lipschitz, se pueden seguir dos métodos: acotar los pesos de la red entre $[-c, c]$ siendo c un híper-parámetro o utilizar la "Penalización de Gradiente".

3.2. Penalización de Gradiente

La penalización del gradiente es una técnica más suave para forzar que el "crítico" sea 1-Lipschitz continuo. Se añade un término de regularización a la función de error de la red W-GAN que penaliza al "crítico" cuando su norma es mayor a 1. Esta regularización calcula la interpolación aleatoria ($\hat{x}$) de las imágenes reales (x) y las generadas ($g(z)$), multiplicado por un valor λ (otro híper-parámetro). La aleatoriedad de la interpolación viene dada por $\epsilon \sim P_{Uniforme}[0, 1]$ y la imagen interpolada se escribe como $\hat{x} = \epsilon \cdot x + (1 - \epsilon) \cdot g(z)$. Con esta imagen interpolada se calcula finalmente el parámetro de regularización $(\| \nabla c(\hat{x}) \|_2 - 1)^2$ y se completa la función de coste:

$$\min_g \max_c \left[\mathbb{E}(c(x)) - \mathbb{E}(c(g(z))) - \lambda(\| \nabla c(\hat{x}) \|_2 - 1)^2\right] \tag{3.6}$$

Se recomienda utilizar esta técnica en vez de acotar los valores de los pesos pues con este último método se restringe mucho el aprendizaje del "crítico". También es cierto que, dado que la regularización puede ser más suave, hay que ajustar los híper-parámetros para obtener el mejor resultado.

3.3. Distancia de Incepción de Fréchet (FID)

La Distancia de Incepción de Frechet, o FID para abreviar, es una métrica para evaluar la calidad de las imágenes generadas y desarrollada específicamente para evaluar el rendimiento de las redes generativas adversarias. El FID fue propuesto por Martin Heusel en 2017 [29].

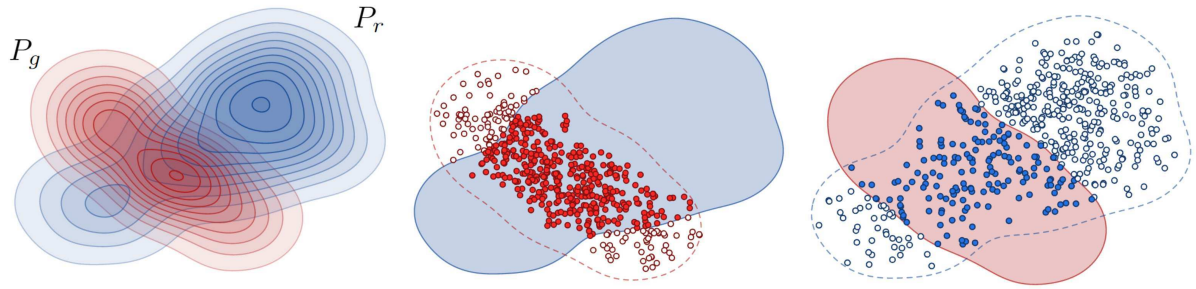
El FID utiliza el modelo *Inception v3* [32], un modelo de clasificación de imágenes de alto rendimiento que puede clasificar 1000 objetos conocidos. En concreto, la capa de codificación del modelo (la última capa de *pooling* antes de la clasificación de salida de las imágenes) se utiliza para adquirir las características de la imagen de entrada. Estas activaciones se calculan para las imágenes reales y las generadas por el modelo a evaluar. Las activaciones se pueden resumir como una gaussiana multivariable calculando la media y la covarianza (μ, σ) de las imágenes (de $p_g(z)$ para las generadas y de $p_{datos}(x)$ para las reales) y es con esos datos que se calcula la distancia entre estas dos distribuciones mediante la distancia de Frechet $d((\mu_g, \sigma_g), (\mu_{datos}, \sigma_{datos}))$, también conocida como la distancia de Wasserstein-2 [33]:

$$d((\mu_g, \sigma_g), (\mu_{datos}, \sigma_{datos})) = \| \mu_g - \mu_{datos} \|_2^2 + Tr(\sigma_g + \sigma_{datos} - 2\sqrt{\sigma_g \cdot \sigma_{datos}}) \tag{3.7}$$

Al entrenar un modelo de este tipo surgen dos objetivos: las muestras obtenidas por el generador deben ser fieles a los ejemplos reales (han de ser de "alta calidad"), y su variación debe coincidir con la distribución de datos original. El FID consigue agrupar estos aspectos en un solo valor. El problema es que no se puede ver el equilibrio entre ambos, lo que dificulta el diagnóstico del rendimiento del modelo en cierta medida.

3.4. Precision y recall

En el año 2019, en la publicación de *"Improved Precision and Recall Metric for Assessing Generative Models"* [34] se presentaron las métricas *precision* y *recall* para poder evaluar el rendimiento de los modelos generativos viendo el equilibrio entre la calidad y la variabilidad de los datos generados por dichos modelos. La métrica *precision* devuelve la probabilidad de que una imagen generada se encuentre en la variedad estimada de las imágenes reales (véase Figura 3.4b). Por otro lado, la métrica *recall* devuelve la probabilidad de que una imagen real se encuentre en la variedad estimada de las imágenes generadas (véase Figura 3.4c).



(a) Distribuciones (b) *Precision* (c) *Recall*

Figura 3.4: Dos distribuciones de datos con su representación gráfica de *precision* y *recall*.

Para obtener dichos valores, se obtienen ejemplos de ambas distribuciones como $X_r \sim P_r$ (reales) y $X_g \sim P_g$ (generadas), y se transforman a un nuevo espacio de características a través del modelo VGG-16 [25]. Los vectores de características de cada conjunto se denotan como $\phi_r \in \Phi_r$ y $\phi_g \in \Phi_g$. Para aplicar estas métricas, se requiere que $\mid \Phi_r \mid = \mid \Phi_g \mid$.

Posteriormente, para cada conjunto de vectores $\Phi \in \{\Phi_r, \Phi_g\}$ se calcula la distancia euclídea por pares entre todos los vectores del conjunto, creando después una híper-esfera con un radio igual a la distancia del vecino k-ésimo más cercano de cada vector del conjunto al que pertenece. La unión de estas híper-esferas forman un volumen que sirve como la estimación de la variedad real de los datos. Para determinar si un ejemplo ϕ está dentro de este volumen, se define la siguiente función binaria

$$f(\phi, \Phi) = \begin{cases} 1 & \text{si } \exists\ \phi' \in \Phi : \parallel \phi - \phi' \parallel_2 \leq \parallel \phi' - NN_k(\phi', \Phi) \parallel_2 \\ 0 & \text{en otro caso} \end{cases} \tag{3.8}$$

donde $NN_k(\phi', \Phi)$ devuelve el vector característico k-ésimo más cercano de ϕ' del conjunto Φ. Con esto se definen las dos métricas

$$\text{precision}(\Phi_r, \Phi_g) = \frac{1}{\mid \Phi_g \mid} \sum_{\phi_g \in \Phi_g} f(\phi_g, \Phi_r) \qquad \text{recall}(\Phi_r, \Phi_g) = \frac{1}{\mid \Phi_r \mid} \sum_{\phi_r \in \Phi_r} f(\phi_r, \Phi_g) \tag{3.9}$$

3.5. Conditional-GAN

Como se ha comentado antes, el objetivo de este proyecto es el de generar datos faciales según las clases "Edad", "Raza", "Género" y "Expresión". Por ello se plantea utilizar redes GAN condicionales, ya que permiten generar datos de esta manera. En la Figura 3.5 se puede observar algunos resultados obtenidos mediante redes GAN condicionales, con el mismo vector latente y cambiando únicamente la clase "Raza".



Figura 3.5: Caras generadas por modelo Conditional-GAN en base a la clase "Raza".

El entrenamiento habitual de las redes GAN no incluye la condicionalidad de la salida pues se calcula $P(X)$. En el caso de las redes GAN condicionales, se calcula $P(X|Y)$, lo que permite que la red aprenda a separar las clases de la distribución de datos, permitiendo así condicionar su salida. Para hacer esto, se modifica la función de coste habitual añadiéndole la clase Y:

$$\min_g \max_c \left[\mathbb{E}_{x,y\sim p_{datos}(x,y)}[\log D(x,y)] + \mathbb{E}_{z\sim p_g(z),y\sim p_{datos}(y)}[\log(1 - D(G(z,y)))]\right] \tag{3.10}$$

También se puede añadir esta condicionalidad a las redes W-GAN con penalización de gradiente modificando su función de coste añadiéndole como entrada la clase Y:

$$\min_g \max_c \left[\mathbb{E}(c(x|y)) - \mathbb{E}(c(g(z|y))) - \lambda(\| \nabla c(\hat{x}|y) \|_2 - 1)^2\right] \tag{3.11}$$

En la práctica, los modelos GAN condicionados aplican una sola capa adicional en la entrada y en la salida para añadir esta condicionalidad. En la entrada se suele concatenar el vector de entrada z con la clase codificada en formato *one-hot* que codifica las clases como se muestra en la Tabla 3.1.

		Codificación *one-hot*						
Nº	**Clase**	Enfado	Asco	Miedo	Felicidad	Neutral	Tristeza	Sorpresa
1	Asco	0	1	0	0	0	0	0
2	Felicidad	0	0	0	1	0	0	0
3	Enfado	1	0	0	0	0	0	0
4	Neutral	0	0	0	0	1	0	0
5	Tristeza	0	0	0	0	0	1	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
N	Sorpresa	0	0	0	0	0	0	1

Tabla 3.1: Ejemplo de codificación *one-hot*.

Normalmente, una vez codificadas las clases y concatenadas al vector latente z, se condensa la información a un vector del mismo tamaño que el vector latente z. En el caso de la condicionalidad en el discriminador, a cada imagen se le concatena un bloque de matrices con el tamaño de la imagen a modo vector *one-hot* correspondiente a la imagen antes de ser administrada al discriminador. Podemos ver en la Figura 3.6 como se aplican estas capas en la entrada (en amarillo y verde) y en el discriminador (en amarillo y gris).

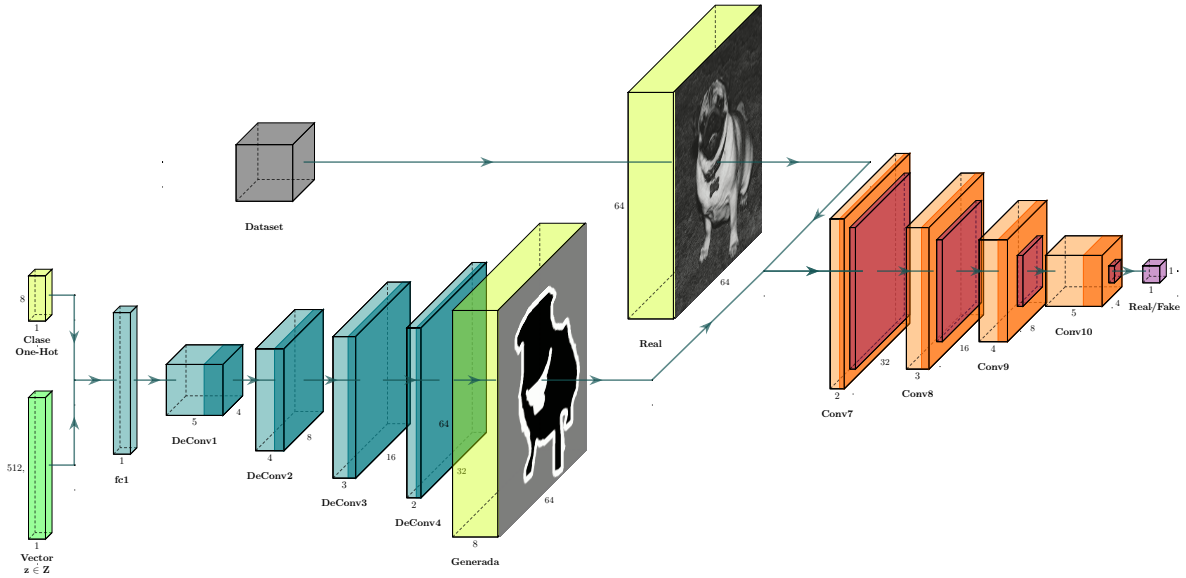


Figura 3.6: Arquitectura de red GAN condicionada.

3.6. Normalización de Instancias Adaptativa (AdaIN)

La normalización de instancias adaptativa es un método de normalización basado en la normalización de instancias [35] que alinea la media y la varianza de las características del contenido de la imagen con las de las características de su estilo. Fue presentado por Xun Huang y Serge Belongie en 2017 [30].

La normalización de instancias [35] normaliza la entrada a un único estilo especificado por los parámetros. Pero en AdaIN, se recibe una entrada de contenido x y una entrada de estilo y, y se alinean la media (μ) y la varianza (σ) de x para que coincidan con las de y. A diferencia de la normalización por lotes, la normalización por instancias o la normalización por instancias condicional, AdaIN no tiene parámetros entrenables. En su lugar, calcula de forma adaptativa los parámetros afines a partir de la entrada de estilo:

$$AdaIN(x, y) = \sigma(y) \left(\frac{x - \mu(x)}{\sigma(x)} \right) + \mu(y) \tag{3.12}$$



Figura 3.7: Transferencia de estilos.

Esta normalización AdaIN se utiliza principalmente en modelos de transferencia de estilos (como los de la Figura 3.7) entre el codificador y el decodificador. También se utiliza en modelos como StyleGAN o StyleGAN2, modelos generativos de alta calidad. En este caso se utiliza en cada bloque del generador como se utilizaría por ejemplo *Batch Normalization*.

3.7. Pro-GAN

La capacidad de las redes generativas de generar imágenes de gran calidad fue un problema complicado de resolver, ya que la única manera de generar esas imágenes era entrenar modelos durante muchas iteraciones. Pero cuanto más largos son los entrenamientos de estas redes, más probabilidades hay de que el generador se quede atascado en un "colapso de moda". Por eso, después de varias investigaciones, en 2017 se publicó el artículo *"Progressive Growing of GANs for Improved Quality, Stability, and Variation"* [12] donde se presentó un nuevo método para entrenar redes generativas de gran resolución.

El principio del *Progressive Growing* se basa en el entrenamiento progresivo del generador y el discriminador, empezando por entrenar imágenes de tamaño 4×4, luego de 8×8, después de 16×16 y así hasta 1024×1024. En la Figura 3.8 (basado en [12]) podemos ver esa progresión.

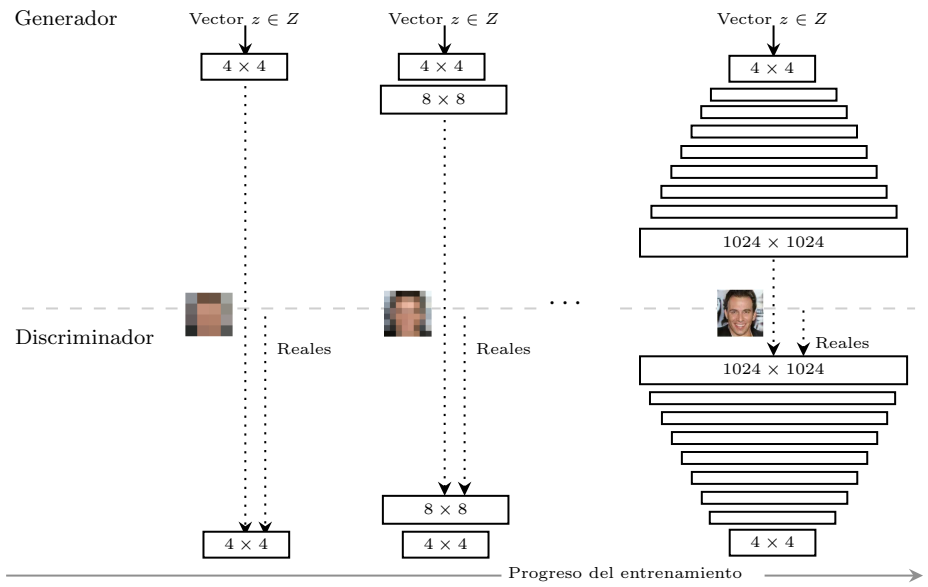
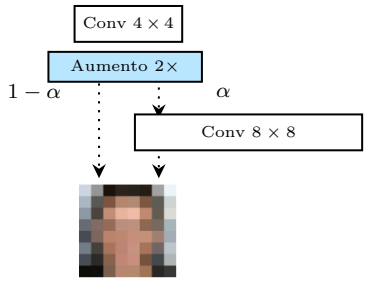
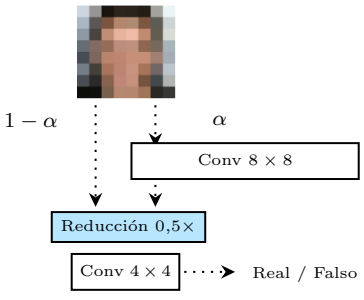


Figura 3.8: Entrenamiento progresivo de una red Pro-GAN.

Cada vez que el generador y el discriminador se estabilizan durante el entrenamiento, se empieza a añadir progresivamente el siguiente aumento. En el generador, se empieza por utilizar un $1-\alpha$ de la imagen aumentada $2\times$ (siendo α un parámetro que va cambiando progresivamente en cada iteración) y un α del resultado de la deconvolución (véase Figura 3.9a). En cuanto al discriminador, esta progresión se hace de manera análoga al generador, utilizando un $1-\alpha$ de la imagen reducida $0{,}5\times$ y un α del resultado de la convolución (véase Figura 3.9b).



(a) Generador

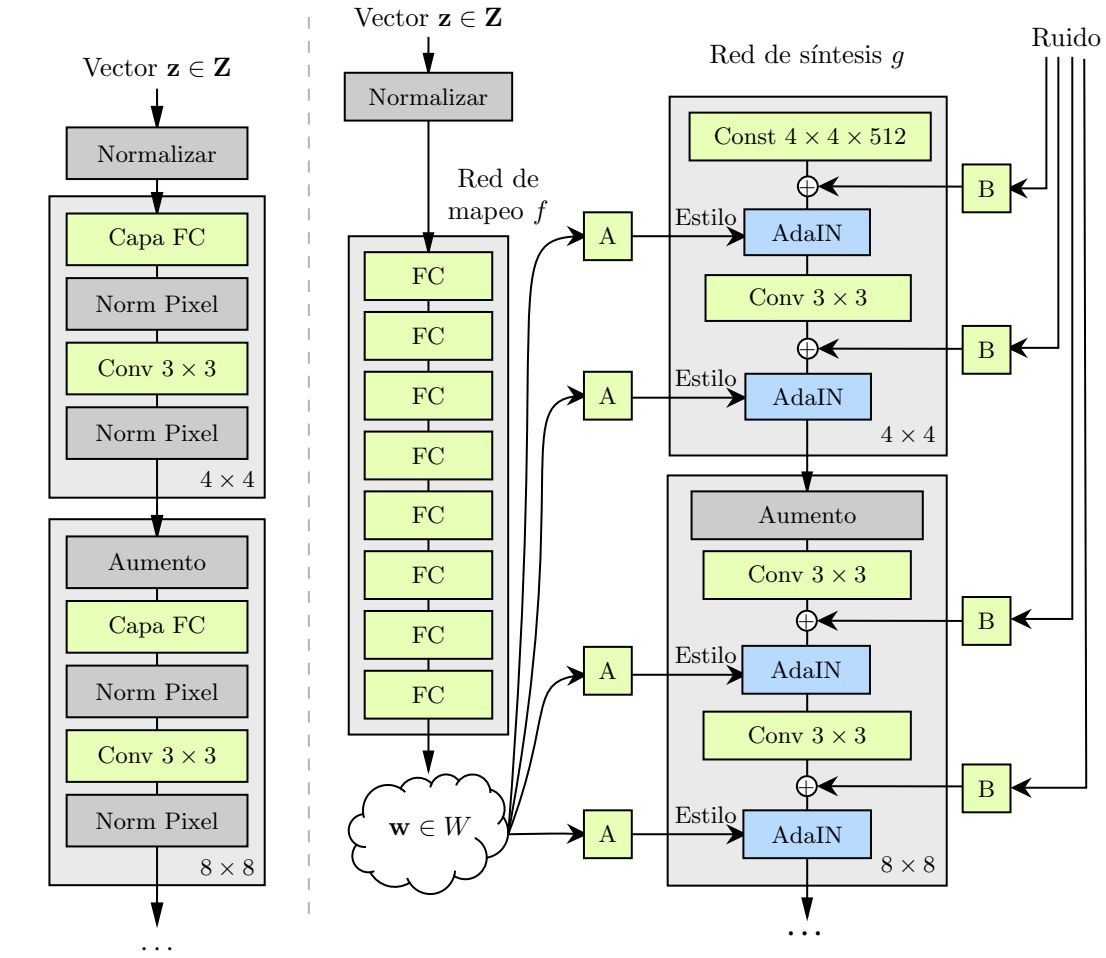


(b) Discriminador

Figura 3.9: Método progresivo de inclusión de nuevo escalado.

3.8. StyleGAN

Los generadores convencionales introducen el vector latente z al generador aumentando su tamaño hasta conseguir una imagen generada que pueda ser evaluada por el discriminador. Incluso en arquitecturas como Pro-GAN [12], que utilizan el crecimiento progresivo de la red, siguen este método (Figura 3.10a). En StyleGAN [31] se utiliza el mismo método de progresión que en Pro-GAN, pero la diferencia es que el vector latente z pasa primero por una red $f : Z \to W$ que produce $w \in W$ (Figura 3.10b). Ese nuevo vector w se utiliza para controlar el generador a través de la normalización de instancias adaptativa en cada capa de convolución mediante una transformación entrenable (A en Figura 3.10b). Además de eso, antes de cada normalización AdaIN se añade una imagen de ruido gaussiano con un factor de escalado entrenable (B en Figura 3.10b). Las razones de añadir este ruido se explican en [31].



(a) Generador Pro-GAN

(b) Generador StyleGAN

Figura 3.10: Diferencias entre el generador de Pro-GAN y StyleGAN.

Con este método de mapear el vector latente $z \in Z$ a otro espacio latente W se consigue desentrelazar el vector latente, haciendo que los atributos finales de la imagen sean independientes entre sí. Para entender que significa este "desentrelazado", he aquí un ejemplo: imagínese que se tiene un rostro de una mujer y se quiere modificar esa imagen para que la mujer tenga barba. Si no se desenlazan los atributos, al intentar cambiar este atributo, lo más probable es que la imagen resultante cambie el género a hombre. Esto se debe a que el modelo generativo habrá

aprendido de la distribución real de los datos, donde la extrema mayoría de personas que tienen barba son hombres (teniendo en cuenta que la gran mayoría de *datasets* tienen este sesgo). Por lo tanto, el modelo correlaciona las variables según esos datos con su sesgo correspondiente. StyleGAN, gracias a esta red $f : Z \rightarrow W$, consigue evitar esto.

Para demostrar este "desentrelazado", en [31] se propone una nueva métrica que cuantifica lo bien que se pueden separar los puntos del espacio latente en dos conjuntos distintos a través de un híper-plano lineal, de modo que cada conjunto corresponda a un atributo binario específico de la imagen. Se utiliza un modelo auxiliar para clasificar varias imágenes y se compara dicha clasificación con una red pre-entrenada calculando la entropía $H(Y|X)$, siendo Y la clase definida por el modelo pre-entrenado y X la clase definida por el modelo auxiliar. Con esto se calcula el valor de divisibilidad para todos los atributos i con $exp(\sum_i H(Y_i|X_i))$. Valores más pequeños sugieren mejor divisibilidad entre clases. En la Figura 3.11 podemos ver como las variaciones en el vector latente en relación con el pelo no afectan a la identidad ni a la pose.



(a) Imagen generada (b) Variación estocástica (c) Desviación estándar

Figura 3.11: Ejemplos de variación estocástica.

Aunque la calidad de las imágenes generadas por este modelo son muy altas, sigue habiendo algunos problemas. Uno de ellos es la aparición de artefactos en algunas de las imágenes generadas (véase Figura 3.12) que podrían ser problemáticos para el entrenamiento de reconocimiento de expresiones, debido al tipo de ruido que se genera en las imágenes. Es por eso que se plantea utilizar StyleGAN2 [1], que elimina este problema y mejora aún más la calidad de las imágenes.

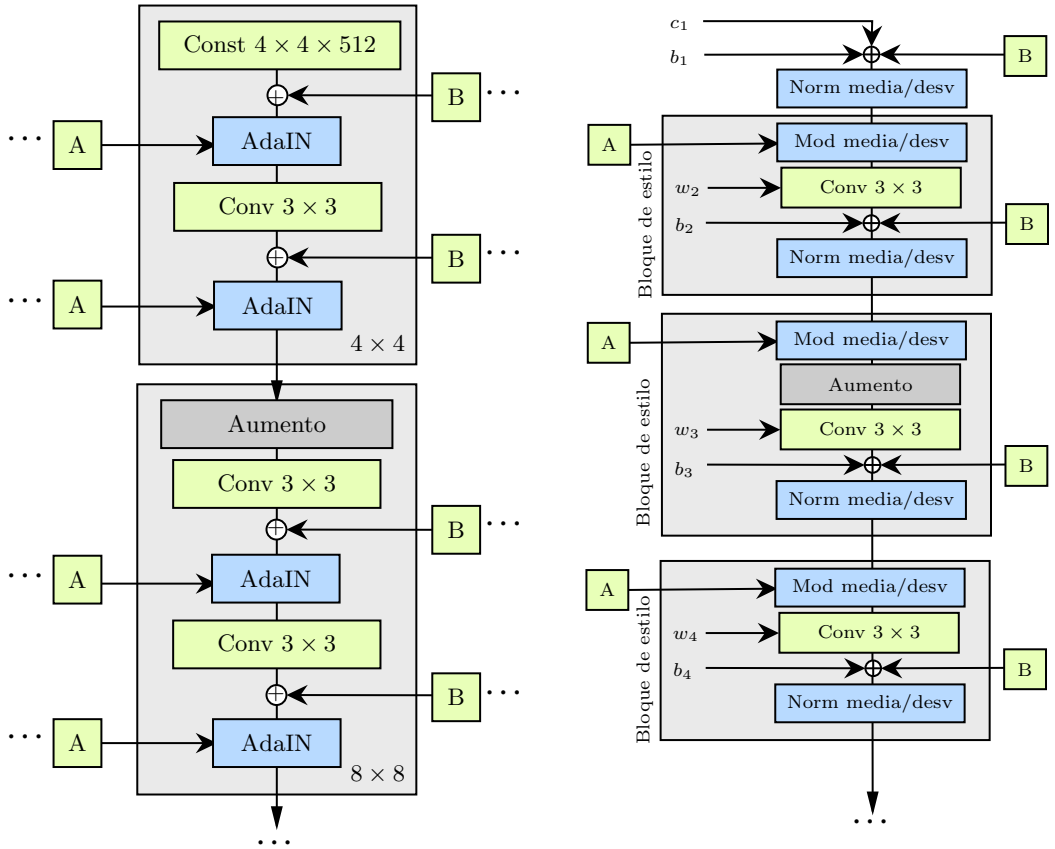


Figura 3.12: Artefactos en imágenes generadas por StyleGAN [31].

3.9. StyleGAN2

El modelo de StyleGAN [31] fue durante varios meses el modelo más avanzado en el estado del arte de las redes GAN en cuanto a generación de rostros humanos. Aún así, no tardaron ni un año en presentar una nueva versión de este modelo en el artículo *"Analyzing and Improving the Image Quality of StyleGAN"* [1], donde analizan y mejoran la calidad de las imágenes, deshaciéndose del problema de los artefactos generados por motivo de la normalización en el bloque de estilo del modelo original.

En [1] teorizan que la causa de estas anomalías en las imágenes (Figura 3.12), aunque poco perceptibles en las imágenes finales, se debe a la normalización de instancias en el bloque de estilos empezando desde las capas de tamaño 64×64. Su hipótesis se basa en el hecho de que la Normalización de Instancias Adaptativa normaliza la media y la desviación de cada mapa de características de manera separada, perdiendo información y haciendo que el generador genere estos artefactos de gran magnitud localizados para añadir esta información de manera artificial con el propósito de engañar al discriminador. Demuestran esto con los resultados obtenidos quitando el proceso de normalización, en los cuales estos artefactos desaparecen. En la Figura 3.13 se puede ver el diagrama detallado de StyleGAN donde aparecen todos los pasos del bloque de estilo (en el cual aparece dicha normalización de la media y la desviación estándar).



(a) StyleGAN

(b) StyleGAN (detallado)

Figura 3.13: Estructura detallada de StyleGAN [1].

Para solventar estos problemas sin eliminar completamente la Normalización de Instancias Adaptativa, la cual ofrece un control sobre el estilo de las imágenes generadas, en [1] ofrecen una solución que mantiene ese control de estilo. Primero eliminan la modulación y la normalización de la media en cada proceso, pues los resultados no varían y el FID del modelo tampoco se ve alterado. Luego modifican esa modulación y normalización de la desviación estándar, aplicando una modulación seguida de una de-modulación que viene dada por

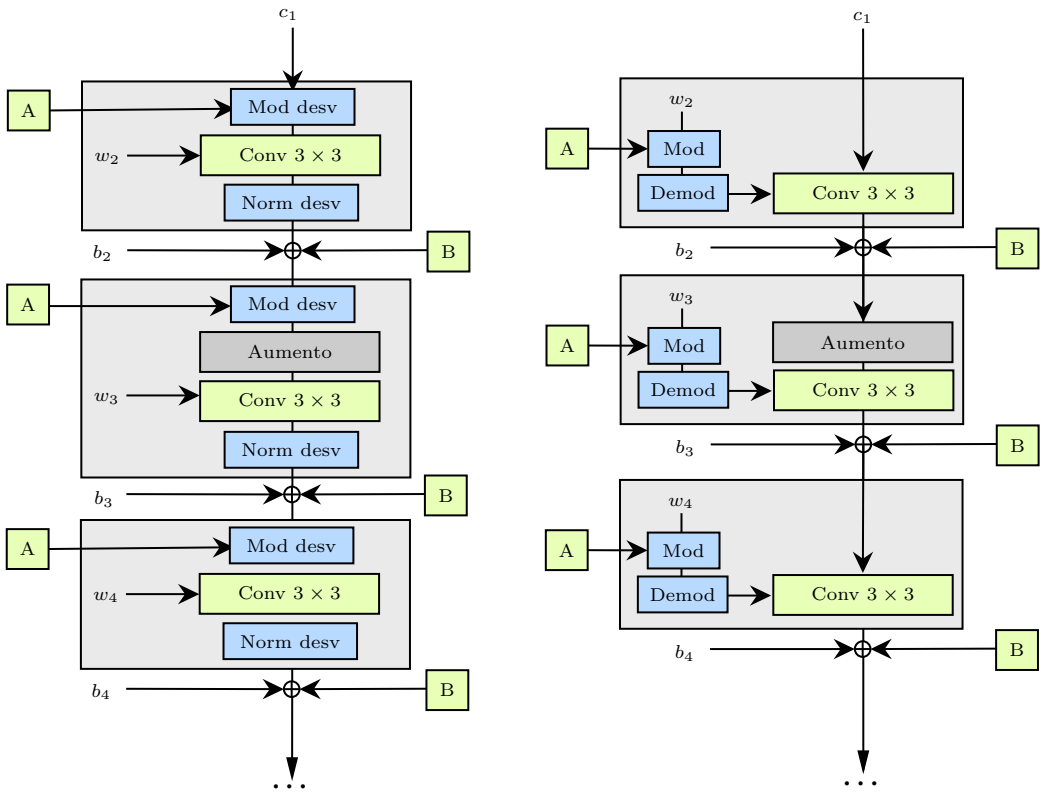
$$w''_{ijk} = w'_{ijk} \Big/ \sqrt{\sum_{i,k} {w'_{ijk}}^2 + \epsilon}, \tag{3.13}$$

donde i corresponde al mapa de características i-ésimo y donde j y k corresponden al mapa de características de salida y a la información espacial de la convolución respectivamente. El valor de w'_{ijk} se define como

$$w'_{ijk} = s_i \cdot w_{ijk}, \tag{3.14}$$

donde s_i corresponde a la escala correspondiente al mapa de características i-ésimo.

Además de esta nueva de-modulación de pesos, la adición de la imagen de ruido B se hace fuera del bloque de estilo (bloque en gris). En la Figura 3.14 se puede observar esta nueva arquitectura.



(a) Arquitectura revisada

(b) De-modulación de pesos

Figura 3.14: Nueva arquitectura y de-modulación de pesos.

Para mejorar aún más la calidad de las imágenes generadas, en [1] plantean otra manera de

obtener imágenes de gran resolución sin utilizar la técnica de *Progressive Growing*, ya que genera artefactos a los que denominan de "fase". En la Figura 3.15 se puede apreciar como los dientes se quedan en la misma posición delimitada por al línea azul mientras que la pose cambia.

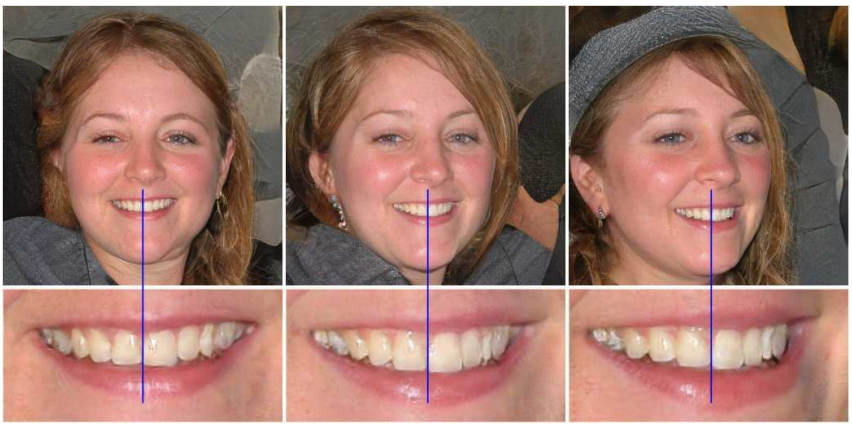
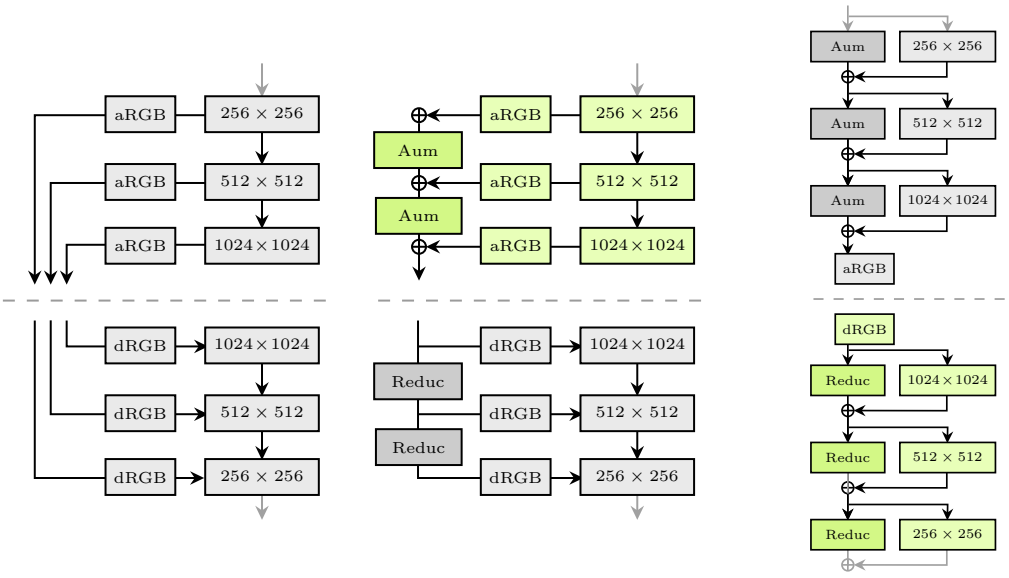


Figura 3.15: Artefactos de "fase" al cambiar la postura de un rostro en StyleGAN.

La solución que proponen es basarse en la arquitectura de MSG-GAN [36], un modelo que utiliza conexiones entre distintas capas del generador y lo mismo en el discriminador. En la Figura 3.16 se pueden observar distintas arquitecturas de generadores y discriminadores. En las Subfiguras 3.16b y 3.16c los procesos Aum y Reduc denotan aumentos y reducciones bilineales. En todas ellas se incluyen también los procesos aRGB y dRGB que representan conversiones entre datos híper-dimensionales de cada píxel y RGB. Las arquitecturas de generador (arriba) y discriminador (abajo) utilizadas en el mejor modelo de [1] están marcadas en verde.



(a) MSG-GAN [36] (b) Saltos de entrada y salida (c) ResNets

Figura 3.16: Tres arquitecturas de generadores y discriminadores estudiados en [1].

Será finalmente con una versión modificada de esta arquitectura de red GAN que se proponen los métodos para generar datos de, teóricamente, alta calidad y variedad para el posterior entrenamiento de modelos de detección de expresiones faciales con datos más equilibrados y menos sesgados.

4. BÚSQUEDA DE BASES DE DATOS

A lo largo del desarrollo del proyecto se han ido experimentando con distintas arquitecturas de redes GAN y distintos *datasets* para conseguir una combinación que ofreciera resultados de buena calidad. Al principio del proyecto, durante unas cuantas semanas, se estudió el estado del arte consultando más de 50 artículos académicos. Gracias a esos conocimientos se podían realizar grandes cantidades de pruebas. Por ello, en este apartado se exponen los *datasets* estudiados y utilizados a lo largo de las pruebas realizadas, previas a los experimentos de los métodos propuestos, y el *dataset* definitivo utilizado en los métodos finales. Se ofrecen descripciones breves sobre su estructura y contenido, así como pequeñas muestras de los datos.

4.1. TinyFace

El primer *dataset* utilizado tuvo que ser uno en el que las imágenes eran de tamaño pequeño para poder realizar las primeras pruebas, ya que el nivel de cómputo del equipo disponible en ese entonces era limitado pues se trataba de un equipo propio y algo antiguo. Por eso se decidió utilizar el *dataset "TinyFace: Face Recognition in Native Low-resolution Imagery"* [37]. Consiste en 5139 rostros etiquetados dados por 169 403 imágenes de rostros de baja resolución (una media de 20×16 píxeles) diseñado para reconocimiento facial 1:N. Dado que las etiquetas faciales no eran de interés en el proyecto, solo se utilizaron las imágenes. En la Figura 4.1 se pueden ver unos cuantos ejemplos.



Figura 4.1: Ejemplos del *dataset* TinyFace [37].

Dado que la calidad de las imágenes no es la mejor y que las etiquetas del *dataset* no son las que se necesitan para el trabajo, simplemente se utilizó el *dataset* para descartar algunas de las arquitecturas al inicio del desarrollo del proyecto.

4.2. CelebA

En la búsqueda de imágenes de mejor calidad, se encontró el *dataset "Large-scale CelebFaces Attributes (CelebA) Dataset"* [38]. Consiste en más de 200 000 imágenes de personas famosas con 40 atributos binarios. En este *dataset* se cubren bastantes poses y fondos diversos. En la Figura 4.2 se muestran algunos ejemplos de éste.

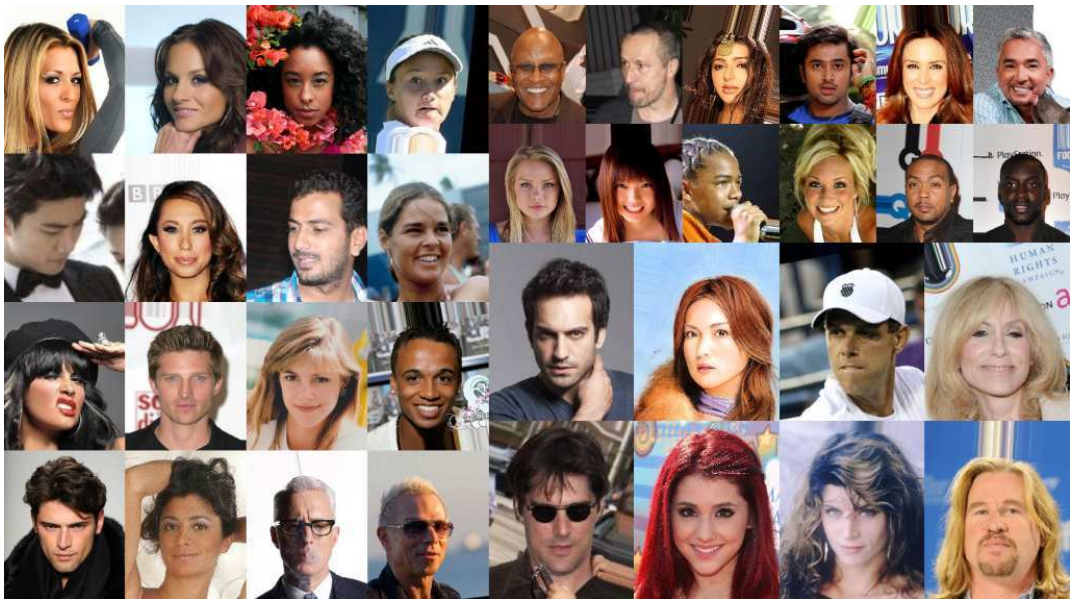


Figura 4.2: Ejemplos del *dataset* CelebA [38].

Los atributos disponibles en este *dataset* son los siguientes (en inglés): *"5_o_Clock_Shadow"*, *"Arched_Eyebrows"*, *"Attractive"*, *"Bags_Under_Eyes"*, *"Bald"*, *"Bangs"*, *"Big_Lips"*, *"Big_Nose"*, *"Black_Hair"*, *"Blond_Hair"*, *"Blurry"*, *"Brown_Hair"*, *"Bushy_Eyebrows"*, *"Chubby"*, *"Double_Chin"*, *"Eyeglasses"*, *"Goatee"*, *"Gray_Hair"*, *"Heavy_Makeup"*, *"High_Cheekbones"*, *"Male"*, *"Mouth_Slightly_Open"*, *"Mustache"*, *"Narrow_Eyes"*, *"No_Beard"*, *"Oval_Face"*, *"Pale_Skin"*, *"Pointy_Nose"*, *"Receding_Hairline"*, *"Rosy_Cheeks"*, *"Sideburns"*, *"Smiling"*, *"Straight_Hair"*, *"Wavy_Hair"*, *"Wearing_Earrings"*, *"Wearing_Hat"*, *"Wearing_Lipstick"*, *"Wearing_Necklace"*, *"Wearing_Necktie"* y *"Young"*. Las únicas interesantes serían *"Male"*, *"Smiling"* y *"Young"*.

Este *dataset* fue bastante útil para empezar a entrenar modelos más grandes con equipos más capaces. Sirvió para descartar unas cuantas ideas y para solventar los errores más comunes a la hora de entrenar redes GAN como las que se han explicado anteriormente en el Apartado 3.1. Aún así, ya que las etiquetas que ofrece este *dataset* no cubrían las necesidades del proyecto, se siguieron explorando más posibilidades.

4.3. UTKFace+

Ya que uno de los propósitos del proyecto es generar una base de datos de rostros variada, se buscó un *dataset* que tuviese etiquetas como la "Edad", la "Raza" y el "Género". El *dataset "UTKFace Large Scale Dataset"* [39] cumplía con estos requisitos. Consiste en 20 000 imágenes

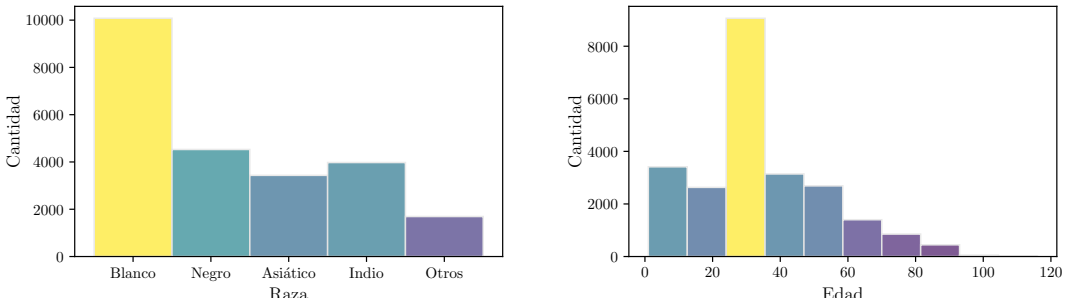
con anotaciones de la “Edad”, el “Género” y la “Raza”. El atributo “Edad” toma valores desde 0 hasta 116 años, el atributo “Género” puede ser 0 (“Hombre”) o 1 (“Mujer”) y el atributo “Raza” puede tomar valores de 0 a 4, denotando “Blanco”, “Negro”, “Asiático”, “Indio” y “Otros” (como Hispano, Latino, etc.). Además, las imágenes cubren una gran variedad de posturas, expresiones faciales, iluminación, oclusión, etc. En la Figura 4.3 se muestran algunos ejemplos.



Figura 4.3: Ejemplos del *dataset* UTKFaces [39].

Con este *dataset* se empezaron a realizar algunas pruebas con modelos más complejos de redes GAN como las GANs convolucionales o W-GAN.

El único problema con este *dataset* es que, aunque los autores aseguran que es muy variado, en realidad hay bastantes sesgos en cuanto a “Raza” y “Edad” (véase Figura 4.4). La mayoría de instancias tienen la etiqueta “Blanco” en la clase “Raza” y hay un gran numero de instancias con edades en un rango de 25 a 35 años. Además, no está disponible la clase de expresión facial, por lo que se siguieron buscando otras alternativas.



(a) Raza (b) Edad

Figura 4.4: Sesgo en las clases “Raza” y “Edad” en UTKFaces [39].

4.4. Fer2013

Ya que en este trabajo se intentan generar rostros etiquetados con expresiones faciales, se decidió probar con uno de los *datasets* más utilizados en reconocimiento de emociones: FER2013, acrónimo de *Facial Expression Recognition*, fue introducido en la *ICML 2013 Workshop* [40].

El *dataset* consiste en imágenes de rostros en escala de grises de 48×48 píxeles extraídos automáticamente para que la cara esté más o menos centrada y ocupe aproximadamente el mismo espacio en cada imagen. Los datos están etiquetados según siete categorías: "Enfado", "Asco", "Miedo", "Felicidad", "Tristeza", "Sorpresa" y "Neutral". El conjunto de entrenamiento consta de 28 709 ejemplos y el conjunto de *test* tiene 3589 ejemplos. En la Figura 4.5 se muestran algunos ejemplos.



Figura 4.5: Ejemplos del *dataset* FER2013 [40].

Aunque el *dataset* podía ser interesante de cara a generar los datos condicionados, los resultados obtenidos con las redes convencionales fueron algo mediocres. Seguramente se debe a que la resolución de las imágenes era demasiado baja, por lo que la generación de datos fieles a los reales (al igual que pasaba con *TinyFaces*) se vieron afectados por la falta de información en las imágenes originales. Eso ligado a que las imágenes están en blanco y negro, seguramente afectó negativamente a la generación de datos deseados. Por estos motivos se descartó este *dataset* y se siguió buscando.

4.5. FairFace

Como se ha comentado antes, uno de los objetivos del trabajo, a parte de generar datos con expresiones faciales etiquetadas, es intentar generar una base de datos equilibrada en cuanto a "Edad", "Raza" y "Género". Por eso mismo se decidió realizar más pruebas con el *dataset* FairFace [41]. Contiene 108.501 imágenes de 7 grupos étnicos diferentes: blancos, negros, indios, asiáticos del este, del sudeste asiático, de Oriente Medio y latinos. Las imágenes se recogieron del conjunto de datos de Flickr YFCC-100M y vienen etiquetados según raza, género y edad. En la Figura 4.6 se pueden ver algunos ejemplos.

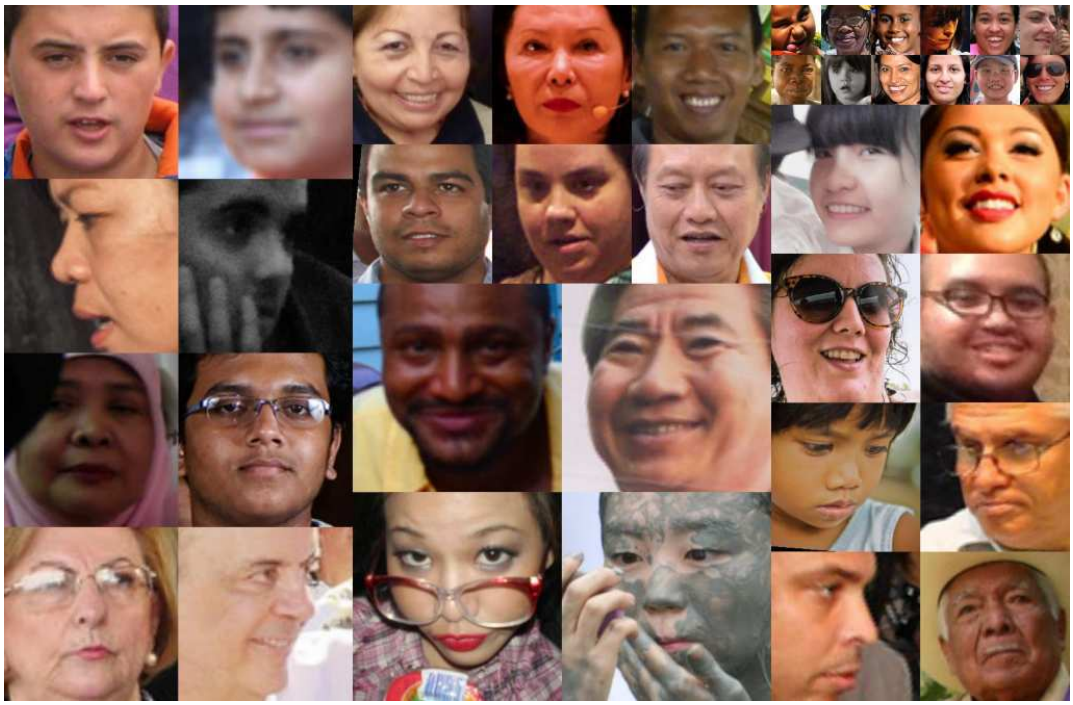


Figura 4.6: Ejemplos del *dataset* FairFace [41].

Este fue uno de los *datasets* más utilizados hasta el final del desarrollo de las pruebas previas, cuando aún se estaban desarrollando los primeros modelos de GANs y GANs condicionales. Algunos de los mejores resultados obtenidos fueron con este *dataset* (los ejemplos mostrados en la Figura 3.5 son algunos de ellos), pero el único problema era que no se tenían las etiquetas de la expresión facial. Por ese motivo se decidió utilizar otro *dataset*.

4.6. AffectNet

En la búsqueda del *dataset* adecuado para las pruebas se fueron encontrando algunos que eran prometedores, pero todos tenían algún problema: no tenían las etiquetas deseadas, la calidad de las imágenes era mediocre o no estaba suficientemente equilibrado. Para solventar algunos de los problemas se decidió utilizar el *dataset* AffectNet [42]. Este *dataset* cuenta con 420 299 imágenes etiquetadas con las expresiones faciales "Neutral", "Felicidad", "Tristeza", "Sorpresa", "Miedo", "Enfado", "Asco", "Desprecio", "Incierto" y "No rostro".

El problema es que el *dataset* original no tiene etiquetas de "Género", "Edad" ni "Raza". Pero la calidad de las imágenes es lo suficientemente grande (224×224 en la versión final) como para poder ser utilizadas ya que dan bastante información al modelo generativo dentro de las limitaciones de hardware de las que se disponen para el proyecto. Por ese motivo, se plantea utilizar una versión modificada del *dataset* utilizado en [43], y facilitado por Iris Dominguez-Catena, estudiante de doctorado en la Universidad Pública de Navarra.

En este artículo se utilizan los modelos preentrenados de Fairface [44], para determinar las etiquetas demográficas del *dataset* y se aplican recortes para centrar los rostros en las imágenes de 224×224 píxeles mediante el uso de la librería `dlib`. En este trabajo se utiliza el *dataset* etiquetado demográficamente utilizando el modelo de Fairface, con las imágenes originales y las recortadas. Con este procesado, se añade la información de las clases "Raza" (con etiquetas "Blanco", "Latino", "Negro", "Medio Este", "Asiático Este", "Indio" y "Asiático Sud-Este"),

"Raza 4" (con etiquetas "Blanco", "Asiático", "Negro" e "Indio"), "Edad" (con etiquetas de "0-2", "3-9", "10-19", "20-29", "30-39", "40-49", "50-59", "60-69" y "70+") y "Género" (con etiquetas "Mujer" y "Hombre") además de la clase "Expresión" que ya venía con el *dataset* original.

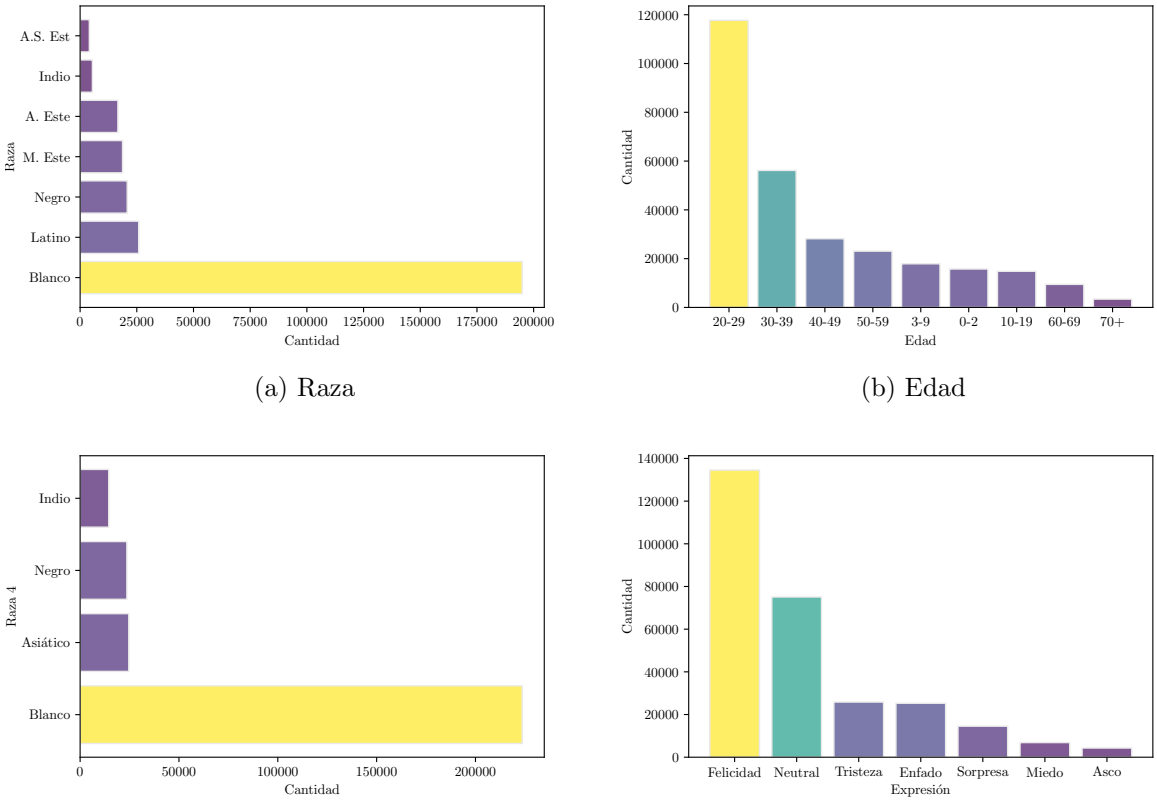
Hay que señalar que las anotaciones proporcionadas por FairFace son imperfectas y muy subjetivas, y éstas constituyen solo una medida aproximada de cada instancia. Por ejemplo, la clasificación binaria de género de acuerdo a Fairface provoca un sesgo importante contra personas no binarias [43, 45]. Además, hace falta recalcar que el *dataset* original no está libre de fallos. En la Figura 4.7 se pueden observar varias instancias erróneas.



(a) Felicidad (b) Neutral (c) Tristeza (d) Enfado (e) Sorpresa (f) Miedo (g) Asco

Figura 4.7: Instancias erróneas del *dataset* AffectNet.

Aunque el *dataset* modificado es mejor que los demás en varios aspectos, sigue teniendo grandes problemas de sesgo en cuanto a las etiquetas de "Raza", "Raza 4", "Edad" y "Expresión". En la Figura 4.8 podemos ver como casi todas las clases tienen grandes sesgos. Por ejemplo en la clase "Raza" (4.8a), hay una diferencia muy grande entre la etiqueta "Blanco" y las demás.



(a) Raza (b) Edad

(c) Raza 4 (d) Expresión

Figura 4.8: Sesgo en las clases de AffectNet [42].

Aún así, a falta de encontrar un *dataset* que cumpliera con todos los requisitos, se decidió utilizar este *dataset* modificado para el entrenamiento de algunos de los modelos estudiados y propuestos. En la Figura 4.9 se muestran algunos ejemplos del *dataset* final, los cuales no difieren del original pues no se cambian las imágenes, solo se añade información a cada instancia como se ha explicado antes.



Figura 4.9: Ejemplos del *dataset* AffectNet [42].

5. ENTORNO DE DESARROLLO

En este trabajo se han utilizado distintas herramientas de programación y hardware específico para el entrenamiento de modelos. En este apartado se describen esas herramientas y el hardware utilizado, así como las limitaciones en cuanto a éste para el entrenamiento y ejecución de los modelos desarrollados con el fin de ofrecer una configuración adecuada.

5.1. Lenguaje de programación y herramientas

La totalidad del trabajo se ha desarrollado en Python en su versión 3.9.6 [46] con el gestor de paquetes, dependencias y entornos Conda en su versión 4.10.3 [47]. En cuanto a las herramientas de desarrollo, se han utilizado los IDE PyCharm 2021.2 (Community Edition) [48] y Visual Studio Code [49] con las extensiones de Jupyter-Notebook [50] para crear las figuras y las gráficas mostradas en este trabajo. No se recomienda entrenar los modelos generativos en el entorno de Jupyter-Notebook, ya que el *kernel* del entorno ha fallado en numerosas ocasiones.

5.1.1. Librerías

Las principales librerías utilizadas en el entorno de desarrollo Conda son:

- **PyTorch:** librería *torch* para desarrollo de redes neuronales.
- **Cuda:** para el manejo de la GPU. Viene incluido en la librería *torch*.
- **Torch Vision:** para el manejo de imágenes y modelos pre-entrenados.
- **Torch Optimizer:** contiene muchos optimizadores para el entrenamiento.
- **Scikit-Learn:** para preprocesar los datos (codificadores de etiquetas).
- **Numpy:** para manejo de datos en forma de vectores y matrices.
- **Pandas:** para utilizar y manejar DataFrames y leer de ficheros .csv.
- **Math:** para operaciones matemáticas.
- **PIL:** para manejo de imágenes
- **OS:** para llamadas al sistema.
- **PathLib:** para el manejo de directorios.
- **JSON:** para el manejo de ficheros tipo .json.
- **Fire:** biblioteca para generar automáticamente interfaces de línea de comandos (CLI) a partir de absolutamente cualquier objeto de Python.
- **TQDM:** para las barras de progreso.
- **Random:** para generar números aleatorios.
- **Dataset**: para facilitar el manejo de datos en el entrenamiento.

5.2. Hardware utilizado

Dado que algunos de los modelos a utilizar en este trabajo necesitan de gran nivel de cómputo, ha sido necesario utilizar hardware específico para entrenarlos. En la siguiente Tabla 5.1 se muestran las especificaciones del sistema que se proponen como requisitos recomendados para obtener los resultados conseguidos en este trabajo:

Sistema operativo	Windows 10 Enterprise LTSC
Procesador	Intel(R) Core(TM) i5-8500 CPU @ 3.00GHz
Memoria RAM	20,0 GB
Tipo de sistema	Procesador x64, 64 bits
GPU	NVIDIA GeForce RTX 3080
Memoria VRAM (GPU)	10GB
Memoria Almacenamiento	1TB HDD + 480GB SSD

Tabla 5.1: Especificaciones técnicas del sistema.

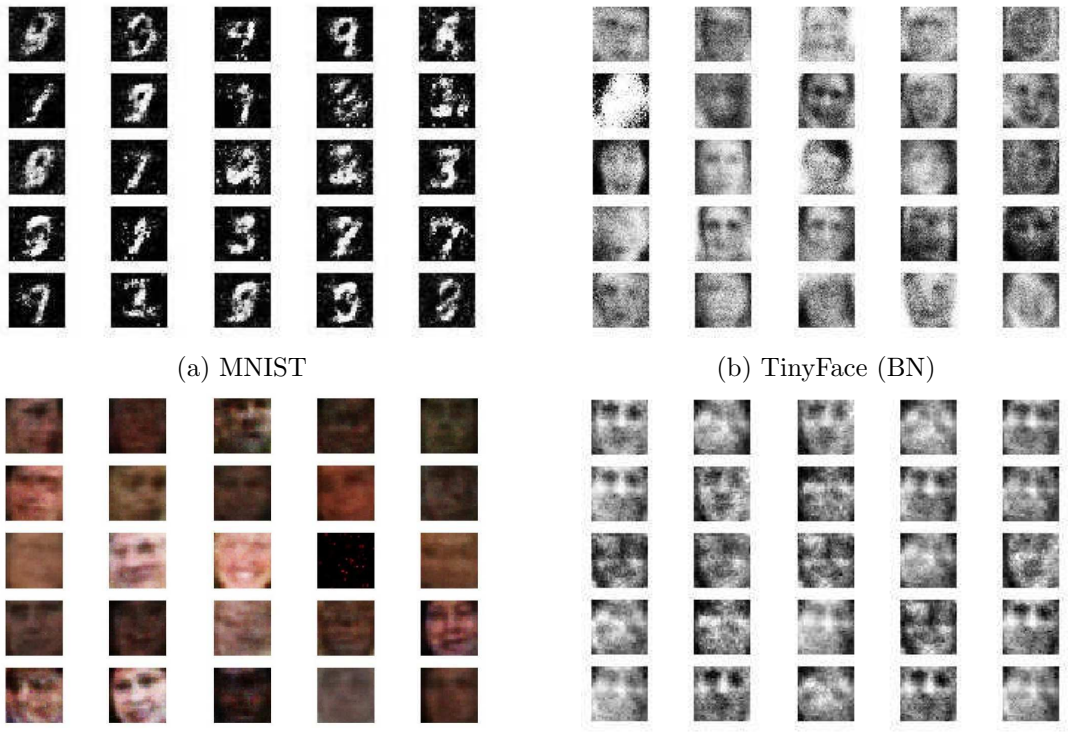
En este trabajo se utiliza una versión de GPU de la marca NVIDIA [51] pues son las más fáciles de utilizar con las librerías de desarrollo de modelos de redes neuronales actuales, mediante el uso de drivers CUDA [52]. Por eso mismo, para obtener los resultados en la misma cantidad de tiempo que en las pruebas realizadas, es necesario usar dichas tarjetas gráficas. Aún así, se pueden intentar entrenar los modelos más avanzados (i.e. StyleGAN2) mediante el uso de la CPU. Los entrenamientos serán considerablemente más largos, pero se deberían obtener los mismos resultados que utilizando la GPU (aunque no se recomienda).

6. EXPERIMENTOS PREVIOS

6.1. Experimentación con redes GAN

Para poder desarrollar los métodos propuestos más adelante, antes se vio necesaria la experimentación con varias implementaciones de redes GAN simples, ya que el nivel de conocimiento sobre la implementación de éstas al principio del proyecto era prácticamente nula. Por ello, antes de empezar a exponer los métodos propuestos, se presentan varias de las pruebas realizadas y los resultados de los experimentos con varias de las redes generativas explicadas anteriormente.

Las primeras pruebas se realizaron usando redes GAN profundas (no convolucionales) para ver cómo se comportaban con pequeñas cantidades de datos y para ver cuáles eran los problemas más habituales (como los mencionados en el Apartado 3.1). Para empezar, se utilizó una implementación de una red GAN con el *dataset* de MNIST [53], un *dataset* de números manuscritos con imágenes en blanco y negro de $28 \times 28 \times 1$ píxeles. Para ello se utilizó una arquitectura simple con un vector de entrada z_{100} con 100 valores aleatorios muestreados de una distribución normal ($z \sim N(\mu, \sigma)$ con $\mu = 0$ y $\sigma = 1$) para el generador, siguiendo con capas densas de 256, 512, 1024 y $28 \times 28 \times 1 = 784$ neuronas. Esta última capa se redimensiona al tamaño de la imagen original para poder visualizarla una vez generada, pero durante el entrenamiento se mantiene como un vector de tamaño 784. Para el discriminador se conectan las imágenes convertidas a un vector de 784 elementos con capas de 1024, 512, 256 y 1 (capa de salida) neuronas. Después de ver que el modelo generaba instancias de números medianamente reconocibles (véase Figura 6.1a) se pasó a utilizar el *dataset* TinyFace [37] en escala de grises así como en RGB reescaladas a tamaño 28×28 por simplicidad en la arquitectura. En las Figuras 6.1b y 6.1c se pueden observar los resultados obtenidos.



(a) MNIST

(b) TinyFace (BN)

(c) TinyFace (RGB)

(d) TinyFace (colapso de moda)

Figura 6.1: Resultados obtenidos con red GAN profunda.

En los resultados anteriores se puede ver que la calidad de las imágenes generadas es realmente baja y se puede observar que algunos entrenamientos sufren de colapso de moda (véase Figura 6.1d). Como se explica antes, el *dataset* tiene imágenes de pequeño tamaño, por lo que el modelo generador no es capaz de obtener suficiente información para crear imágenes que puedan engañar al discriminador y mucho menos convencer a un ser humano de que la imagen es real. Por eso, se decidió utilizar otros *datasets* para seguir probando el mismo modelo: CelebA [38] y UTKFace+ [39]. Para estos dos se mantuvo la misma arquitectura, cambiando únicamente el tamaño final de salida del generador tanto como el tamaño de entrada del discriminador que pasaría a ser de $178 \times 218 \times 3$ neuronas para el *dataset* de CelebA (véase Figura 6.2) y de $128 \times 128 \times 3$ para el *dataset* de UTKFaces+.

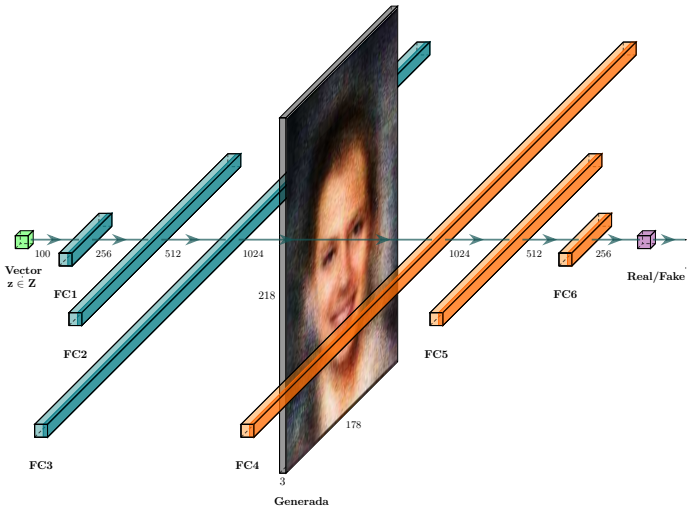
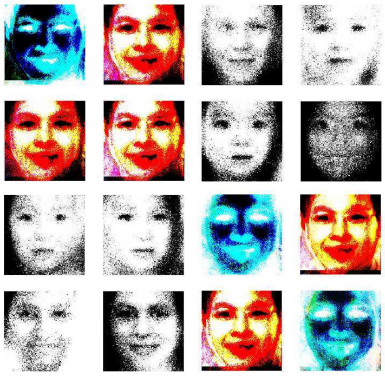


Figura 6.2: Arquitectura de GAN para *dataset* CelebA.

Con estos dos *datasets* se consiguieron resultados interesantes (al menos con CelebA) y se corrigieron varios problemas sobre colapso de moda y desvanecimiento del gradiente (véase Figura 6.3b) por medio del uso de modelos W-GAN. Se sabe que estos modelos corrigen notablemente estos errores, a expensas de entrenamientos más largos aunque tampoco excesivos, dado que se entrena un modelo no convolucional. Con este método se consiguieron resultados con mejor calidad en cuanto a estructura facial y diversidad (véase Figura 6.3a), aunque se puede observar un ruido característico en las imágenes. Esto puede deberse a que el modelo generador devuelve un vector unidimensional que luego se redimensiona a las dimensiones de la imagen, por lo que se pierde mucha información espacial.



(a) CelebA



(b) Problemas con UTKFace+

Figura 6.3: Resultados obtenidos con red W-GAN profunda.

Dado que no se consiguió un modelo de GAN profundo que ofreciese resultados fieles y realistas, se decidió utilizar modelos convolucionales (DC-GAN) tanto en el generador como en el discriminador para comprobar si estas arquitecturas ofrecían mejores resultados. Se empezó por implementar los modelos mediante la librería Tensorflow [54], la cual dio algunos problemas a la hora de entrenar modelos durante cantidades de tiempo elevadas. En la Figura 6.4 se puede ver como el generador se queda atascado intentando dibujar el pelo de las personas, perdiendo mucha calidad en el rostro y acabando por tener un colapso de moda.

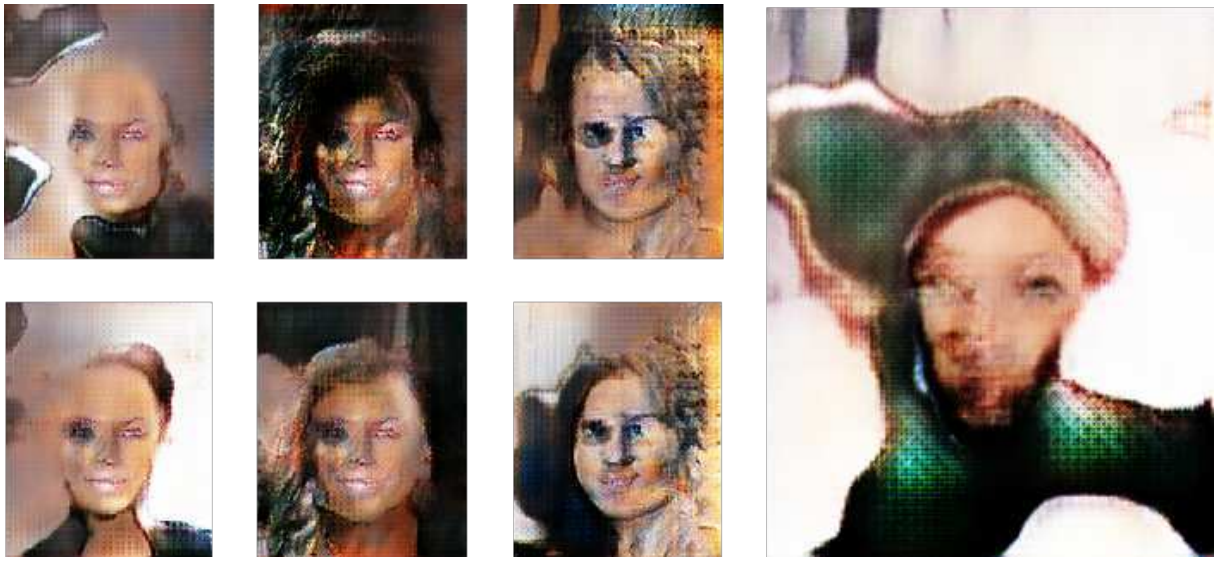
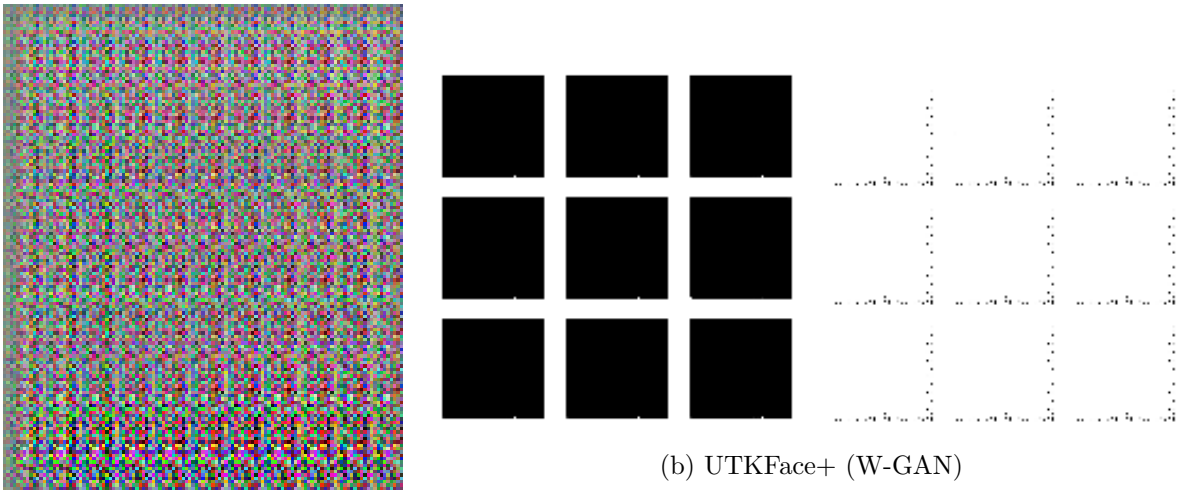


Figura 6.4: Problemas con implementaciones de redes DC-GAN en Tensorflow.

Además de estos problemas con el colapso de moda y las deformaciones del cabello, en varias ocasiones, las implementaciones con dicha librería dieron problemas complicados de resolver. Incluso usando modelos WGAN convolucionales (WDC-GAN), varias veces, el modelo se quedaba atascado en problemas de desvanecimiento de gradiente (véase Figura 6.5)



(a) CelebA

(b) UTKFace+ (W-GAN)

Figura 6.5: Desvanecimiento de gradiente con modelos implementados en Tensorflow.

Después de varios intentos infructuosos de implementar los modelos con esta librería, se decidió migrar a `PyTorch` [55]. Utilizando las mismas arquitecturas y los mismos métodos de entrenamiento pero utilizando la librería `PyTorch`, los resultados cambiaron drásticamente (véase Figura 6.6). Aunque no son resultados muy reales ni muy fieles, el modelo DC-GAN sirvió como punto de partida en la dirección correcta.

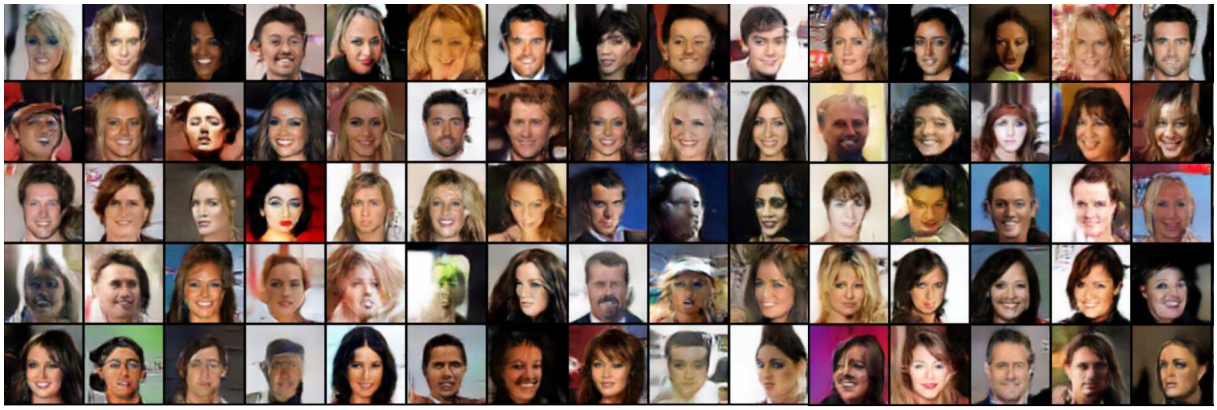


Figura 6.6: Resultados de red DC-GAN (GAN convolucional profunda con `PyTorch`).

Igual que se hizo con los modelos GAN profundos, se implementó una red WDC-GAN para probar un entrenamiento más estable, aunque más largo. En la Figura 6.7 se puede observar el progreso del entrenamiento de ésta. Después de varios entrenamientos con esta arquitectura, se fue observando que los resultados eran bastante parecidos, aunque los entrenamientos eran bastante más largos (véase Tabla 6.1).



Figura 6.7: Progreso de entrenamiento de red WDC-GAN (de izquierda a derecha).

			Modelo	
Dataset	**Resolución**	**Hardware**	DCGAN	W-DCGAN
Celeba	64x64	GPU	8h	15h 35min
UTKFace+	64x64	GPU	8h 15min	16h
Celeba	128x128	GPU	14h 10min	21h 15min

Tabla 6.1: Tiempos de entrenamiento para distintas configuraciones de redes DC-GAN.

A partir de los resultados obtenidos con este tipo de redes, se planteó la experimentación con modelos condicionados de redes generativas para ver los resultados que se podrían obtener con éstas. En el siguiente apartado se explican los experimentos realizados y los resultados conseguidos, así como los problemas enfrentados.

6.2. Experimentación con redes GAN condicionales

Vistos los resultados que se podían llegar a obtener mediante las redes DC-GAN, se siguió experimentando con ellas. Esta vez se implementaron modelos condicionales, tanto DC-GAN como WDC-GAN, para ver como se comportaban con el *dataset* de FairFace [44]. Utilizando una versión modificada de la clase `Dataset` de la librería `torch.utils.data`, se codificaron las distintas clases del *dataset* FairFace al formato *one-hot* y se concatenaron con el vector de entrada z como se ha explicado en el Apartado 3.5 y como se puede observar en su Figura 3.6.

Las primeras pruebas realizadas con las redes condicionales cWDC-GAN no dieron resultados satisfactorios en cuanto a calidad (véase Figura 6.8). Es por ese motivo, además de los altos tiempos de entrenamiento, que se decidió seguir experimentando con redes cDC-GAN, pues se podían obtener resultados con menos tiempo de entrenamiento y con calidades superiores a las de cWDC-GAN en ese mismo tiempo.



Figura 6.8: Resultados obtenidos mediante cWDC-GAN.

Después de probar varias arquitecturas, aquella que con la que se obtuvieron los mejores resultados fue la que se puede ver en la Figura 6.9. Con esta arquitectura se consiguieron resultados prometedores (véase Figura 6.10). Aunque algunas de las imágenes resultantes tenían ciertas deformidades, la mayoría de rostros empezaban a ser de calidad aceptable y tenían las características propias de su clases.

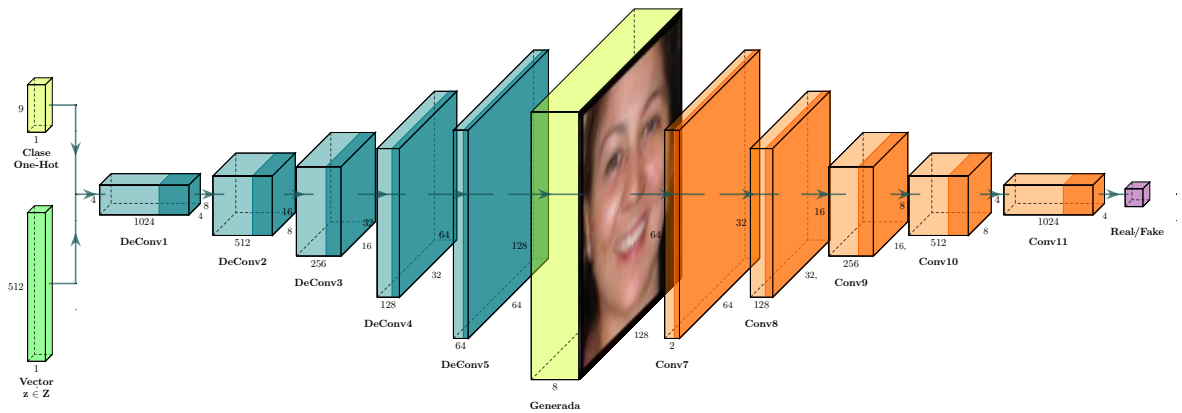


Figura 6.9: Arquitectura de red cDC-GAN.

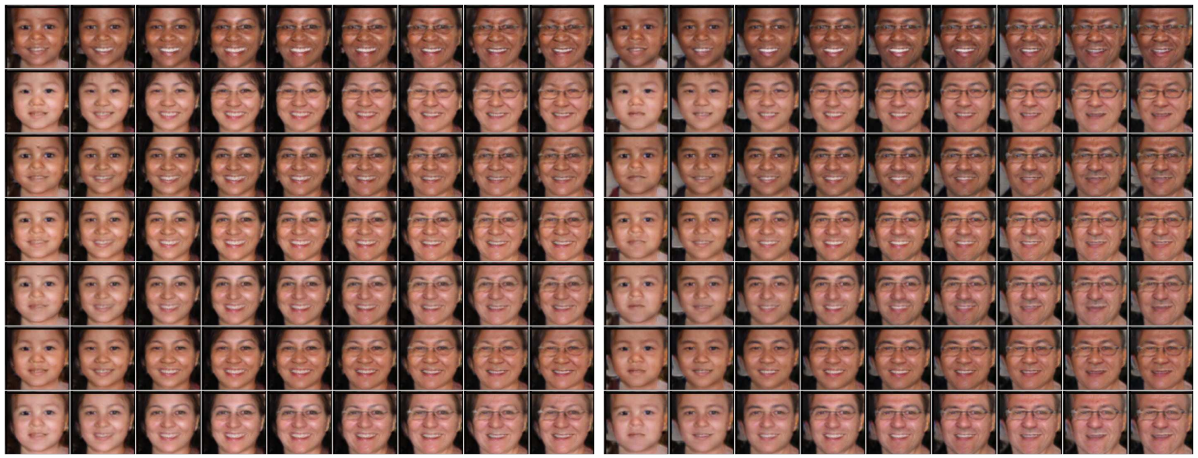
Aún así, seguía habiendo ciertos problemas con esta arquitectura. En la Figura 6.11 se puede observar cómo las imágenes van cambiando de "Edad" de izquierda a derecha y de "Raza" de



(a) Hombre asiático de 20-29 años (b) Mujer negra de 10-19 años

Figura 6.10: Resultados obtenidos mediante cDC-GAN.

arriba a abajo. Es evidente que los rasgos característicos de la demografía de dichas imágenes no es muy notoria. Las diferencias son muy sutiles y casi no se nota el cambio entre ellas. Hay que decir que, si se ignora la poca variabilidad en la clase "Raza", los resultados en general son aceptables, pero sigue haciendo falta mayor calidad y variabilidad en los datos.



(a) Mujer (b) Hombre

Figura 6.11: Resultados por "Edad" (de izquierda a derecha) y "Raza" (de arriba a abajo).

Es evidente que con estos resultados sería complicado generar datos con suficiente variación y calidad como para poder entrenar modelos de reconocimiento de expresiones, que es el objetivo principal de este trabajo. Es por ese motivo que, en el siguiente apartado, se exponen dos nuevos métodos basados en todo el conocimiento adquirido a lo largo del estudio en el estado del arte de las redes GAN y de toda la experimentación realizada anteriormente.

7. PROPUESTA

Motivado por los resultados obtenidos en los experimentos previos y dada la media-baja calidad de estos, se decidió investigar sobre métodos y arquitecturas que permitiesen obtener una calidad superior de sintetización de rostros tan complejos. Después de leer sobre este tema en numerosos artículos, se contemplaron dos métodos que compartían un mismo modelo: StyleGAN2 [1]. Como se ha explicado en el Apartado 3.9, StyleGAN2 ha sido durante varios meses el modelo más avanzado en el estado del arte (recientemente superado por modelos como StyleGAN3 [13] o DALLE-2 [56]). Es por eso que se proponen dos métodos que hacen uso de este modelo, capaz de ofrecer resultados de gran calidad a la vez que gran soltura en cuanto a la personalización de la salida de los datos dado su desentrelazado del vector latente de entrada.

Los dos métodos propuestos para generar rostros condicionados son los siguientes:

- **Método 1:** Generación de rostros mediante StyleGAN2 condicional.
- **Método 2:** Generación de rostros mediante StyleGAN2 y aprendizaje auto-supervisado contrastivo [57].

A pesar de proponer dos métodos distintos, solo se ha podido desarrollar completamente uno de los dos: el Método 1. El principal motivo es la falta de recursos en cuanto a tiempo (por sus altos costes de entrenamiento) y en cuanto a hardware, ya que en la mayoría de experimentos realizados en el artículo original del Método 2 [57] son necesarios elevados recursos de cómputo dada la complejidad de éstos. Aún así, se expone el método y los resultados que teóricamente se podrían llegar a conseguir, con sus ventajas y desventajas.

Dado que el objetivo principal del trabajo propuesto es estudiar la viabilidad de generar datos etiquetados de rostros para el reconocimiento de expresiones, cualquiera de los dos métodos intenta generar una base de datos de imágenes de rostros humanos etiquetadas con información sobre el género, la edad, la raza y la expresión. Estas imágenes podrían ser manipulables con mayor libertad que los datos originales, en cuanto a la Ley de Protección de Datos [16]. Aún así, haría falta confirmar esto con expertos en derecho ya que, aunque los datos generados no representan a ningún ser humano real, el modelo habría sido entrenado con datos reales y todavía no está claro si se podría obtener cierta información de los datos originales a partir de éste.

7.1. Método 1: Generación de rostros mediante StyleGAN2 condicional

En este método se extiende la arquitectura de StyleGAN2 para admitir la condicionalidad de la salida desde la entrada del generador. Dado que la arquitectura de StyleGAN2 es mucho más compleja que cualquiera de las implementadas en experimentos previos, para el discriminador se utiliza una función de coste diferente a la de una red C-GAN convencional. Dicha modificación del discriminador es, en comparación, más sencilla de implementar que las modificaciones del generador. Todas las modificaciones explicadas a partir de ahora, están basadas en la implementación y el informe disponibles en [58, 59].

En la arquitectura original de StyleGAN, el discriminador tiene como salida un único valor que se utiliza para saber cuanto de “real” o “falsa” es la instancia. Para obtener una versión condicional, se modifica la última capa, a la que se denota como Δ_c, para que devuelva el valor de la “veracidad” de la instancia, dado por,

$$\Delta_c(x, c) = \sum[D_{L-1}(x) \cdot l(c)], \tag{7.1}$$

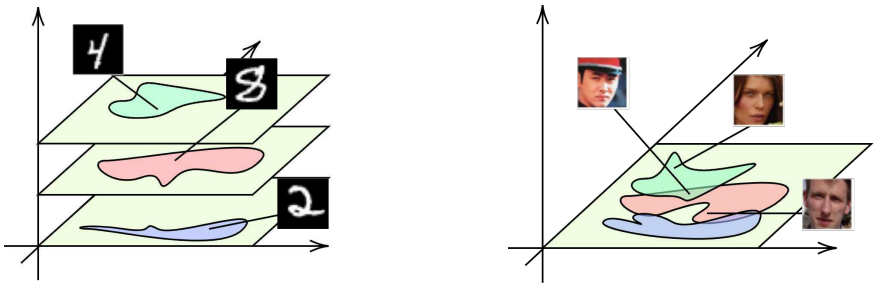
siendo $D_{L-1}(x)$ la salida del Discriminador en la penúltima capa y $l(c)$ el vector concatenado de las etiquetas en formato *one-hot*.

Como l es un vector concatenado de vectores *one-hot* de cada una de las categorías, sería equivalente a obtener la combinación de la veracidad de la clase de cada categoría. Es decir, Δ_c calcula el valor discriminatorio para cada combinación de clases de todas las categorías y solo usa la que corresponde, aunque la imagen pueda ser correcta para otra combinación. Para que se entienda mejor, he aquí un ejemplo: se tienen dos categorías "Género" con posibles clases "Hombre" y "Mujer", y "Expresión" con las posibles clases "Felicidad", "Enfado" y "Tristeza". Si las etiquetas reales de una instancia son c = ["Mujer", "Felicidad"], el resultado de concatenar las codificaciones en *one-hot* sería $l(c) = [0, 1, 1, 0, 0]$, siendo "Mujer" $= [0, 1]$ y "Felicidad" $= [1, 0, 0]$. Imagínese que el discriminador devuelve el *array* $D_{L-1}(x) = [0.1, 0.8, 0.2, 0.9, 0]$ en la penúltima capa. El resultado que devolvería la última capa, según la Fórmula 7.1, sería $\Delta_c = \sum[0, 0.8, 0.3, 0, 0] = 1.1$. Con el valor que se obtiene con los datos reales y los datos generados, se calcula la función de coste del discriminador L_D y la del generador L_G mediante una modificación de la función de coste *hinge loss* o función de coste de bisagra [60] adaptada a la función de coste adversaria de StyleGAN2:

$$L_D = \frac{\sum[\text{máx}(0, 1 + \Delta_{c_{\text{reales}}}) + \text{máx}(0, 1 - \Delta_{c_{\text{generados}}})]}{n_{\text{mini-lote}}} \tag{7.2}$$

$$L_G = \frac{\sum \Delta_{c_{\text{generados}}}}{n_{\text{mini-lote}}} \tag{7.3}$$

En cuanto al generador, hay dos maneras en las que se puede añadir la condicionalidad. Una es concatenando el vector de $l(c)$ con el vector latente z antes de la transformación $f : Z \to W$. Esto serviría únicamente si se asume que la variedad del vector latente está compuesta por subvariedades disjuntas (véase 7.1a), lo que puede ser útil en la gran mayoría de *datasets* ya que las clases suelen ser mutuamente excluyentes. El problema es que el *dataset* utilizado en este trabajo tiene clases que podrían superponerse (véase 7.1b). La segunda manera de condicionar el generador sería pasar el vector $l(c)$ a la hora de aplicar el estilo de w en cada bloque del generador. El inconveniente es que las clases podrían codificarse con múltiples elementos del vector latente, haciendo que el modelo esté más entrelazado.



(a) Sub-variedades disjuntas

(b) Sub-variedades superpuestas

Figura 7.1: Diferentes conjuntos de subvariedades.

Sea cual sea la manera en la que se condiciona el generador G, el resultado se pasa al discriminador D que devolverá el valor de Δ_c. Más adelante, se experimentará con ambas maneras para ver cual de ellas se adecúa más al *dataset* utilizado en este trabajo. En la Figura 7.2 se puede ver la arquitectura completa de principio a fin, con los distintos colores asociados a las distintas maneras de condicionar el generador (en ■ y ■) y la manera de condicionar el discriminador (en ■).

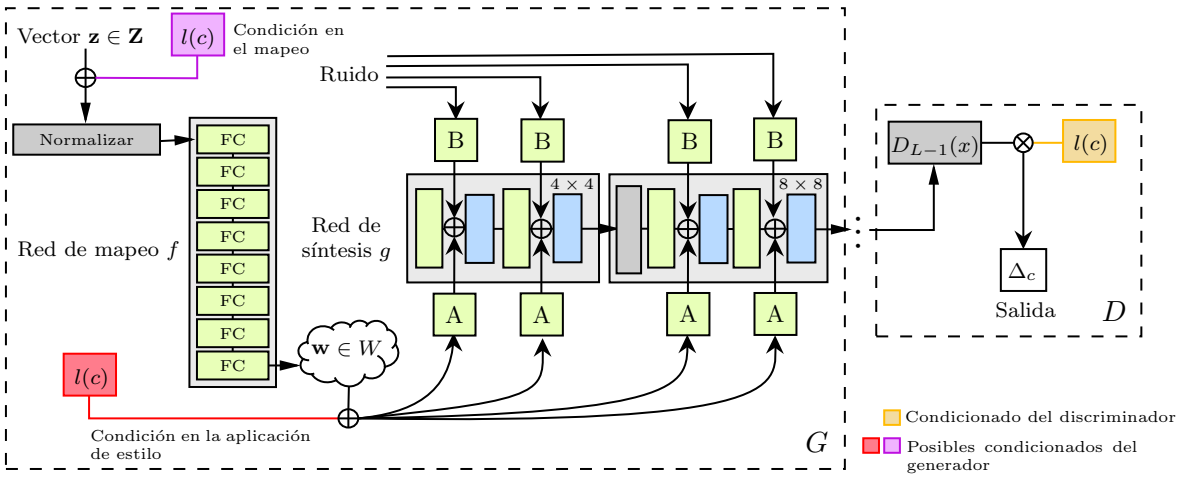
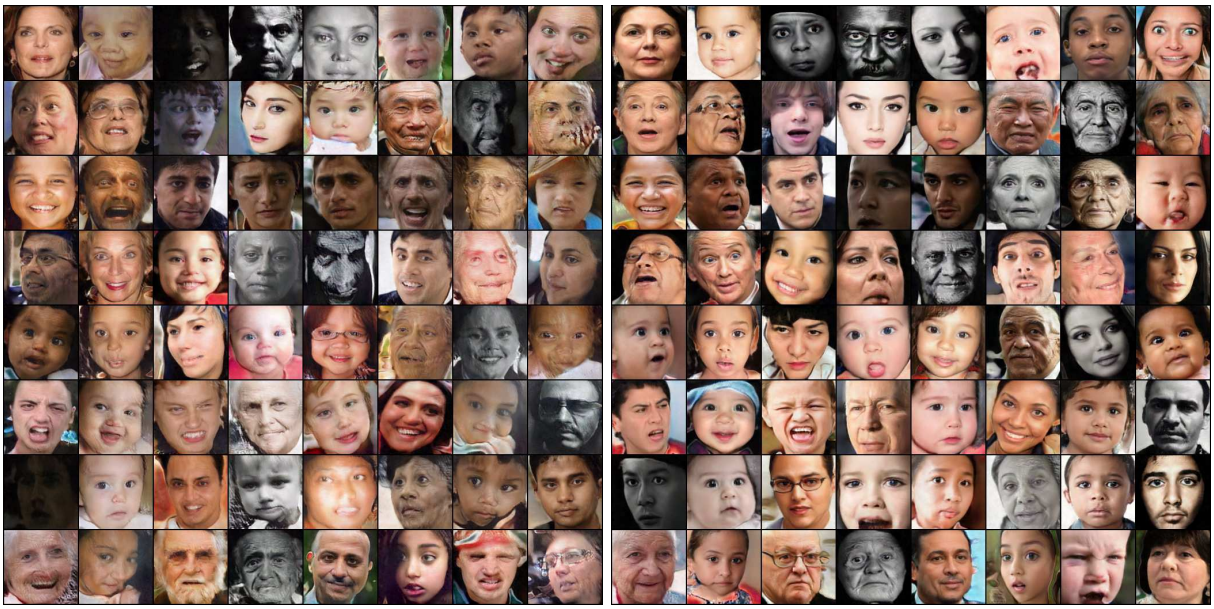


Figura 7.2: Arquitectura de StyleGAN2 condicional.

Además, para mejorar los resultados, en este método se usan las "Medias Móviles Exponenciales" (MME) con los pesos de los parámetros de la red de mapeo y el generador para estabilizar el entrenamiento y así obtener imágenes de mejor calidad. En la Figura 7.3 pueden verse las diferencias obtenidas mediante el generador con los parámetros calculados a través de este método y sin utilizar MME, con los mismos vectores latentes $z \in Z$. En general, los resultados son más realistas utilizando este método.



(a) Generador tradicional (b) Generador con MME

Figura 7.3: Diferencias entre el generador tradicional y el generador con MME.

7.2. Método 2: Generación de rostros mediante StyleGAN2 y aprendizaje auto-supervisado contrastivo

Este método está extraído del artículo *"GAN-Control: Explicitly Controllable GANs"* [57], donde se presenta un marco para el entrenamiento de GANs con control explícito sobre las imágenes generadas. Son capaces de controlar la imagen generada mediante la configuración de atributos como la edad, la pose, la expresión, etc. Se basan en el aprendizaje contrastivo, obteniendo GANs con un espacio latente explícitamente desentrelazado. Este desentrelazamiento se utiliza para entrenar codificadores de control que asignan entradas interpretables por el ser humano a vectores latentes adecuados, permitiendo así un control explícito. En este trabajo demuestran el control de la identidad, la edad, la pose, la expresión, el color del pelo y la iluminación.

El método que se propone, aún siendo simple, es bastante efectivo y consta de dos fases de entrenamiento (véase Figura 7.4):

1. **Desentrelazado por aprendizaje constrastivo:** utilizar un modelo con propiedades desentrelazantes que resulte en un espacio latente dividido en subespacios que codifican cada propiedad de la imagen. En este caso, se utiliza el modelo StyleGAN2 [1].
2. **Control explícitamente interpretable:** para cada atributo, se entrena una red para mapear los valores de control de los parámetros al subespacio latente correspondiente.

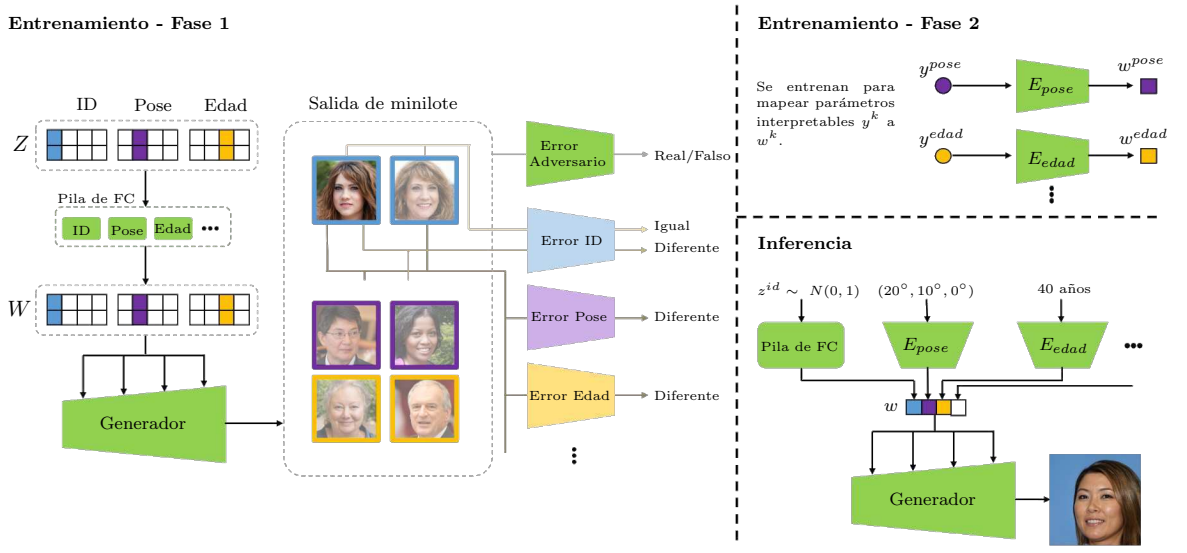


Figura 7.4: Arquitectura y fases de entrenamiento de GAN-Control (basada en [57]).

Primero, se dividen los dos espacios latentes Z y W en $N+1$ subespacios separados $\{Z^k\}_{k=1}^{N+1}$ y $\{W^k\}_{k=1}^{N+1}$ donde N es el número de atributos de control (edad, pose, expresión, etc.). Se modifica la arquitectura de StyleGAN2 para que cada atributo de control tenga su propia red de mapeo de 8 capas. Así pues, se denota $z = (z^1 z^2 \dots z^{N+1})$ y $w = (w^1 w^2 \dots w^{N+1})$ como la concatenación de los subvectores de ambos espacios latentes. Al final, el vector w se pasa al generador.

Para forzar el desentrelazado de los atributos, supóngase que $I_i = G(z_i)$ es una imagen generada por el vector latente z_i y que $B = \{z_i\}_{i=1}^{N_B}$ representa a un mini-lote de vectores de tamaño N_B. La función de coste contrastiva factorizada se define como,

$$L_c = \sum_{\substack{z_i, z_j \in B \\ i \neq j}} \sum_{k=1}^{N} l_k(z_i, z_j), \tag{7.4}$$

donde l_k es la función de coste contrastiva por atributo, la cual se define como,

$$l_k(z_i, z_j) = \begin{cases} \frac{1}{C_k^+} \max(d_k(I_i, I_j) - \tau_k^+, 0), & \text{si } z_i^k = z_j^k \\ \frac{1}{C_k^-} \max(\tau_k^- - d_k(I_i, I_j), 0), & \text{en otro caso,} \end{cases} \tag{7.5}$$

donde z_i^k denota el subvector k-ésimo de z_i, d_k es la función de distancia para el atributo k, $\tau_k^{\pm}$ son los umbrales de cada atributo asociados con los mismos y distintos subvectores y $C_k^{\pm}$ son constantes que normalizan el error acorde al número de componentes que son iguales y que son diferentes, es decir, $C_k^+ = \sum_{i,j} \mathbb{1}\{z_i^k = z_j^k\}$ y $C_k^- = \sum_{i,j} \mathbb{1}\{z_i^k \neq z_j^k\}$.

Para poder entrenar, se construyen mini-lotes que contienen pares de vectores latentes que comparten un subvector. Es decir, para cada atributo $k \in \{1, ..., N\}$ se crea un par de vectores latentes z_i y z_j donde $z_i^k = z_j^k$ y $z_i^r \neq z_j^r$ para $r \in \{1, ..., N+1 \mid r \neq k\}$. Así, la función de coste para el atributo k penalizará las diferencias entre las imágenes obtenidas de dos vectores z_i y z_j que comparten el subvector k-ésimo y penalizará también las similitudes que estos puedan tener con el resto de vectores que no compartan el subvector k-ésimo con ellos.

Para poder establecer las similitudes y diferencias para cada atributo k, se tienen varias funciones $M_k : I \to \mathbb{R}^{D_k}$ que representan a varios modelos pre-entrenados para cada uno de los atributos. Se asume que las imágenes con atributos parecidos caen cerca entre ellas y las imágenes con atributos distintos caen lejos entre ellas una vez pasadas por cada modelo. Por eso, se define la distancia entre ellas como,

$$d_k(I_i, I_j) = dist(M_k(I_i), M_k(I_j)), \tag{7.6}$$

donde $dist(\cdot, \cdot)$ es una función de distancia como L_1, L_2, distancia del coseno, etc. Para cada atributo se selecciona una función diferente. Por ejemplo, para el atributo ID, se utiliza un modelo M_{ID} que extrae vectores con los que se calcula la distancia del coseno.

Una vez terminada la primera fase del entrenamiento, se pasa a entrenar los codificadores de parámetros comprensibles por el usuario al espacio latente de w. En este caso, también se utilizan los modelos M_k para extraer un *dataset* de subvectores $\{\{w_i^k, y_i^k\}_{i=1}^{N_s}\}_{k=1}^{N}$ con los que poder entrenar los codificadores. Una vez entrenados, solo hace falta seleccionar la lista de atributos, pasar cada uno de ellos por su codificador correspondiente y concatenar sus salidas para obtener el vector latente w que se pasa al generador.

Se puede observar que la complejidad de éste método es algo más elevada que la del método anterior. Además, el modelo diseñado es bastante más grande dado que en este caso se tiene un modelo por atributo, además del discriminador adversario del modelo original. Lo cual supone un problema si no se tiene suficiente memoria para realizar el entrenamiento.

8. EXPERIMENTACIÓN Y RESULTADOS

En este apartado se exponen los resultados obtenidos con la implementación y la experimentación del Método 1, al igual que los problemas que se han tenido a la hora de entrenar con distintas configuraciones. En cuanto al Método 2, como se ha explicado anteriormente, no ha habido suficientes recursos en cuanto a tiempo y hardware para poder probarlo. Aún así, más adelante se discute sobre los resultados que teóricamente podrían conseguirse (al igual que los resultados obtenidos del Método 1) y sobre las diferentes ventajas y desventajas.

Para poder implementar el modelo planteado en el Método 1, basado en el código disponible en [58, 59], primero hubo que modificar el código de la clase `Dataset` desarrollado en el Apartado 6.2 para adecuarse al código base, además de realizar las modificaciones que se plantean en el Método 1. Una vez implementado el código y asegurándose que todo funcionaba correctamente se pasó a la experimentación. Hay que aclarar que todos los modelos se entrenan desde cero.

8.1. Experimento 1

En primer lugar se probó a entrenar el modelo con todas las imágenes del *dataset* recortadas y centradas (de la partición de entrenamiento) con la configuración de híper-parámetros de la Tabla 8.1. El híper-parámetro "Condición en mapeo" hace referencia al método de condicionar al generador, siendo "True" el método de concatenar las etiquetas al vector $z \in Z$ y "False" el método de aplicar las etiquetas después de generar el vector $w \in W$ y aplicarlo a cada bloque de estilo del generador (en la Figura 7.2 "True" en ▭ y "False" en ▭). En este caso se entrena un modelo con el valor a "False" para ver si el modelo es capaz de separar las clases de las categorías que podrían solaparse sin entrelazar demasiado los vectores latentes con los datos de entrenamiento y las etiquetas de estos. El híper-parámetro "Acumulado gradiente" establece el número de veces en las que se calcula el gradiente antes de actualizar los parámetros del modelo.

Híper-parámetro	**Valor**	**Híper-parámetro**	**Valor**
Tamaño del mini-lote GPU	8	Tamaño de imagen	128
Tamaño del mini-lote gradiente	64	Canales	3
Acumulado gradiente	4	Capacidad de red	16
Condición en mapeo	False	Profundidad de estilo	8
Iteraciones	110K	Ratio de aprendizaje	2e-4
Tiempo de entrenamiento:			67h 34m 11s

Tabla 8.1: Híper-parámetros del Experimento 1.

En la Figura 8.1 se pueden ver algunos de los resultados obtenidos por condicionado aleatorio para ver como se comporta en cuanto a variedad y calidad de los datos. Se puede observar como los marcos negros, producto del recentrado y reposicionamiento de los rostros del *dataset*, hacen que bastantes de los ejemplos tengan colores más oscuros como marcos, iluminación oscura, imágenes en blanco y negro y, curiosamente, incluso gafas de sol oscuras. Esto puede deberse a que al no añadir las etiquetas en la red de mapeo, las etiquetas deben entrelazarse con los atributos del vector latente $w \in W$, haciendo que las instancias que tengan recuadros negros obliguen al generador a generar estos "artefactos" negros para seguir engañando al discriminador. Además de los "artefactos" que oscurecen las imágenes, se encuentran también ciertos ejemplos con unas deformaciones recurrentes en el área de la boca (véase Figura 8.2), pudiendo ser por el mismo motivo que antes. Es decir, puede ser debido al recentrado de las imágenes ya que el



Figura 8.1: Resultados del Experimento 1 con condicionado en estilo y rostros centrados.

generador intenta generar datos para rellenar esas zonas de las imágenes a las que les falta la zona de la boca. Además, como el modelo parece estar entrelazando algunos de los atributos con los vectores $w \in W$, es posible que lo haga de la misma manera para varias de las instancias que comparten estas características.



Figura 8.2: Imágenes generadas con deformaciones en el área de la boca.

8.2. Experimento 2

Por estos motivos, se plantea experimentar con el *dataset* de imágenes sin recorte y recentrado para ver si los resultados mejoran. Dado que, en definitiva, es preferible tener un generador que devuelva instancias sin esos marcos negros a los lados de la imagen. También, se realizan los siguientes experimentos con el método de condicionado por mapeo (en la Figura 7.2). Así pues, la configuración para el siguiente experimento se presenta en la Tabla 8.2.

Híper-parámetro	**Valor**	**Híper-parámetro**	**Valor**
Tamaño del mini-lote GPU	8	Tamaño de imagen	128
Tamaño del mini-lote gradiente	64	Canales	3
Acumulado gradiente	4	Capacidad de red	16
Condición en mapeo	True	Profundidad de estilo	8
Iteraciones	200K	Ratio de aprendizaje	2e-4
Tiempo de entrenamiento:			93h 42m 10s

Tabla 8.2: Híper-parámetros del Experimento 2.

En la Figura 8.3 se puede observar que los resultados son, en general, algo más variados que en el Experimento 1. Además, las imágenes parecen tener menos deformidades y parece que las etiquetas no se están entrelazando como en el método de condicionado anterior.



Figura 8.3: Resultados del Experimento 2 con condicionado en mapeo.

Al igual que en el Experimento 1, se usaron todos los datos del *dataset*. Por lo que el modelo resultante contiene el mismo sesgo que tienen los datos. En la Figura 8.4 se puede ver como las instancias generadas de aquellas clases que tienen menos ejemplos en el *dataset* son de menor calidad, debido a la poca información que puede extraer el modelo de la cantidad de datos para esas clases. En este caso, la Figura 8.4a contiene ejemplos generados para la clase "Felicidad" y con edades de 10 a 40 años que, como se puede ver en el Apartado 4.6, son de las clases que más instancias tiene el *dataset*. A diferencia de ésta, la Figura 8.4b contiene ejemplos generados para la clase "Miedo" con edades de entre 50 y 70+ años que son de las clases con menos instancias del *dataset*.



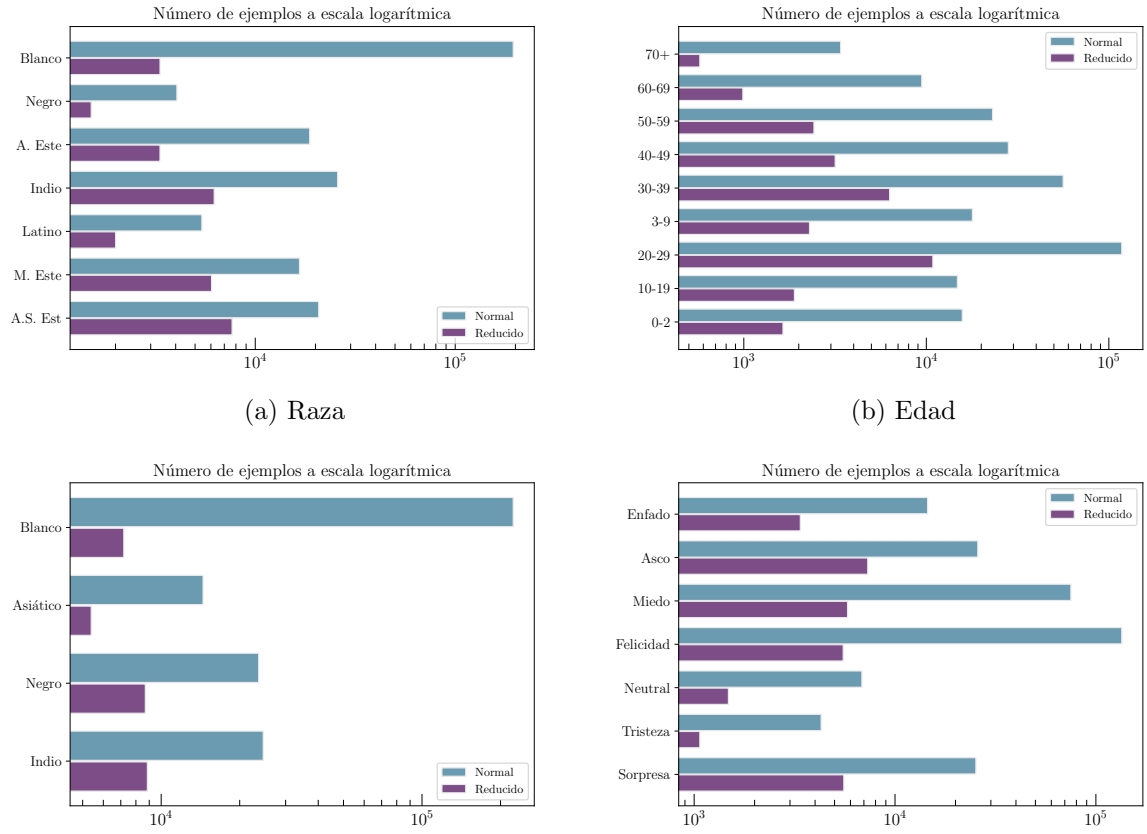
(a) Clase "Felicidad" (b) Clase "Miedo"

Figura 8.4: Diferencias en calidad de ejemplos por clases desbalanceadas.

Es por eso que en siguientes experimentos se plantea limitar el número de ejemplos por clase para así intentar mitigar el sesgo en el entrenamiento. Aunque se prevé un decremento en la calidad (FID mas alto) de los modelos debido a que se reduce el número de datos con los que se entrena el modelo.

8.3. Experimento 3

En este experimento se prueba a intentar mitigar el sesgo inherente de las etiquetas demográficas de AffectNet [42] reduciendo las instancias de aquellas clases cuyo número de instancias es más alto. Para reducir estas instancias, se calcularon las intersecciones de las clases con más ejemplos (por ejemplo las clases "Blanco", "Felicidad" y "20-29", entre otras combinaciones) y se marcaron para su eliminación. En la Figura 8.5 se puede observar este cambio en el número de ejemplos por clase antes y después del intento de mitigar el sesgo. Hay que tener en cuenta que las gráficas están representadas con el eje X en escala logarítmica porque los cambios en el número de datos eran muy grandes. El problema de reducir instancias en un *dataset* con tantas categorías es que al cambiar el número de instancias de una de las categorías las demás también cambian. Por eso se intentó ser lo más precavido posible con el número de instancias eliminadas de cada una de las clases de cada categoría para no cambiar demasiado las demás. Aún así, la reducción de datos fue sustancial. Se pasan de tener 282 853 imágenes a 30 126 imágenes, una reducción de más o menos un 89,349 %.



(a) Raza

(b) Edad

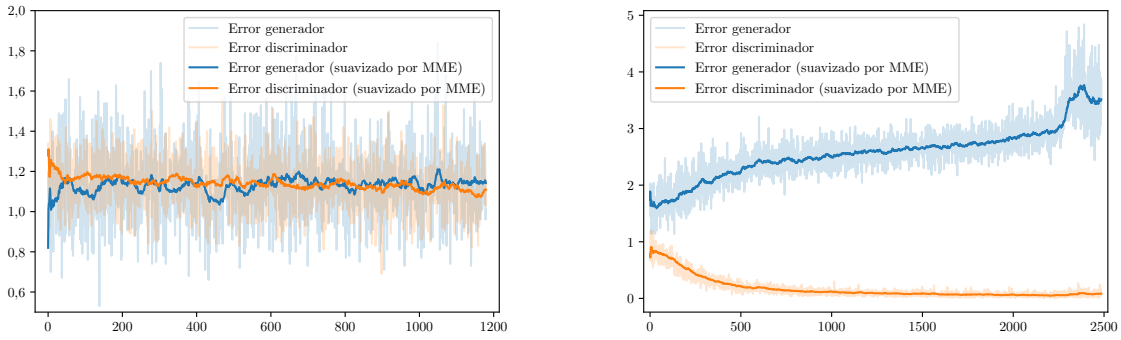
(c) Raza 4

(d) Expresión

Figura 8.5: Número de ejemplos antes y después de la reducción de instancias.

En cuanto a la configuración de híper-parámetros, en este experimento se utiliza la misma tabla que en el Experimento 2 (Tabla 8.2). Al entrenar el modelo con esta configuración de datos se observa que el entrenamiento es algo más inestable y propenso a caer en problemas de colapso de moda. Puede que se deba a la cantidad reducida de datos con las que se entrena. En definitiva se puede observar como el error del generador no deja de aumentar por mucho que pasen las iteraciones (véase Figura 8.6b) a diferencia de en el Experimento 2, donde los errores del generador como el del discriminador parecen estar estabilizados a lo largo del entrenamiento

(véase Figura 8.6a). Esto puede deberse también a que el discriminador del Experimento 3 se vuelve "experto" en identificar los ejemplos del generador, impidiendo que éste logre engañarlo. Aún así, las gráficas de los errores del modelo no aportan la información necesaria sobre los datos generados.



(a) Experimento 2

(b) Experimento 3

Figura 8.6: Valores de la función de coste adversario durante el entrenamiento.

Por eso, en la Figura 8.7 se pueden ver, al igual que se han mostrado en apartados anteriores, ejemplos aleatorios ofrecidos por el generador. Se puede apreciar claramente el colapso de moda de éste ya que la mayoría de las imágenes están repetidas y comparten características similares, aunque todos los vectores latentes $z \in Z$ de entrada sean aleatorios.

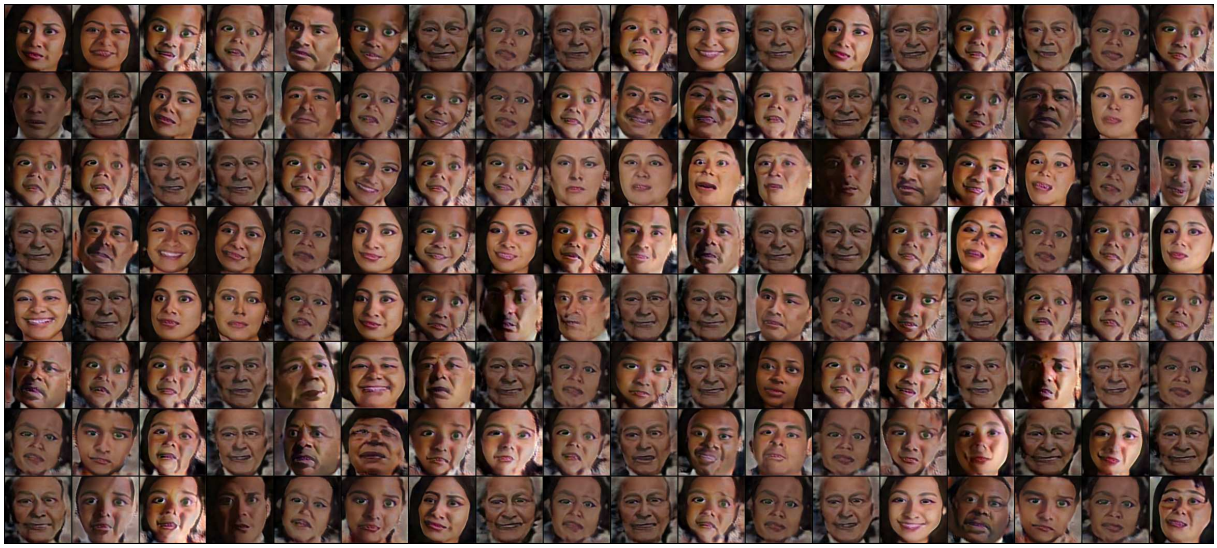


Figura 8.7: Resultados del Experimento 3 con claro colapso de moda.

8.4. Experimento 4

En este experimento se plantea entrenar un modelo mezclando las configuraciones de datos del Experimento 2 y el Experimento 3 de manera gradual. Es decir, al principio del entrenamiento se utilizan todos los datos y a lo largo del entrenamiento se van reduciendo los ejemplos de las clases hasta que los datos queden como en el Experimento 3. Con eso se intenta reducir el sesgo de manera gradual para que el modelo, en las primeras iteraciones, tenga acceso a más datos y se reduzcan las probabilidades de que caiga en los mismos problemas que ocurren en el Experimento

3. Además, se plantea utilizar una configuración diferente en la que la principal diferencia es la resolución de las imágenes. En este caso se pasa a entrenar al modelo con imágenes de 256×256 píxeles para intentar ofrecer más información al modelo. A causa de esto, es necesario reducir algunos de los híper-parámetros como la profundidad o capacidad del modelo y el tamaño del mini-lote de la GPU. Esto se debe a que cuanto más grande es la imagen, más grande se vuelve el modelo y más ocupa en la GPU a la hora de entrenar.

Así pues, la configuración de híper-parámetros para este experimento se muestra en la Tabla 8.3.

Híper-parámetro	**Valor**	**Híper-parámetro**	**Valor**
Tamaño del mini-lote GPU	2	Tamaño de imagen	256
Tamaño del mini-lote gradiente	32	Canales	3
Acumulado gradiente	4	Capacidad de red	8
Condición en mapeo	True	Profundidad de estilo	8
Iteraciones	200K	Ratio de aprendizaje	2e-4
Tiempo de entrenamiento:			110h 28m 3s

Tabla 8.3: Híper-parámetros del Experimento 4.

Aunque fue necesario por motivos de capacidad del hardware, el problema en esta configuración está en el valor de la capacidad de la red. En este caso se reduce a la mitad del valor de los demás experimentos. Según los autores de StyleGAN2, reducir la capacidad de la red repercute directamente en la calidad de las imágenes generadas. Por lo tanto, se espera una reducción en la calidad de las imágenes.

En la Figura 8.8 se pueden ver, como en experimentos anteriores, ejemplos aleatorios que ofrece el modelo generador (se ofrecen más resultados de este modelo en el Anexo A). Por lo que se puede observar en ésta, aunque se esperaban resultados de peor calidad, la verdad es que la mayoría de las imágenes son de una calidad más que aceptable. Habrá que ver al comparar los modelos de los anteriores experimentos mediante las métricas explicadas anteriormente, si vale la pena intentar generar imágenes de 256×256 píxeles o si con generar imágenes de 128×128 píxeles es suficiente.



Figura 8.8: Resultados del Experimento 4 con tamaño 256×256.

8.5. Extracción de métricas

Para poder comparar más adelante varios de los experimentos realizados, es necesario extraer primero las métricas de los *checkpoints* guardados de cada uno de los experimentos, los cuales se guardan cada 250 iteraciones. Para ello se propone utilizar las métricas de *precision*, *recall* y FID. Como se ha comentado antes, el FID da un valor condensado de la calidad y la variabilidad de los datos generados con respecto a los datos reales. Por eso se utiliza la *precision* y el *recall*, para separar estos dos atributos y poder comparar los *checkpoints* de una manera más precisa acorde a lo que es necesario comparar. En este caso, se prefiere un modelo que sea capaz de generar instancias de alta calidad a la vez que de alta variación. Pero si se quiere, también se puede elegir un modelo que ofrezca más variación o más precisión, dependiendo de qué se quiera lograr. Para ello, se han calculado los valores de *precision* y *recall* de los *checkpoints* guardados en los Experimentos 2, 3 y 4.

En cuanto a la implementación del cálculo de estas métricas, al principio se empezó usando una versión modificada del código disponible en [61] para las métricas de *precision* y *recall*, cuya implementación está basada en el artículo original [34]. El único problema con esta implementación es que era extremadamente ineficiente si se quieren extraer dichas métricas para tantos *checkpoints*. Por ello se cambió la función `manifold_estimate` que calcula las dos métricas dependiendo del orden de los argumentos. En el Código 8.1 se presenta el código original de [61] quitando las llamadas a funciones innecesarias y modificándolo para adecuarse al código implementado.

```python
def manifold_estimate(A_features, B_features, k=3):
    KNN_list_in_A = {}
    # Calcula la matriz de distancias por pares de A_features.
    for aa, A in enumerate(A_features):
        pairwise_distances = np.zeros(shape=(len(A_features)))

        for i, A_prime in enumerate(A_features):
            d = torch.norm((A - A_prime), 2) # Calcula la distancia de A a A_prime.
            pairwise_distances[i] = d

        v = np.sort(pairwise_distances)[k] # Obtiene la distancia al vecino k más cercano a A.
        KNN_list_in_A[aa] = v
    n = 0 # Contador.
    # Comprueba si B es más cercano que el k-vecino de algún A_prime de A_features.
    for bb, B in enumerate(B_features):
        for aa, A_prime in enumerate(A_features):
            d = torch.norm((B - A_prime), 2) # Calcula la distancia de B a A_prime.
            if d <= KNN_list_in_A[aa]:
                n += 1
                break # Si encutra al menos uno, sale del bucle.

    return n / len(B_features)
```

Código 8.1: Código basado en el método `manifold_estimate` de [61].

El tiempo medio de ejecución de esta función es de más o menos 100-120 minutos para 10 000 imágenes. Para mejorar este tiempo se pensó en utilizar la clase `NearestNeighbors` y la función `pairwise_distances` de la librería `sklearn` (Scikit learn) ya que permitiría usar su *backend* de

multithreading para acelerar el proceso. Una vez calculados los k vecinos más cercanos de la matriz de distancias por pares de A, solo habría que calcular la matriz de distancias entre las características de B y las características de A. Después, bastaría con restar a cada fila de esa matriz el *array* de distancias de los k-vecinos calculado antes para ver cuantas de esas distancias son negativas o iguales a 0. Si al menos una cumple esto, se marca la característica B como verdadera. Al final, solo hay que sumar el *array* de booleanos y dividir entre el número de características. Con esta implementación, se reduce el tiempo de ejecución a 2-3 minutos. El único inconveniente es que en este caso se calculan todos los ejemplos aunque en el algoritmo propuesto en [34] se dice que basta con que $\exists\ \phi' \in \Phi : \parallel \phi - \phi' \parallel_2 \leq \parallel \phi' - NN_k(\phi', \Phi) \parallel_2$. Aún así, se procede con esta implementación ya que consigue reducir el tiempo de ejecución de manera sustancial. En el Código 8.2 se presenta la implementación de la función modificada.

```python
def manifold_estimate(A_features, B_features, k=3):
    KNN = NearestNeighbors(metric="precomputed", n_neighbors=k)
    # Al usar la métrica "precomputed", espera una matriz de distancias.
    # Por eso se usa la función pairwise_distances de sklearn.
    KNN_list_in_A = KNN.fit(pairwise_distances(A_features, n_jobs=-1)).kneighbors()[0][:, k-1]
    B_A_distances = pairwise_distances(B_features, A_features, n_jobs=-1)
    Near_B_A_k = np.sum((B_A_distances - KNN_list_in_A) <= 0, axis=1)
    n = np.sum(Near_B_A_k > 0)

    return n / len(B_features)
```

Código 8.2: Código modificado de `manifold_estimate`.

Para calcular el FID, se calcula la formula presentada en el Apartado 3.3 con la función del Código 8.3.

```python
def calculate_fid(real_embeddings, generated_embeddings):
    mu2, sigma2 = generated_embeddings.mean(axis=0), np.cov(generated_embeddings, rowvar=False)
    mu1, sigma1 = real_embeddings.mean(axis=0), np.cov(real_embeddings, rowvar=False)
    ssdiff = np.sum((mu1 - mu2) ** 2.0)
    covmean = linalg.sqrtm(sigma1.dot(sigma2))
    if np.iscomplexobj(covmean):
        covmean = covmean.real

    fid = ssdiff + np.trace(sigma1 + sigma2 - 2.0 * covmean)

    return fid
```

Código 8.3: Código para calcular el FID.

Así pues, una vez implementadas las funciones y viendo que devuelven los mismos valores que la implementación original, se pasó a calcular las métricas de los *checkpoints* de los experimentos mencionados. Al tener tantos *checkpoints* por experimento (entre 500 y 780 *checkpoints*) en cada uno de ellos se escogió a mano un rango de ±200 *checkpoints*. Para cada uno de ellos se calcula el FID, la *precision* y el *recall* con 10 000 imágenes generadas y 10 000 imágenes reales. El tiempo de ejecución por modelo es aproximadamente de 9-10 minutos. Por lo que el tiempo total de extracción de métricas fue de ±96 horas (±4 días).

8.6. Comparación de resultados

Una vez extraídas las métricas de los *checkpoints*, es hora de compararlos entre ellos para seleccionar el mejor. Para ello, se presenta la Figura 8.9 con una gráfica de la *precision* frente al *recall* de cada uno de los *checkpoints* seleccionados de los Experimentos 2 (con sesgo), 3 (sesgo mitigado) y 4 (sesgo mitigado gradual).

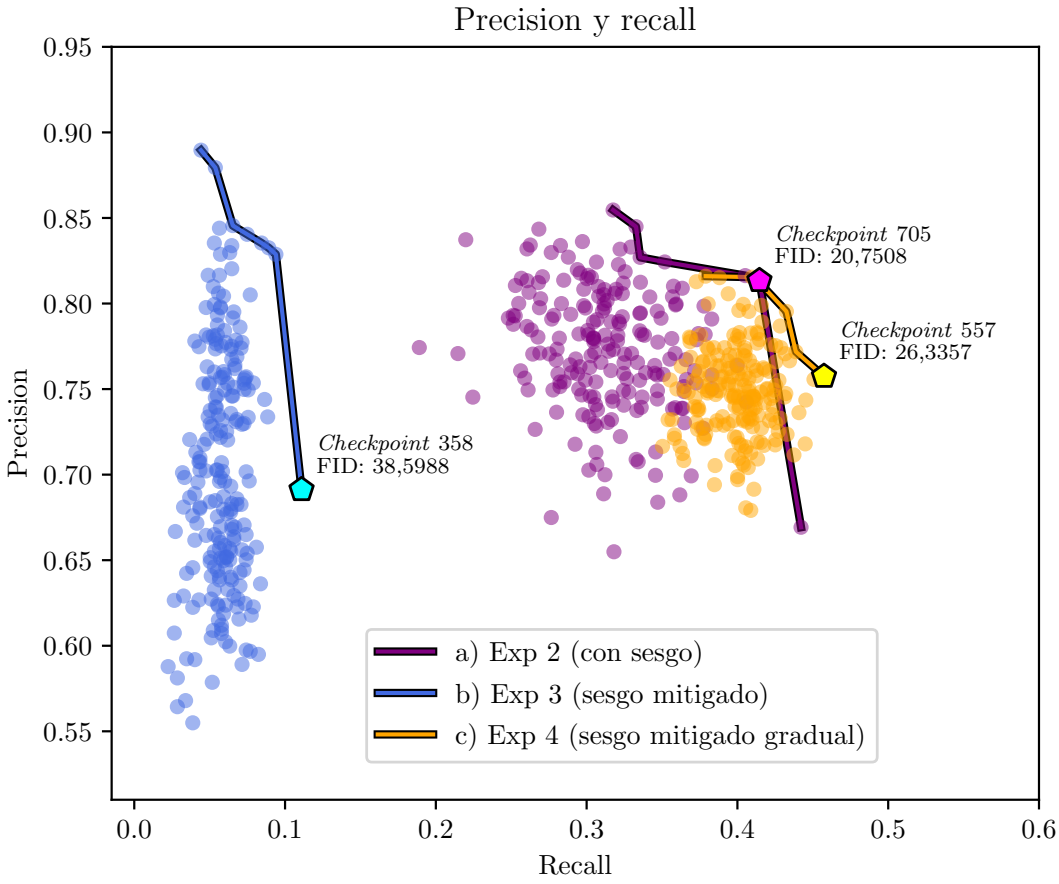


Figura 8.9: *Precision* y *recall* de los *checkpoints* seleccionados en experimentos 2, 3 y 4.

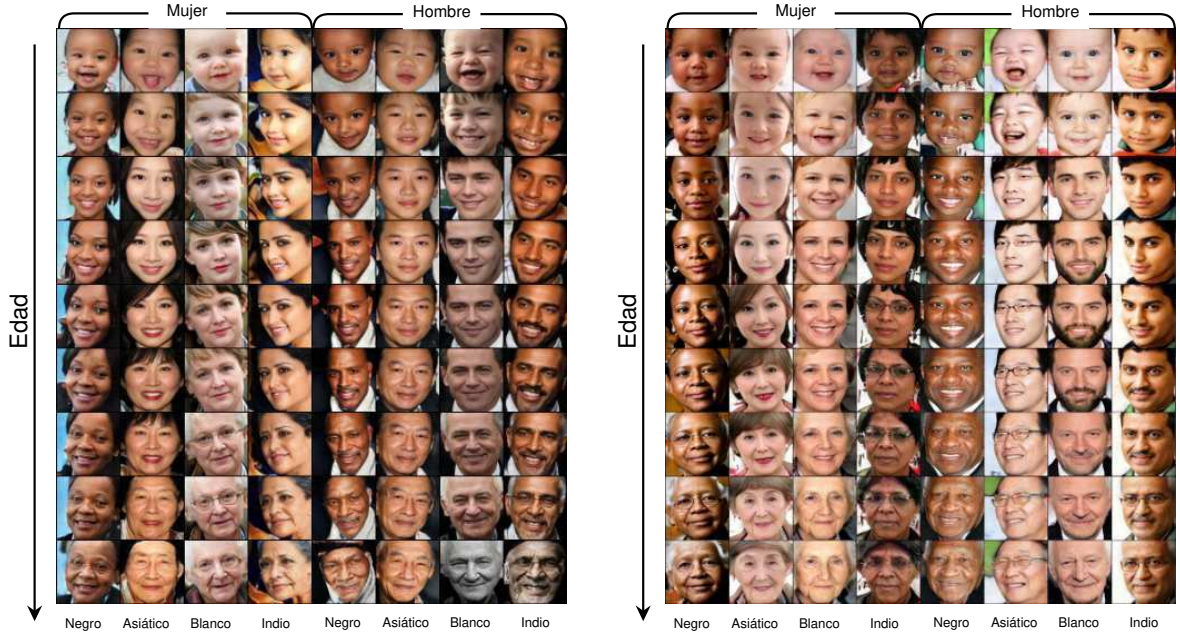
Los puntos con forma pentagonal que aparecen en la Figura anterior representan el mejor *checkpoint* en cuanto a balance de *precision* y *recall*, calculado con la fórmula de $F_1 = \frac{2 \cdot P \cdot R}{P+R}$. En ellos se marca también el valor de FID del *checkpoint*. En cuanto a las líneas de cada nube de puntos, representan la frontera de Pareto superior del valor de F_1.

Es evidente que el Experimento 3 (donde se intentó eliminar el sesgo en los datos) ha resultado, en general, en una configuración más inestable en cuanto a *precision* y sobre todo en cuanto a los resultados en *recall*, los cuales son muy bajos. Por lo que se descartan definitivamente los *checkpoints* de éste. En cuanto a los Experimentos 2 y 4, parecen tener valores parecidos en *precision*. Pero en términos de *recall*, el Experimento 4 tiene los mejores valores. En la Tabla 8.4 se puede ver en negrita los mejores valores en base a cada métrica. El mejor *checkpoint* del Experimento 2 obtiene el menor valor en FID y el mayor valor en *precision* a diferencia del mejor *checkpoint* del Experimento 4, el cual obtiene los mejores valores en *recall* y F_1. Dado que se obtienen conclusiones contradictorias en cuanto a cuál de los dos es el mejor basándose en los valores en F_1 y FID de ambos experimentos, se plantea ahondar en el comportamiento de éstos a la hora de generar imágenes de manera condicionada.

Experimento	**FID** ↓	**Precision** ↑	**Recall** ↑	$\mathbf{F_1}$ ↑
2 (con sesgo)	$\mathbf{20{,}7508 \pm 0{,}5631}$	$\mathbf{0{,}8134 \pm 0{,}0052}$	$0{,}4146 \pm 0{,}0072$	$0{,}5492 \pm 0{,}0057$
3 (sesgo mitigado)	$38{,}5988 \pm 0{,}8482$	$0{,}6912 \pm 0{,}0099$	$0{,}1109 \pm 0{,}0032$	$0{,}1912 \pm 0{,}0051$
4 (sesgo mitigado gradual)	$26{,}3357 \pm 0{,}2772$	$0{,}7574 \pm 0{,}0058$	$\mathbf{0{,}4574 \pm 0{,}0087}$	$\mathbf{0{,}5704 \pm 0{,}0081}$

Tabla 8.4: Métricas de los mejores *checkpoints* de cada experimento.

Así pues, a continuación se muestran ejemplos de cada modelo con etiquetas específicas para ver manualmente si la diferencia en la calidad y variación de datos es claramente visible o si se puede pasar a la siguiente fase con ambos para ver cual de los dos se comporta mejor a la hora de entrenar modelos de reconocimiento de expresiones faciales. En la Figura 8.10 se pueden ver ejemplos variados controlados por etiquetas en los que cada columna se mantiene el mismo vector latente z, variando su edad por filas. En ambas subfiguras, las primeras cuatro columnas son imágenes con la clase "Mujer" en la categoría "Género" y las otras cuatro columnas con la clase "Hombre". A su vez, para cada conjunto de cuatro columnas (para cada género) se han establecido las etiquetas de raza con la forma ["Raza 4", "Raza"] de la siguiente manera: [["Negro, Negro"], ["Asiático", "Asiático del Este"], ["Blanco", "Blanco"], ["Indio", "Indio"]].



(a) Experimento 2

(b) Experimento 4

Figura 8.10: Ejemplos con etiquetas específicas.

Ambos modelos ofrecen un control bastante bueno con las etiquetas y se puede ver que al cambiar la edad de cada instancia los demás atributos no cambian demasiado a excepción de las edades extremas, seguramente debido al desbalanceo en las clases.

Como se puede observar, las diferencias en cuanto a la calidad de los resultados de ambos experimentos no son demasiado notables a la vista. Lo único que se ha observado a lo largo de numerosas pruebas (generando datos con esta configuración y muchas otras de manera repetida) es que las resultados del Experimento 4 tienden a ser más variados y los ejemplos de clases minoritarias suelen ser de mejor calidad y variedad. A pesar de ello, para la siguiente fase se plantea utilizar ambos modelos.

8.7. Resultados teóricos del Método 2

Como se ha explicado antes, aunque no se haya podido implementar el Método 2, se comentan los resultados teóricos que se podrían conseguir una vez implementado el modelo propuesto y entrenado con el mismo *dataset* seleccionado. Basándose en los resultados obtenidos de los modelos pre-entrenados de la implementación oficial del artículo *"GAN-Control: Explicitly Controllable GANs"* [57] representados en la Figura 8.11, se puede ver que se consiguen ejemplos de muy alta calidad a la vez que el nivel de control de los atributos es excepcional.

En cuanto a las etiquetas de "Emoción", en realidad las imágenes no se generan introduciendo la etiqueta de esa manera. El generador necesita parámetros parecidos a 3DMM [62] de 64 dimensiones. Ofrecen un fichero con 1000 *arrays* de emociones codificadas de esta manera, aunque la gran mayoría de ellos se podrían atribuir a la clase "Felicidad". Para estos ejemplos se fue probando manualmente cada uno de los 1000 ejemplos disponibles hasta encontrar ejemplos parecidos a cada clase que se presenta en la Figura 8.11. Dada la lentitud de generación de las imágenes (por ser de 512×512 píxeles) y que la evaluación manual de las expresiones es bastante compleja, el tiempo total de estas pruebas fue de ± 2 días. Gracias a esas pruebas manuales se pudo observar que faltaba mucha variedad en la mayoría de emociones que no eran de la clase "Felicidad".

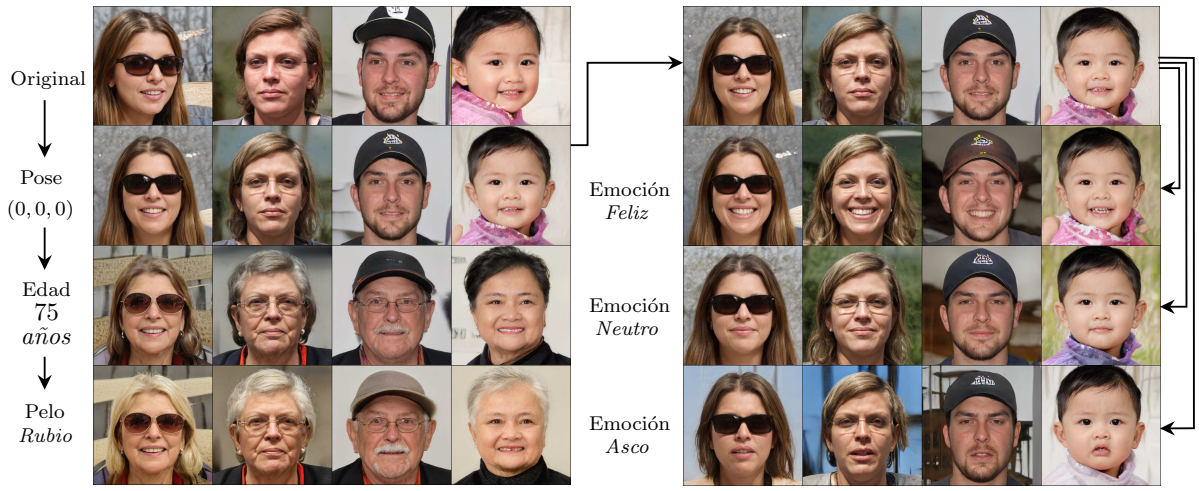


Figura 8.11: Ejemplos conseguidos con modelos pre-entrenados de GAN-Control [57].

Hay que aclarar que estos datos son resultado de la implementación oficial del modelo entrenado con datos del *dataset* FFHQ [31]. Por lo tanto, sus resultados no son completamente comparables con los resultados obtenidos por el Método 1 implementado en este trabajo. Aún así, suponiendo que se pueden llegar a obtener resultados medianamente parecidos a estos últimos, este método se considera, en teoría, mejor que el primero. El único inconveniente de éste son sus altos requisitos en cuanto a hardware y tiempo para poder entrenar el modelo de manera eficiente y según las recomendaciones del artículo oficial. Además, aunque se podrían haber usado los modelos pre-entrenados, el control de las emociones de cada ejemplo no es tan evidente de controlar y tampoco se pueden controlar ni la raza ni el género.

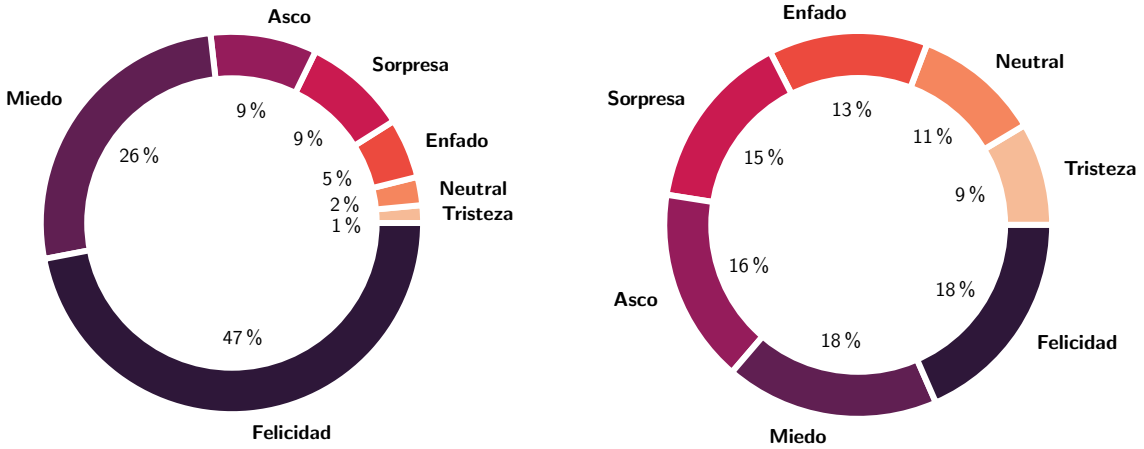
Así pues, a pesar de no poder utilizar modelos reales del Método 2, una vez se han seleccionado los mejores modelos del Método 1, se pasa a la siguiente fase para responder a la pregunta que se plantea al principio del trabajo: ¿Es viable y útil generar datos sintéticos para entrenar modelos de reconocimiento de expresiones faciales?

9. COMPARATIVA DE DATOS Y DISCUSIÓN

Para poder responder a la pregunta planteada antes, es necesario entrenar modelos de reconocimiento de emociones con los datos reales y los datos generados por los dos modelos seleccionados para ver si estos últimos pueden competir con los datos reales. Para ello, se utiliza un modelo ResNet [63] de 50 bloques residuales con ambos conjuntos de datos. Se decidió utilizar este modelo ya que se trata de un modelo convolucional muy utilizado y con buenos resultados en una gran variedad de problemas. Además, se utiliza el optimizador DiffGrad [23] con los valores $\beta_1 = 0{,}5$ y $\beta_2 = 0{,}9$. Como función de coste, se utiliza la entropía cruzada categórica explicada en el Apartado 2.4.3.

Hay que aclarar que el propósito de esta fase es comparar el comportamiento de los datos generados frente a los datos reales. No se pretende buscar un modelo que compita con el estado del arte en este área ya que demandaría mucho tiempo y sería suficiente por sí solo para un Trabajo Fin de Grado nuevo (o incluso un trabajo de mayor nivel).

En cuanto a la generación de datos, se han probado dos maneras. La primera trata simplemente de extraer las etiquetas del *dataset* original y generar los datos basándose en ellas. Así, el modelo generará datos con el mismo desbalanceo entre clases. La segunda trata de obtener las etiquetas únicas del *dataset* original e intentar generar un número balanceado de ejemplos en comparación a cómo lo estaban antes. Es decir, el *dataset* tiene un total de 2337 etiquetas diferentes (por las numerosas combinaciones de los atributos demográficos con las etiquetas de "Expresión"), las cuales están distribuidas por expresión de la siguiente manera: ["Enfado": 310, "Asco": 379, "Miedo": 417, "Felicidad": 431, "Neutral": 248, "Tristeza": 201, "Sorpresa": 351]. Al repetir estas etiquetas de manera lineal hasta sobrepasar el número de ejemplos total del *dataset* se eligen aleatoriamente 282 853, que corresponde al número de ejemplos total de la partición de entrenamiento. Así pues, la distribución de un entrenamiento de ejemplo pasaría a ser: ["Enfado": 37 586, "Asco": 45 859, "Miedo": 50 457, "Felicidad": 52 151, "Neutral": 30 008, "Tristeza": 24 321, "Sorpresa": 42 471]. Esto ofrece más variedad en cuanto al contenido de las imágenes ya que se mezclan más las demás categorías y en cada época del entrenamiento, los vectores latentes $z \in Z$ son diferentes. En la Figura 9.1 se puede ver la diferencia entre el balance entre clases del *dataset* original y el generado durante el entrenamiento.



(a) *Dataset* original

(b) *Dataset* generados

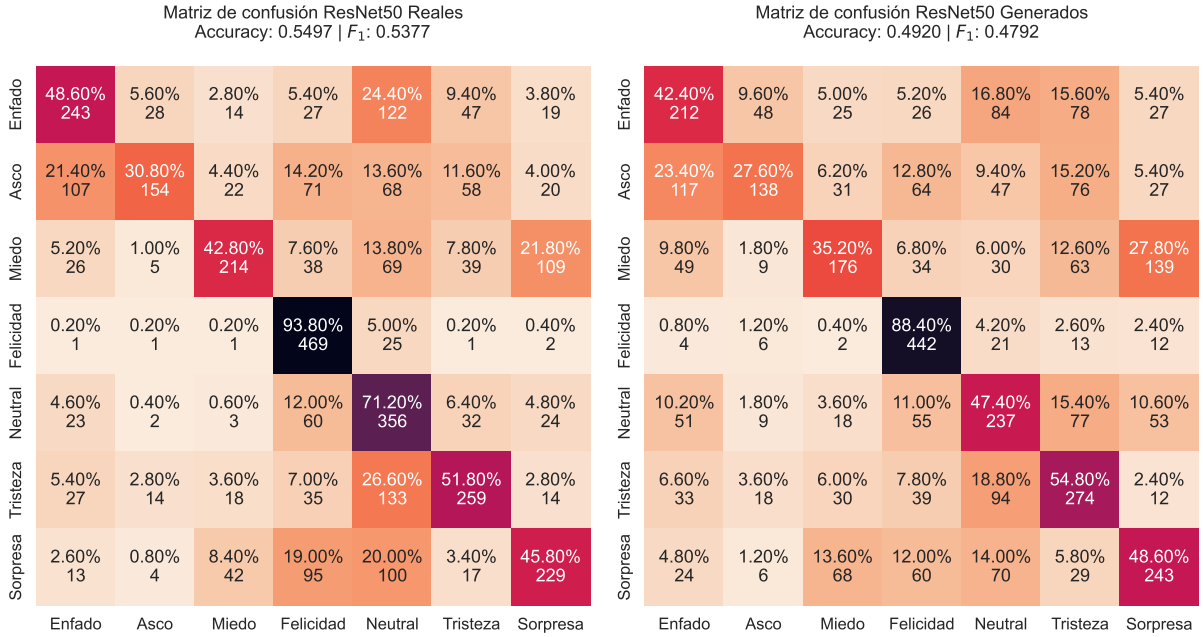
Figura 9.1: Balance entre el número de clases de "Expresión" de cada *dataset* (en %).

Para ver cual de las dos opciones obtiene los mejores resultados, se configuran los entrenamientos para varias épocas (entre 5 y 10) con ambos métodos y se entrenan los modelos con los datos reales y con los datos generados. Para los datos generados se utilizan los dos modelos seleccionados en la fase previa. Después del entrenamiento, se calculan los valores de *accuracy*, *precision*, *recall* y F_1 *score* para cada modelo obtenido y se selecciona el mejor para cada configuración basándose en su valor F_1 *score*. Estos valores se consiguen evaluando la partición de validación que tiene 500 ejemplos para cada clase. En la Tabla 9.1 se pueden ver los valores de cada métrica para los mejores modelos de todas las configuraciones de generación y tipo de datos, a la vez que los distintos modelos seleccionados en la fase anterior. También se añade el tiempo medio de entrenamiento por época. En negrita se marcan los mejores valores por métrica de los modelos entrenados con datos generados.

Modelo	**Balanceo**	**Accuracy ↑**	**Precisión ↑**	**Recall ↑**	**F_1 ↑**	**Tiempo por época**
Datos reales		0.5497	0.5847	0.5497	0.5377	2h 39m 19s
Datos Experimento 2		0.4769	**0.4964**	0.4769	0.4752	38m 58s
	✓	0.4834	0.4924	0.4834	0.4748	40m 23s
Datos Experimento 4		0.4771	0.4903	0.4771	0.4777	1h 1m 9s
	✓	**0.4920**	0.4955	**0.4920**	**0.4792**	59m 42s

Tabla 9.1: Resultados de los modelos ResNet50 de cada configuración.

Como se puede ver en la tabla anterior, el modelo ResNet50 con mejor valor en F_1 es el del Experimento 4 con el balanceo de clases explicado antes. Para simplificar la comparativa entre los datos reales y los generados, a partir de ahora se muestra la información de este modelo. Con esto, se procede a comparar la matriz de confusión del modelo entrenado con los datos reales, al que se denomina a partir de ahora como M_R (véase Figura 9.2a), y la matriz de confusión del modelo entrenado con los datos del Experimento 4 con el balanceo de clases, el cual se denomina M_G (véase Figura 9.2b).



(a) Datos reales M_R

(b) Datos generados M_G

Figura 9.2: Matrices de confusión de los modelos ResNet50 entrenados.

Se puede ver cómo el mapa de calor aplicado a la matriz de confusión de ambos modelos es bastante parecido. Ambos modelos parecen equivocarse de manera parecida en casi todas las clases. En general, el modelo M_G ofrece precisiones más bajas en casi todas las clases comparado con las de M_R, excepto en las clases "Tristeza" y "Sorpresa", donde M_G obtiene valores algo mejores. Aún teniendo mejores valores en esas dos clases, la mayor diferencia está en la clase "Neutral", la cual desciende un 23,8 % en *recall*. A su vez, ambos modelos parecen tener un buen *recall* en la clase "Felicidad", donde consiguen el valor más alto de entre todas las clases. Está claro que entre los dos, el mejor es el modelo M_R pues consigue mejores resultados y tiene mayor valor en F_1 y en *accuracy*.

Hay que aclarar que el modelo M_G obtiene unos resultados satisfactorios si se tiene en cuenta que ha sido entrenado con imágenes generadas y que no se trata de datos de personas reales. Esto demuestra que, aunque los resultados no sean los mejores, se pueden conseguir modelos viables a partir de este tipo de modelos generativos. Además, teniendo en cuenta que dichos modelos han sido entrenados directamente del generador, no es necesario guardar estos datos de manera persistente. Si se requiere, se podría guardar un único fichero CSV con los vectores latentes y sus etiquetas. Así, se reduce enormemente la necesidad de espacio en disco para el guardado de los datos, así como su transporte, ya que solo es necesario tener el código que permite cargar el modelo y el fichero de datos en cuestión. Otra de sus ventajas es que el tamaño necesario para almacenarlo no varía demasiado si se requiere de un *dataset* más grande. A diferencia de si se quiere utilizar un *dataset* de datos reales, situación que requeriría de más almacenamiento para albergar esa cantidad adicional de datos.

Como se ha podido observar a lo largo de los entrenamientos, el tiempo de entrenamiento de los modelos basados en imágenes reales es superior al tiempo de entrenamiento de los modelos basados en imágenes generadas. Una época en los modelos con datos reales tarda entre 2 y 3 horas en entrenar. A diferencia de una época en los modelos con datos generados, que tarda entre 40 minutos y una hora. La diferencia es bastante notoria y da una clara ventaja en cuanto a rapidez de entrenamiento. Esto se debe a que, al tener cargado el modelo generador a la vez que las etiquetas en la memoria de la GPU durante el entrenamiento, se evita el tener que cargar los datos desde el disco duro a la memoria RAM y otra vez a la GPU. Por eso las operaciones son más rápidas y permiten tener esos tiempos de entrenamiento. Aún así, hay que tener en cuenta que esto puede estar pasando dada la configuración del equipo utilizado y sería necesario estudiar con más profundidad estos tiempos de entrenamiento.

Cabe destacar que con el Método 2 se podrían obtener imágenes de mejor calidad que resultasen en un modelo de detección de emociones mejor que el conseguido hasta ahora. Pero esto no se puede demostrar puesto que no se disponen de dichas imágenes por las razones comentadas en el Apartado 7.2.

10. CONCLUSIÓN Y LÍNEAS FUTURAS

El propósito de este trabajo era estudiar la viabilidad de generar rostros humanos etiquetados mediante GANs para entrenar modelos de reconocimiento de expresiones faciales. Para ello se han acometido los objetivos del estudio del arte de las redes GAN, así como la experimentación con modelos condicionados para generar bases de datos de rostros humanos etiquetados. Una vez comparados, se han seleccionado los mejores modelos generativos para su posterior estudio comparativo entre los datos que estos pueden generar y los datos reales. Con los resultados obtenidos se ha podido concluir que, aunque éstos no son perfectos, es viable entrenar modelos de reconocimiento de emociones con los datos generados por una red generativa adversaria.

Está claro que el problema del sesgo en los datos es un tema muy importante y uno de los motivos por los que la generación de datos condicionada de este trabajo muestra defectos en algunos casos. Debido al desbalanceo intrínseco que hay entre algunos de los atributos demográficos en ciertas clases de AffectNet [43], la calidad de las imágenes generadas para las combinaciones de clases que representan a menos ejemplos en los datos reales (o incluso combinaciones no presentes en el *dataset* original) se ve afectada. Por eso se cree que, AffectNet [42] no es la mejor opción para generar datos variados en cuanto a raza, género y edad, aunque pueda ser útil para generar datos en cuanto a la expresión.

A lo largo del trabajo se han ido encontrando problemas complejos de resolver. Pero con el estudio del problema y con perseverancia, se ha conseguido comprender el funcionamiento de estos modelos tan complejos y, lo más importante, cómo utilizarlos para generar rostros humanos de buena calidad con las etiquetas deseadas. También ha sido complicado escoger entre todos los métodos encontrados en la literatura. Dada la creciente oleada de avances, ha sido necesario estudiar el estado del arte de los modelos generativos de manera continuada, aumentando así la cantidad de ideas entre las que escoger.

En definitiva, el área de estudio de los modelos generativos es un campo relativamente nuevo en el que cada año se hacen muchos avances y la literatura sobre estos crece de manera abrumadora. A lo largo del desarrollo del proyecto, se han ido publicando una serie de artículos de gran importancia como el sucesor de StyleGAN2 [1], conocido como Alias-Free Gan[13], o modelos generativos diferentes a las redes GAN, como los modelos de difusión. Modelos como DALLE-2 [56] o artículos como [64] son claros ejemplos de los avances que se están consiguiendo en este ámbito.

Con estos modelos se ofrece un nuevo campo de estudio sobre la posibilidad de crear bases de datos más balanceadas, evitando sesgos intrínsecos de los datos que se suelen encontrar en las bases de datos comunes. Además, se abre la posibilidad de prescindir de las bases de datos convencionales de rostros humanos, sujetas al Reglamento General de Protección de Datos [16]. Por ello, dentro de la línea de trabajo, se plantea seguir estudiando distintos métodos de generación de datos. Entre ellos, la implementación del Método 2 propuesto en este trabajo. Según los resultados teóricos obtenidos, este método debería ser capaz de generar imágenes de mejor calidad y mayor variedad que los conseguidos. Si se continua con esta línea de trabajo, se abriría la posibilidad a que proyectos pequeños puedan generar sus propias bases de datos con las que poder trabajar. Como bien dice Nichole Jordan, socia directora en Grant Thornton LLP: "Ya no requiere de un presupuesto multimillonario para que la IA funcione en su empresa. Esto representa una oportunidad para nivelar el campo de juego para las empresas más pequeñas". Esto, al fin y al cabo, ofrecería más oportunidades de desarrollo y, en definitiva, más avances en la investigación en varias áreas de la Inteligencia Artificial.

REFERENCIAS

[1] Tero Karras et al. «Analyzing and Improving the Image Quality of StyleGAN». En: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (dic. de 2019), págs. 8107-8116. ISSN: 10636919. DOI: 10.48550/arxiv.1912.04958. URL: https://arxiv.org/abs/1912.04958v2.

[2] Aditya Kunar. «Effective and Privacy preserving Tabular Data Synthesizing». En: (ago. de 2021). DOI: 10.48550/arxiv.2108.10064. URL: https://arxiv.org/abs/2108.10064v1.

[3] Geoffrey E Hinton Alex Krizhevsky Ilya Sutskever. «ImageNet Classification with Deep Convolutional Neural Networks». En: (jun. de 2017).

[4] The Economist. *From not working to neural networking.* Jun. de 2016. (Accedido: 16.03.2022).

[5] Raymond Perrault et al. «2019 annual report ar intelligence index Project Manager and Report Editor-in-Chief Steering Committee Steering Commitee Introduction Report Highlights Acknowledgements». En: (2019). URL: https://hai.stanford.edu/sites/default/files/ai_index_2019_report.pdf.

[6] Ian J. Goodfellow et al. «Generative Adversarial Networks». En: (jun. de 2014). DOI: 10.48550/arxiv.1406.2661. URL: https://arxiv.org/abs/1406.2661.

[7] Mengyu Chu et al. «Learning Temporal Coherence via Self-Supervision for GAN-based Video Generation». En: *ACM Transactions on Graphics* 39.4 (nov. de 2018). DOI: 10.1145/3386569.3392457. URL: http://arxiv.org/abs/1811.09393%20http://dx.doi.org/10.1145/3386569.3392457.

[8] *jantic/DeOldify: A Deep Learning based project for colorizing and restoring old images (and video!)* 2019. URL: https://github.com/jantic/DeOldify/.

[9] Martin Arjovsky, Soumith Chintala y Léon Bottou. «Wasserstein GAN». En: *arXiv:1701.07875 [cs, stat]* (2017). URL: http://arxiv.org/abs/1701.07875.

[10] Alec Radford, Luke Metz y Soumith Chintala. «Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks». En: *arXiv:1511.06434 [cs]* (2016). URL: http://arxiv.org/abs/1511.06434.

[11] Xi Chen et al. «InfoGAN: Interpretable Representation Learning by Information Maximizing Generative Adversarial Nets». En: *arXiv:1606.03657 [cs, stat]* (2016). URL: http://arxiv.org/abs/1606.03657.

[12] Tero Karras et al. «Progressive Growing of GANs for Improved Quality, Stability, and Variation». En: (oct. de 2017). DOI: 10.48550/arxiv.1710.10196. URL: http://arxiv.org/abs/1710.10196.

[13] Tero Karras et al. «Alias-Free Generative Adversarial Networks». En: *arXiv:2106.12423 [cs, stat]* (2021). URL: http://arxiv.org/abs/2106.12423.

[14] Bhanu Neupane et al. *UNESCO Series on Internet Freedom Steering AI and Advanced ICTs for Knowledge Societies*. Nov. de 2019. ISBN: 978-92-3-100363-9.

[15] Byoung Chul Ko. «A Brief Review of Facial Emotion Recognition Based on Visual Information». En: *Sensors (Basel, Switzerland)* 18.2 (feb. de 2018). ISSN: 1424-8220. DOI: 10.3390/S18020401. URL: https://pubmed.ncbi.nlm.nih.gov/29385749/.

[16] *BOE.es - DOUE-L-2016-80807 Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (Reglamento general de protección de datos)*. URL: https://www.boe.es/buscar/doc.php?id=DOUE-L-2016-80807.

[17] F Rosenblatt. «THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN 1». En: *Psychological Review* 65.6 (1958), págs. 19-27.

[18] David E. Rumelhart, Geoffrey E. Hinton y Ronald J. Williams. «Learning representations by back-propagating errors». En: *Nature 1986 323:6088* 323.6088 (1986), págs. 533-536. ISSN: 1476-4687. DOI: 10.1038/323533a0. URL: https://www.nature.com/articles/323533a0.

[19] Seppo Linnainmaa. «Taylor expansion of the accumulated rounding error». En: *BIT Numerical Mathematics 1976 16:2* 16.2 (1976), págs. 146-160. ISSN: 1572-9125. DOI: 10.1007/BF01931367. URL: https://link.springer.com/article/10.1007/BF01931367.

[20] Sergey Ioffe y Christian Szegedy. «Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift». En: *32nd International Conference on Machine Learning, ICML 2015* 1 (feb. de 2015), págs. 448-456. DOI: 10.48550/arxiv.1502.03167. URL: https://arxiv.org/abs/1502.03167v3.

[21] Geoffrey E. Hinton et al. «Improving neural networks by preventing co-adaptation of feature detectors». En: (jul. de 2012). DOI: 10.48550/arxiv.1207.0580. URL: https://arxiv.org/abs/1207.0580v1.

[22] Diederik P. Kingma y Jimmy Lei Ba. «Adam: A Method for Stochastic Optimization». En: *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings* (dic. de 2014). DOI: 10.48550/arxiv.1412.6980. URL: https://arxiv.org/abs/1412.6980v9.

[23] Shiv Ram Dubey et al. «diffGrad: An Optimization Method for Convolutional Neural Networks». En: *IEEE Transactions on Neural Networks and Learning Systems* 31.11 (sep. de 2019), págs. 4500-4511. ISSN: 21622388. DOI: 10.48550/arxiv.1909.11015. URL: https://arxiv.org/abs/1909.11015v4.

[24] Joseph Redmon et al. «You Only Look Once: Unified, Real-Time Object Detection». En: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* 2016-December (jun. de 2015), págs. 779-788. ISSN: 10636919. DOI: 10.48550/arxiv.1506.02640. URL: https://arxiv.org/abs/1506.02640v5.

[25] Karen Simonyan y Andrew Zisserman. «Very Deep Convolutional Networks for Large-Scale Image Recognition». En: *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings* (sep. de 2014). DOI: 10.48550/arxiv.1409.1556. URL: https://arxiv.org/abs/1409.1556v6.

[26] Diederik P. Kingma y Max Welling. «Auto-Encoding Variational Bayes». En: *2nd International Conference on Learning Representations, ICLR 2014 - Conference Track Proceedings* (dic. de 2013). DOI: 10.48550/arxiv.1312.6114. URL: https://arxiv.org/abs/1312.6114v10.

[27] Weihao Weng y Xin Zhu. «U-Net: Convolutional Networks for Biomedical Image Segmentation». En: *IEEE Access* 9 (mayo de 2015), págs. 16591-16603. ISSN: 21693536. DOI: 10.48550/arxiv.1505.04597. URL: https://arxiv.org/abs/1505.04597v1.

[28] Ian J. Goodfellow et al. «Generative Adversarial Networks». En: (jun. de 2014). DOI: 10.48550/arxiv.1406.2661. URL: https://arxiv.org/abs/1406.2661.

[29] Martin Heusel et al. «GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium». En: *Advances in Neural Information Processing Systems* 2017-December (jun. de 2017), págs. 6627-6638. ISSN: 10495258. DOI: 10.48550/arxiv.1706.08500. URL: https://arxiv.org/abs/1706.08500v6.

[30] Xun Huang y Serge Belongie. «Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization». En: *Proceedings of the IEEE International Conference on Computer Vision* 2017-October (mar. de 2017), págs. 1510-1519. ISSN: 15505499. DOI: 10.48550/arxiv.1703.06868. URL: https://arxiv.org/abs/1703.06868v2.

[31] Tero Karras, Samuli Laine y Timo Aila. «A Style-Based Generator Architecture for Generative Adversarial Networks». En: *arXiv:1812.04948 [cs, stat]* (2019). URL: http://arxiv.org/abs/1812.04948.

[32] Christian Szegedy et al. «Rethinking the Inception Architecture for Computer Vision». En: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* 2016-December (dic. de 2015), págs. 2818-2826. ISSN: 10636919. DOI: 10.48550/arxiv.1512.00567. URL: https://arxiv.org/abs/1512.00567v3.

[33] D. C. Dowson y B. V. Landau. «The Fréchet distance between multivariate normal distributions». En: *Journal of Multivariate Analysis* 12.3 (sep. de 1982), págs. 450-455. ISSN: 0047-259X. DOI: 10.1016/0047-259X(82)90077-X.

[34] Tuomas Kynkäänniemi et al. «Improved Precision and Recall Metric for Assessing Generative Models». En: *arXiv:1904.06991 [cs, stat]* (2019). URL: http://arxiv.org/abs/1904.06991.

[35] Dmitry Ulyanov, Andrea Vedaldi y Victor Lempitsky. «Instance Normalization: The Missing Ingredient for Fast Stylization». En: (jul. de 2016). DOI: 10.48550/arxiv.1607.08022. URL: https://arxiv.org/abs/1607.08022v3.

[36] Animesh Karnewar y Oliver Wang. «MSG-GAN: Multi-Scale Gradients for Generative Adversarial Networks». En: *arXiv:1903.06048 [cs, stat]* (2020). URL: http://arxiv.org/abs/1903.06048.

[37] *QMUL-TinyFace*. URL: https://qmul-tinyface.github.io/.

[38] *CelebA Dataset*. URL: https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html.

[39] *UTKFace — Large Scale Face Dataset*. URL: https://susanqq.github.io/UTKFace/.

[40] Ian J. Goodfellow et al. «Challenges in Representation Learning: A report on three machine learning contests». En: *Neural Networks* 64 (jul. de 2013), págs. 59-63. ISSN: 18792782. DOI: 10.48550/arxiv.1307.0414. URL: https://arxiv.org/abs/1307.0414v1.

[41] Kimmo Kärkkäinen y Jungseock Joo UCLA. «FairFace: Face Attribute Dataset for Balanced Race, Gender, and Age». En: (ago. de 2019). DOI: 10.48550/arxiv.1908.04913. URL: https://arxiv.org/abs/1908.04913v1.

[42] Ali Mollahosseini, Behzad Hasani y Mohammad H. Mahoor. «AffectNet: A Database for Facial Expression, Valence, and Arousal Computing in the Wild». En: *IEEE Transactions on Affective Computing* 10.1 (ago. de 2017), págs. 18-31. ISSN: 19493045. DOI: 10.1109/taffc.2017.2740923. URL: https://arxiv.org/abs/1708.03985v4.

[43] Iris Dominguez-Catena, Daniel Paternain y Mikel Galar. «Assessing Demographic Bias Transfer from Dataset to Model: A Case Study in Facial Expression Recognition». En: (mayo de 2022). URL: https://arxiv.org/abs/2205.10049v1.

[44] Kimmo Karkkainen y Jungseock Joo. «FairFace: Face Attribute Dataset for Balanced Race, Gender, and Age for Bias Measurement and Mitigation». en. En: *2021 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2021, págs. 1547-1557. ISBN: 978-1-66540-477-8. DOI: 10.1109/WACV48630.2021.00159. URL: https://ieeexplore.ieee.org/document/9423296/.

[45] O S Keyes. «The Misgendering Machines: Trans/HCI Implications of Automatic Gender Recognition». En: 88 (2018), pág. 22. DOI: 10.1145/3274357. URL: https://doi.org/10.1145/3274357.

[46] *Python Release Python 3.9.6 — Python.org*. URL: https://www.python.org/downloads/release/python-396/.

[47] *Conda — conda 4.12.0.post34+55cd442c documentation*. URL: https://docs.conda.io/projects/conda/en/latest/index.html.

[48] *Novedades de PyCharm 2021.2*. URL: https://www.jetbrains.com/es-es/pycharm/whatsnew/2021-2/.

[49] *Visual Studio Code - Code Editing. Redefined*. URL: https://code.visualstudio.com/.

[50] *Project Jupyter — Home*. URL: https://jupyter.org/.

[51] *Artificial Intelligence Computing Leadership from NVIDIA*. URL: https://www.nvidia.com/en-us/.

[52] *CUDA Zone — NVIDIA Developer*. URL: https://developer.nvidia.com/cuda-zone.

[53] Li Deng. «The MNIST database of handwritten digit images for machine learning research». En: *IEEE Signal Processing Magazine* 29.6 (2012), págs. 141-142. ISSN: 10535888. DOI: 10.1109/MSP.2012.2211477.

[54] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: https://www.tensorflow.org/.

[55] Adam Paszke et al. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». En: *Advances in Neural Information Processing Systems 32*. Ed. por H. Wallach et al. Curran Associates, Inc., 2019, págs. 8024-8035. URL: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf.

[56] Aditya Ramesh et al. «Hierarchical Text-Conditional Image Generation with CLIP Latents». En: (2022).

[57] Alon Shoshan et al. «GAN-Control: Explicitly Controllable GANs». En: *Proceedings of the IEEE International Conference on Computer Vision*. Ene. de 2021, págs. 14063-14073. ISBN: 9781665428125. DOI: 10.1109/ICCV48922.2021.01382. URL: http://arxiv.org/abs/2101.02477.

[58] Rignak. *A conditional PyTorch StyleGAN2*. Python. 2022. URL: https://github.com/CIA-Oceanix/Conditional_StyleGAN2_pytorch.

[59] Phil Wang. *Simple StyleGan2 for Pytorch*. Python. 2022. URL: https://github.com/lucidrains/stylegan2-pytorch.

[60] Justin Lizama. «Implicitly Maximizing Margins with the Hinge Loss». En: (jun. de 2020). DOI: 10.48550/arxiv.2006.14286. URL: https://arxiv.org/abs/2006.14286v1.

[61] Hyunsu Kim. *Improved Precision and Recall Metric for Assessing Generative Models — Pytorch Implementation*. Python. 2019. URL: https://github.com/blandocs/improved-precision-and-recall-metric-pytorch.

[62] Volker Blanz y Thomas Vetter. «A morphable model for the synthesis of 3D faces». En: *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1999* (jul. de 1999), págs. 187-194. DOI: 10.1145/311535.311556.

[63] Kaiming He et al. «Deep Residual Learning for Image Recognition». En: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* 2016-December (dic. de 2015), págs. 770-778. ISSN: 10636919. DOI: 10.48550/arxiv.1512.03385. URL: https://arxiv.org/abs/1512.03385v1.

[64] Jooyoung Choi et al. «ILVR: Conditioning Method for Denoising Diffusion Probabilistic Models». En: (ago. de 2021). DOI: 10.48550/arxiv.2108.02938. URL: https://arxiv.org/abs/2108.02938v2.

ANEXOS

A. Primer anexo

En este anexo se proporcionan todas las posibles imágenes que el modelo del Experimento 4 puede generar, con vectores aleatorios $z \in Z$ diferentes para cada combinación de clases. Los resultados están separados en Figuras por las clases "Expresión" y "Raza4". En cada Figura, cada fila representa a una clase de "Raza" ordenados como ["Negro", "Asiático Este", "Indio", "Latino", "Medio Este", "Asiático Sud-Este", "Blanco"] de arriba a abajo, cada bloque de nueve columnas representa una clase de "Género" ordenados por ["Mujer", "Hombre"] de izquierda a derecha y cada columna de cada bloque representa a una clase de "Edad" ordenados como ["0-2", "3-9", "10-19", "20-29", "30-39", "40-49", "50-59", "60-69", "70+"] también de izquierda a derecha. Hay que tener en cuenta que algunas de las combinaciones entre las clases de "Raza" y "Raza4" ofrecen ejemplos bastante sesgados dependiendo de la combinación dado que estos dos están fuertemente correlacionados. Aunque se sabe que ciertas combinaciones no tienen sentido, solo se quiere ver como se comporta el modelo con estas combinaciones.

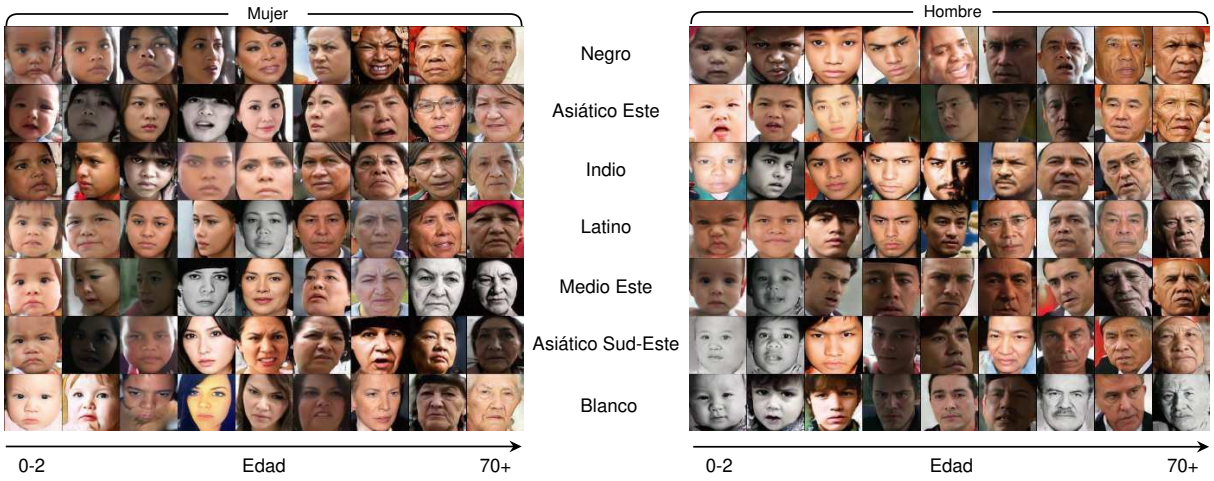


Figura A.1: Expresión "Enfado" y raza "Asiático".

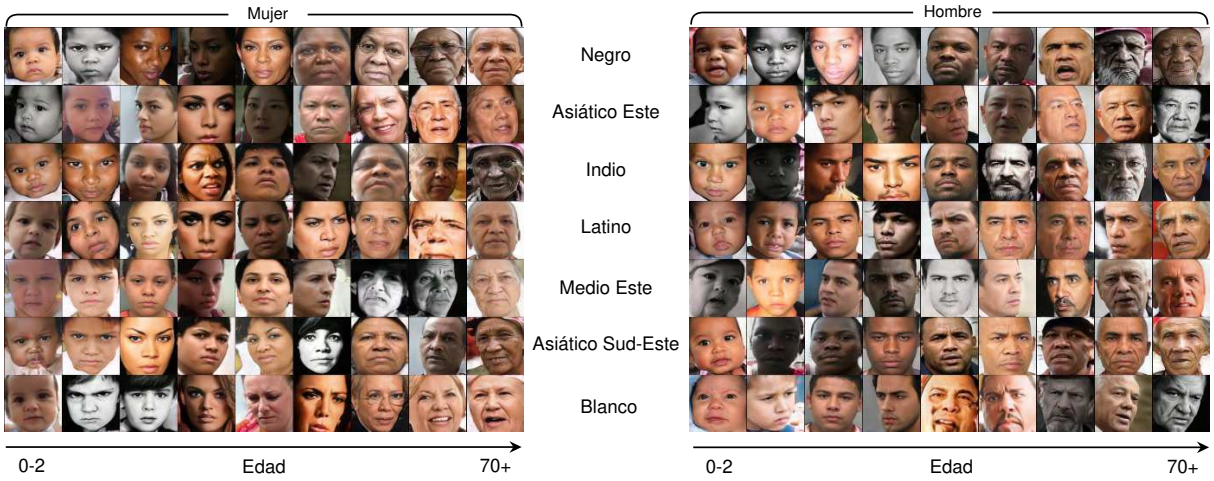


Figura A.2: Expresión "Enfado" y raza "Negro".

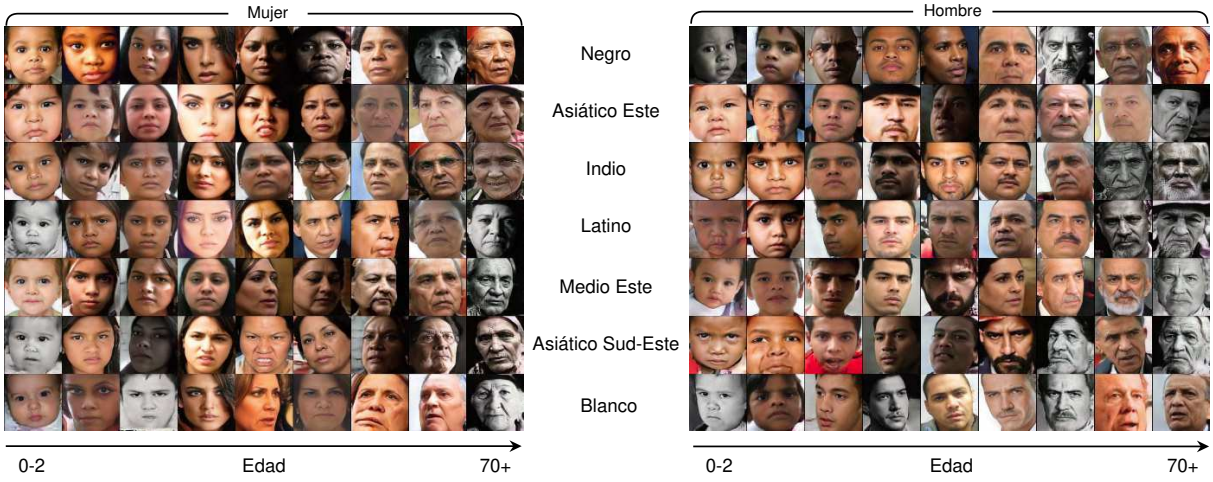


Figura A.3: Expresión "Enfado" y raza "Indio".

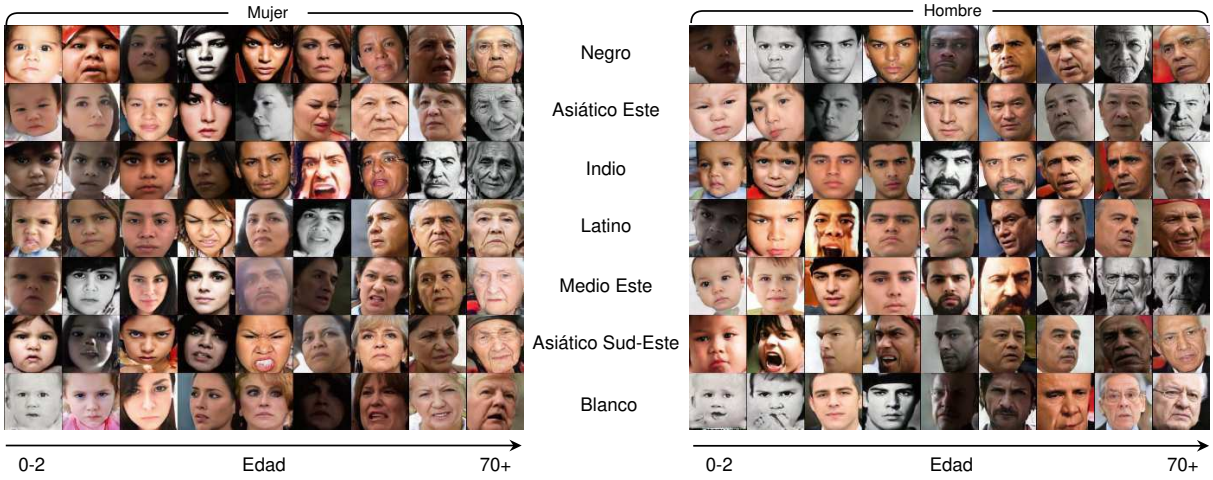


Figura A.4: Expresión "Enfado" y raza "Blanco".

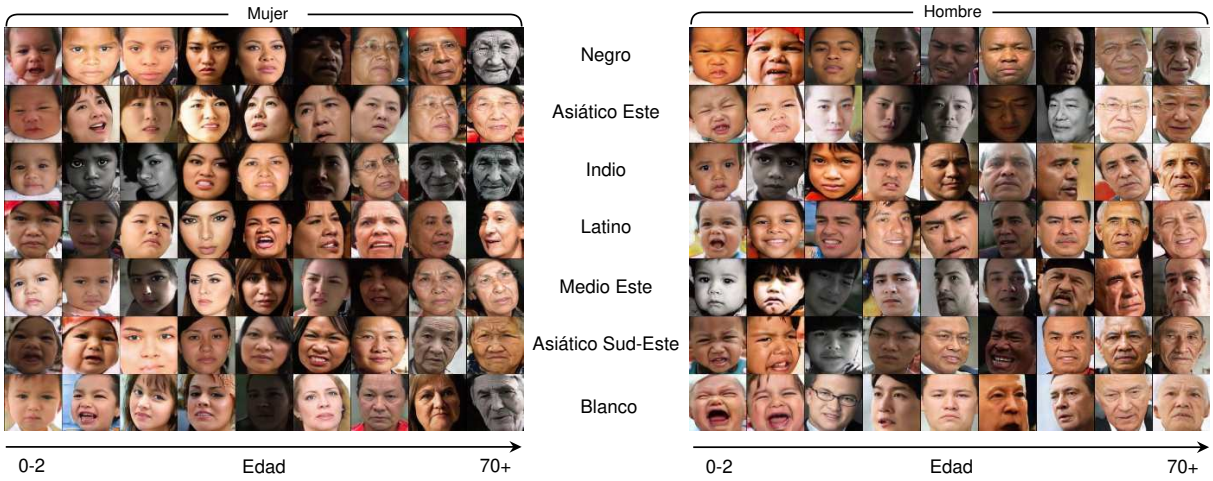


Figura A.5: Expresión "Asco" y raza "Asiático".

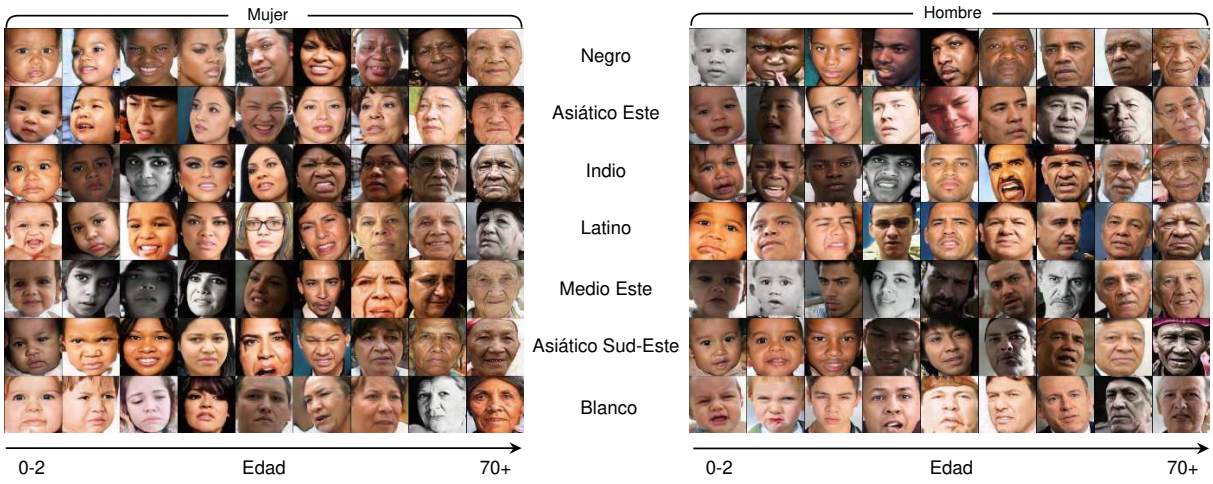


Figura A.6: Expresión "Asco" y raza "Negro".

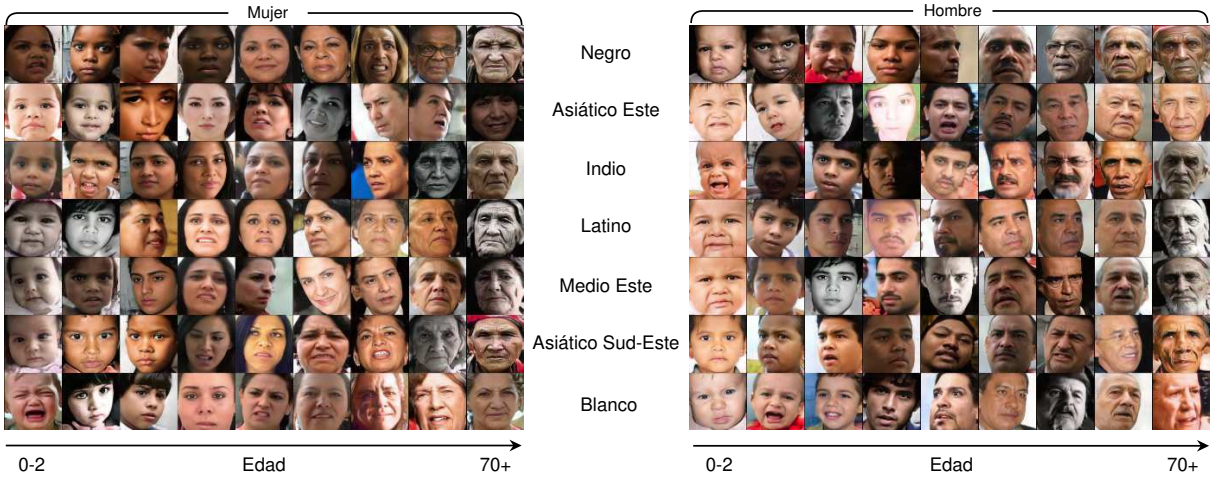


Figura A.7: Expresión "Asco" y raza "Indio".

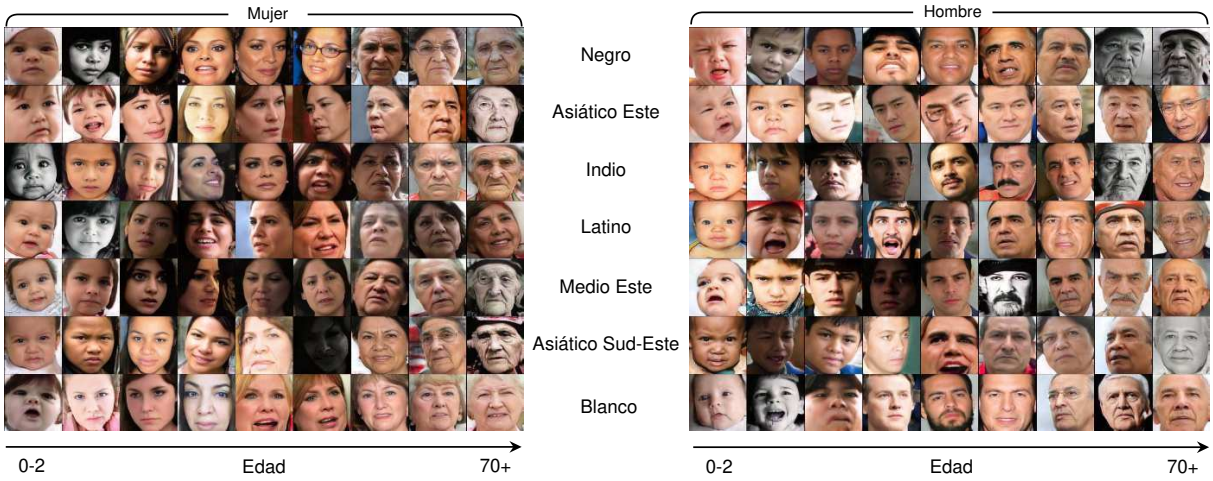


Figura A.8: Expresión "Asco" y raza "Blanco".

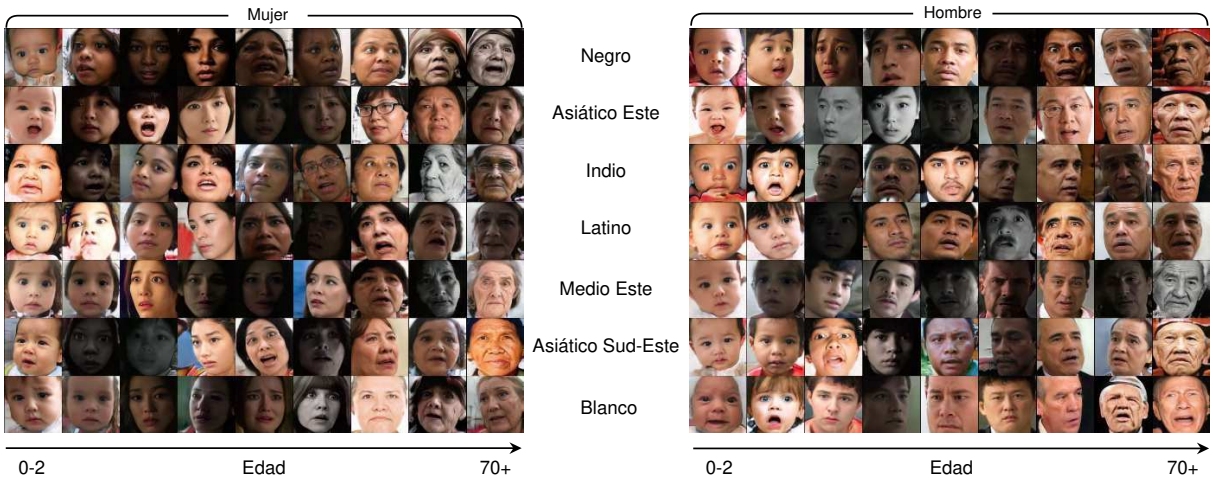


Figura A.9: Expresión “Miedo” y raza “Asiático”.

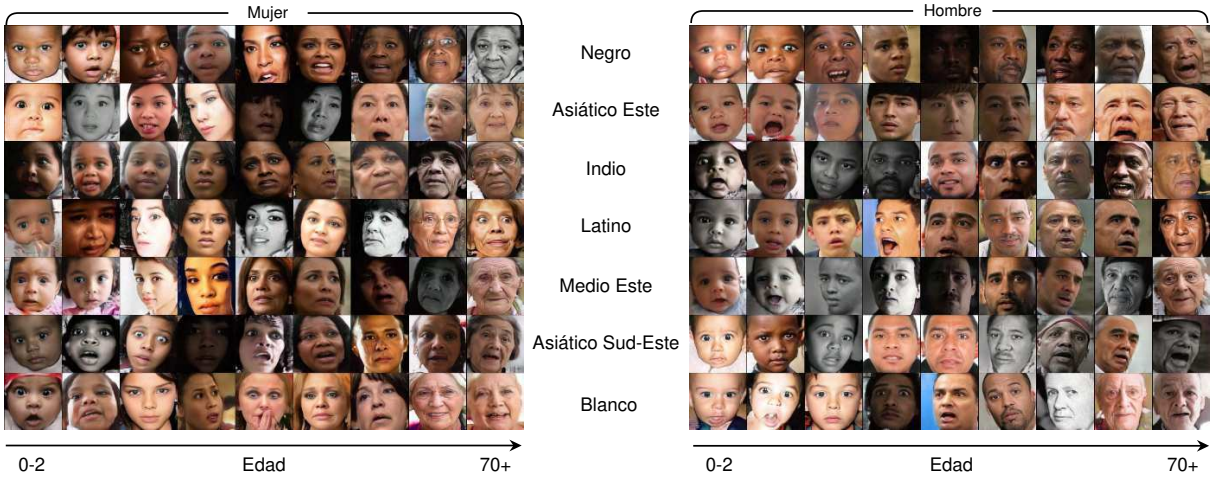


Figura A.10: Expresión “Miedo” y raza “Negro”.

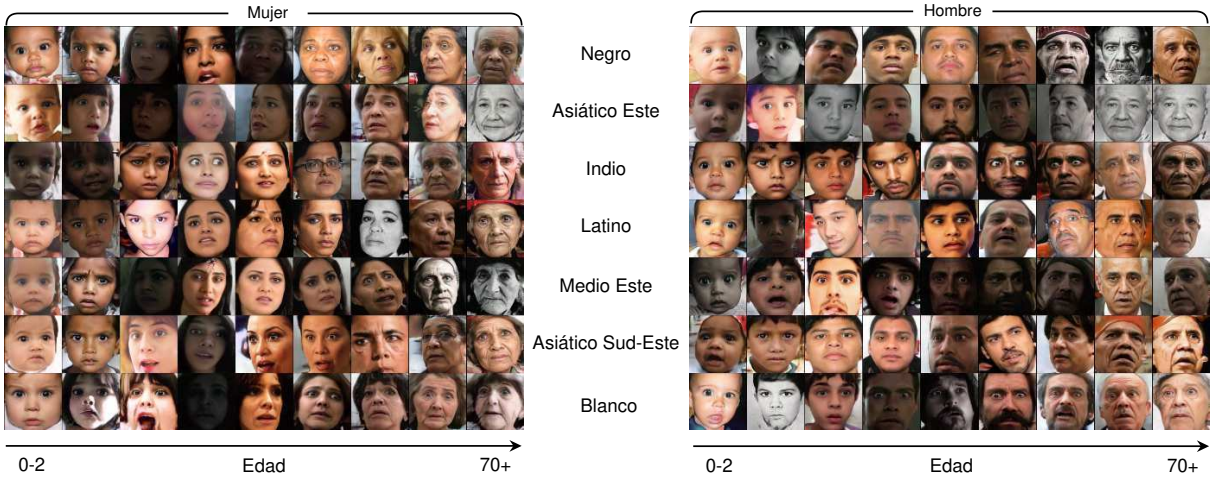


Figura A.11: Expresión “Miedo” y raza “Indio”.

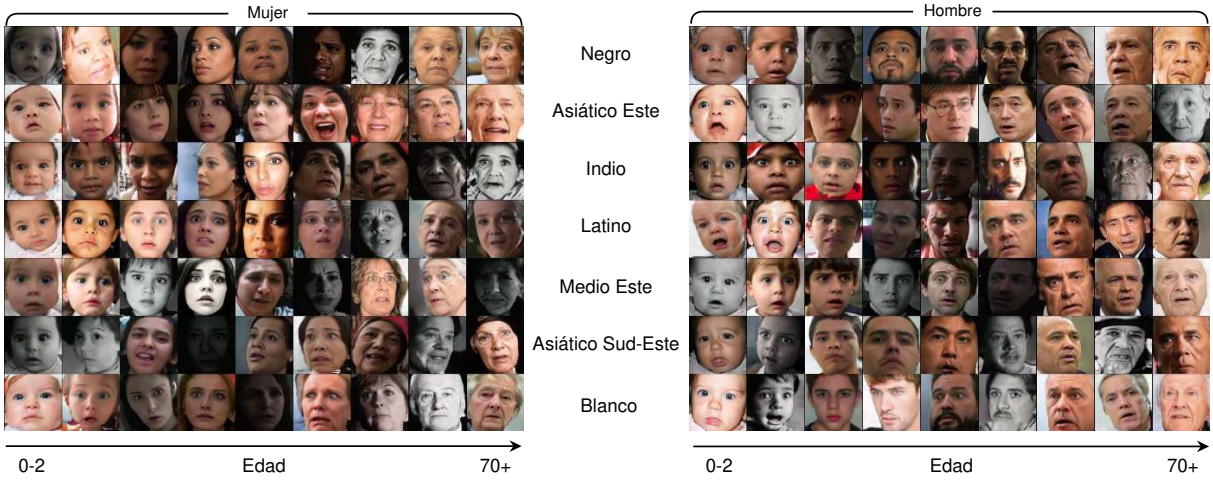


Figura A.12: Expresión "Miedo" y raza "Blanco".

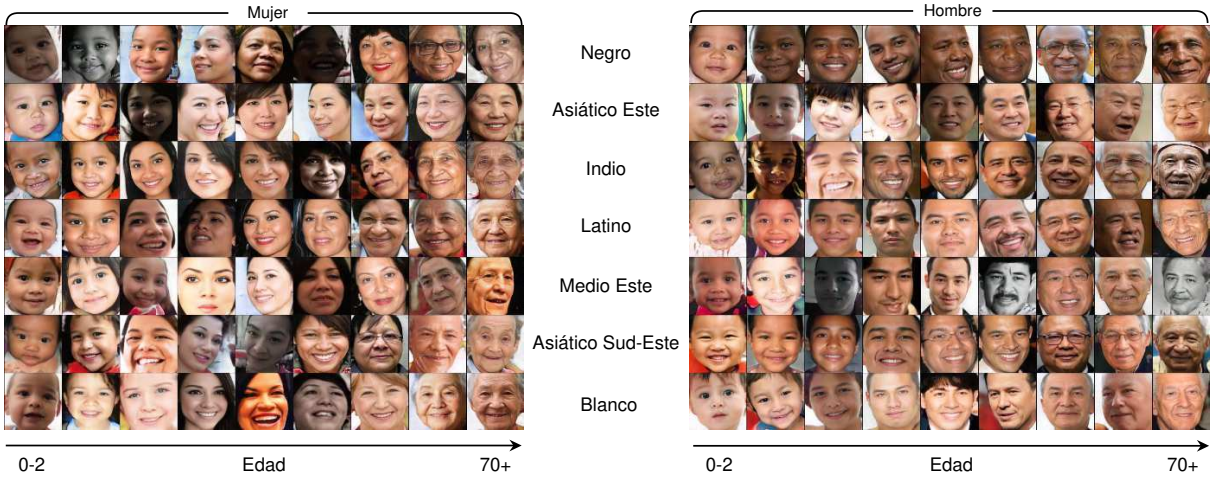


Figura A.13: Expresión "Felicidad" y raza "Asiático".

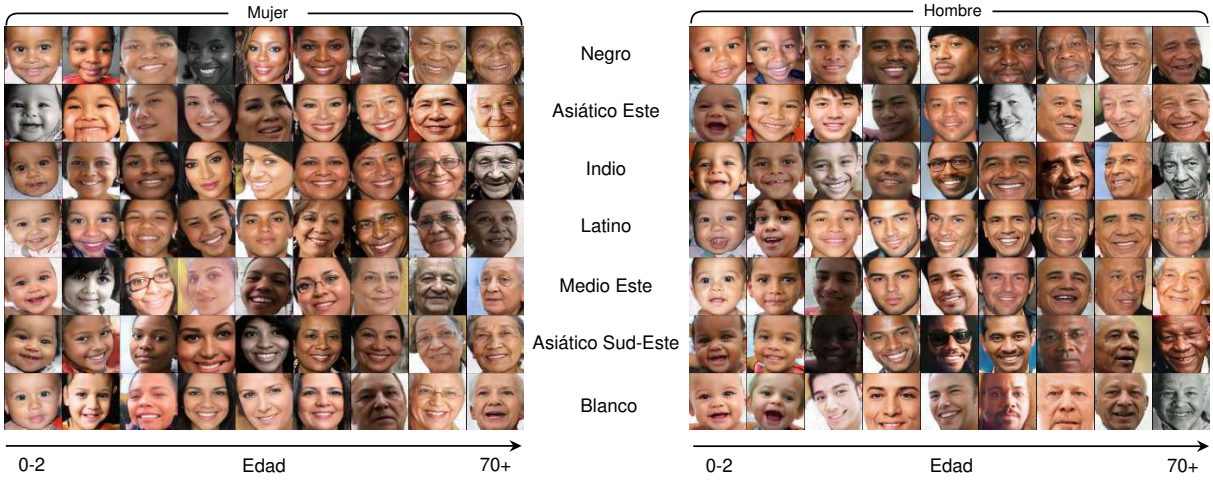


Figura A.14: Expresión "Felicidad" y raza "Negro".

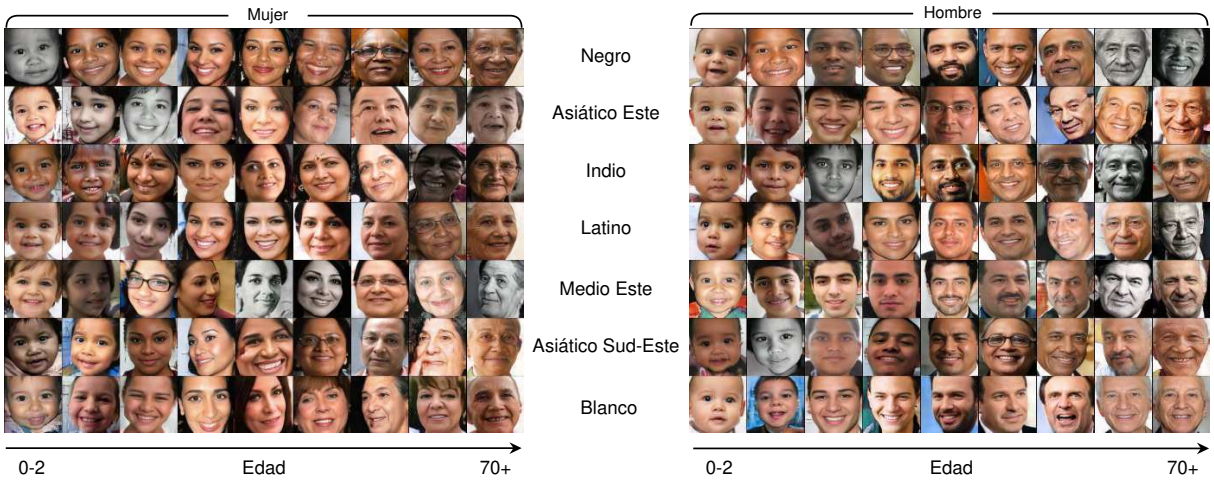


Figura A.15: Expresión “Felicidad” y raza “Indio”.

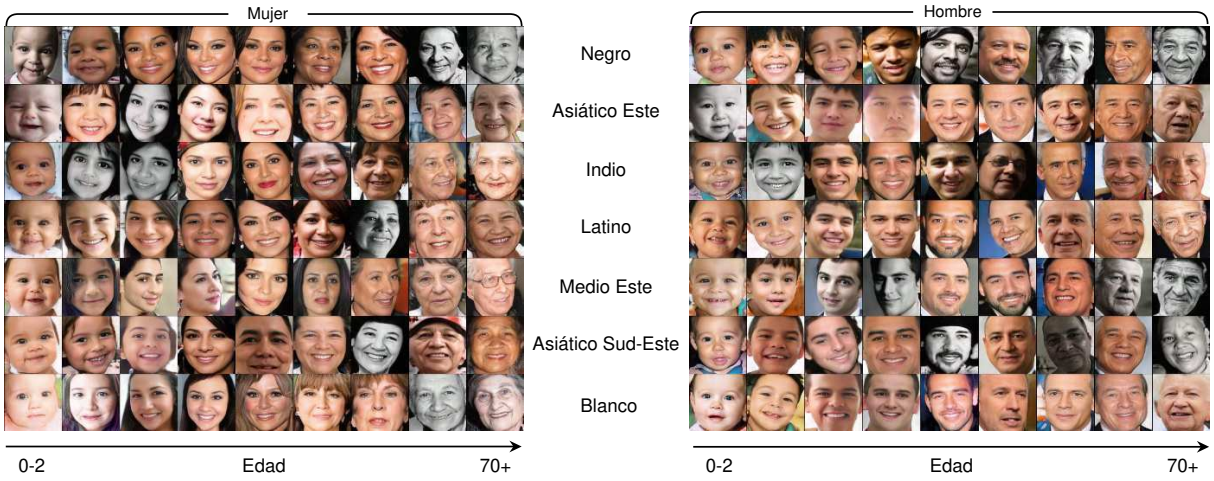


Figura A.16: Expresión “Felicidad” y raza “Blanco”.

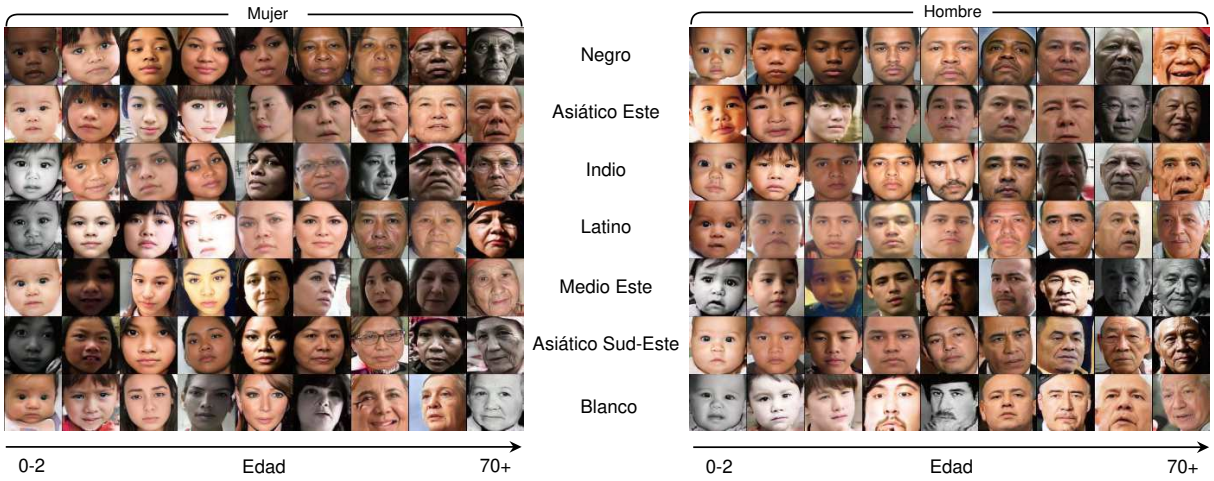


Figura A.17: Expresión “Neutral” y raza “Asiático”.

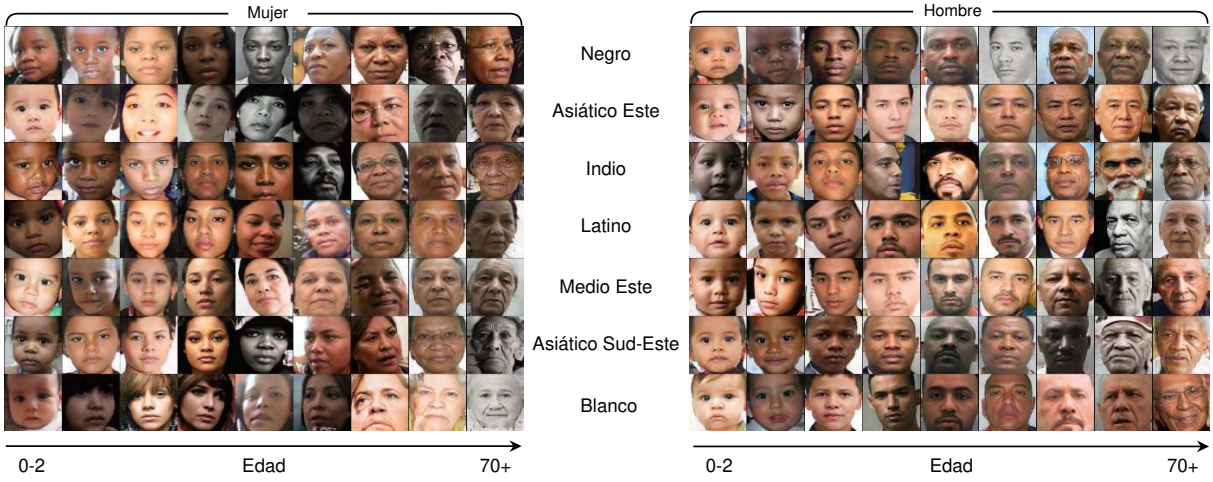


Figura A.18: Expresión "Neutral" y raza "Negro".

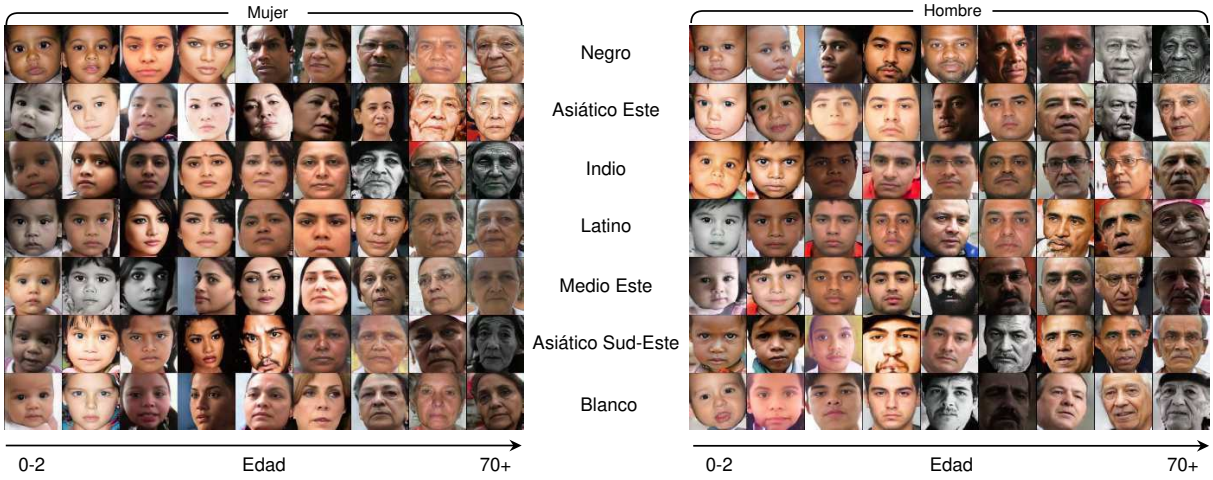


Figura A.19: Expresión "Neutral" y raza "Indio".

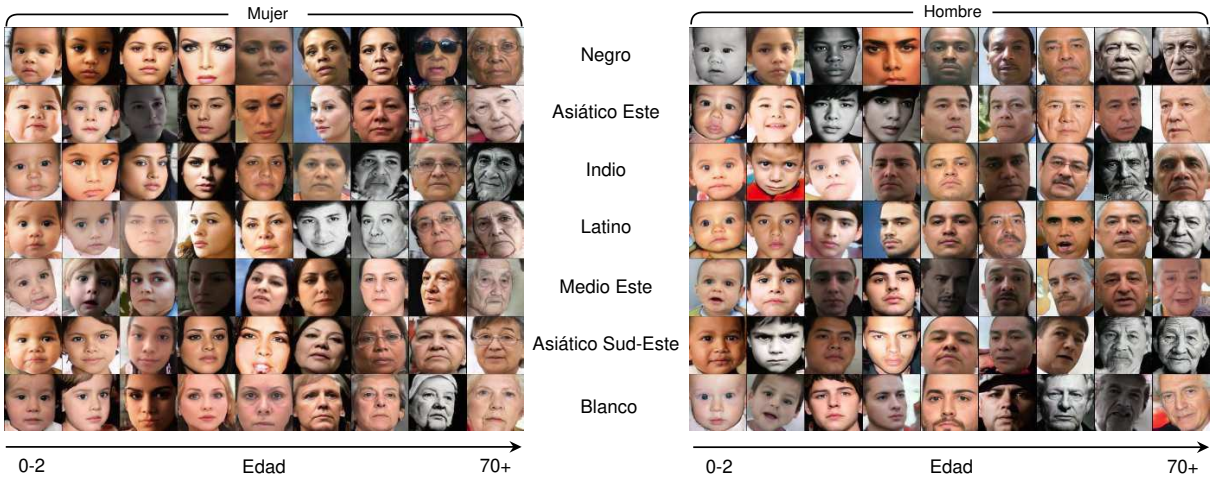


Figura A.20: Expresión "Neutral" y raza "Blanco".

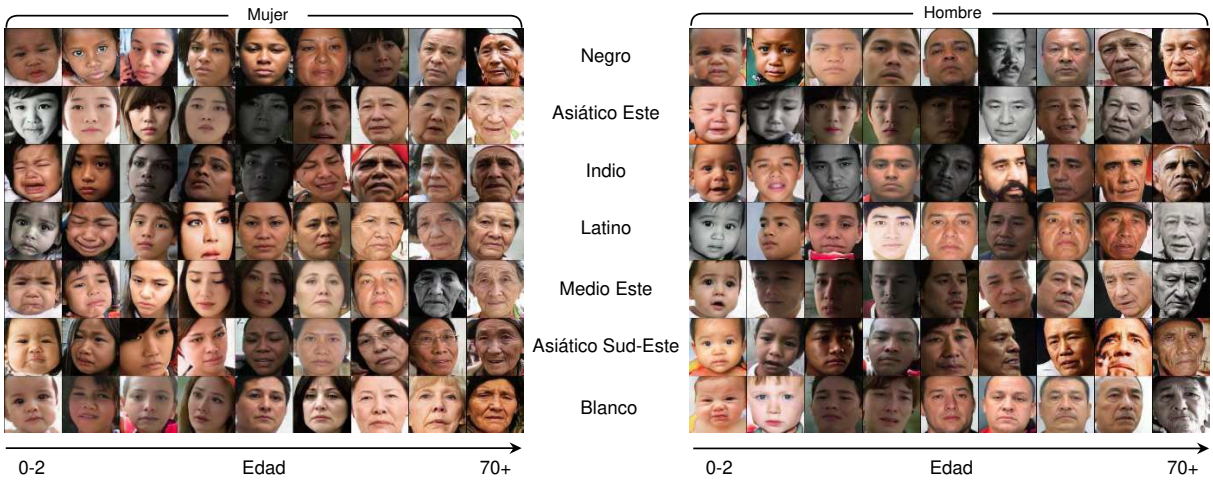


Figura A.21: Expresión "Tristeza" y raza "Asiático".

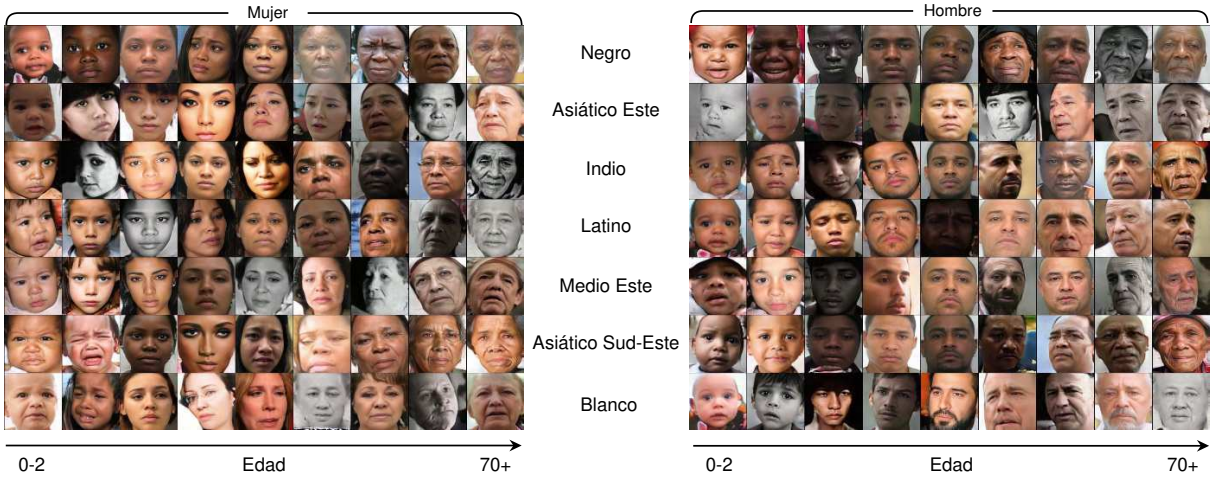


Figura A.22: Expresión "Tristeza" y raza "Negro".

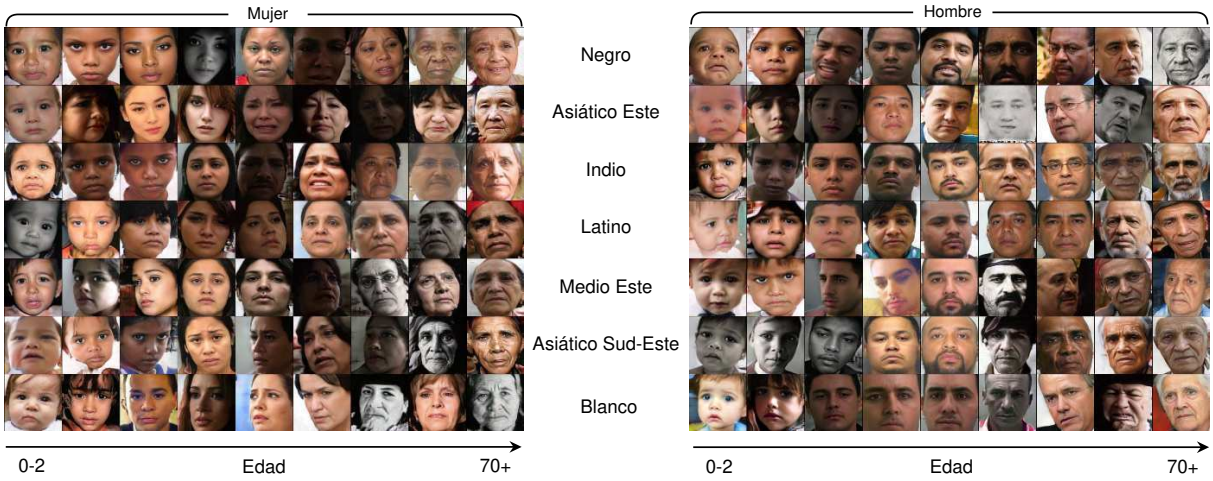


Figura A.23: Expresión "Tristeza" y raza "Indio".

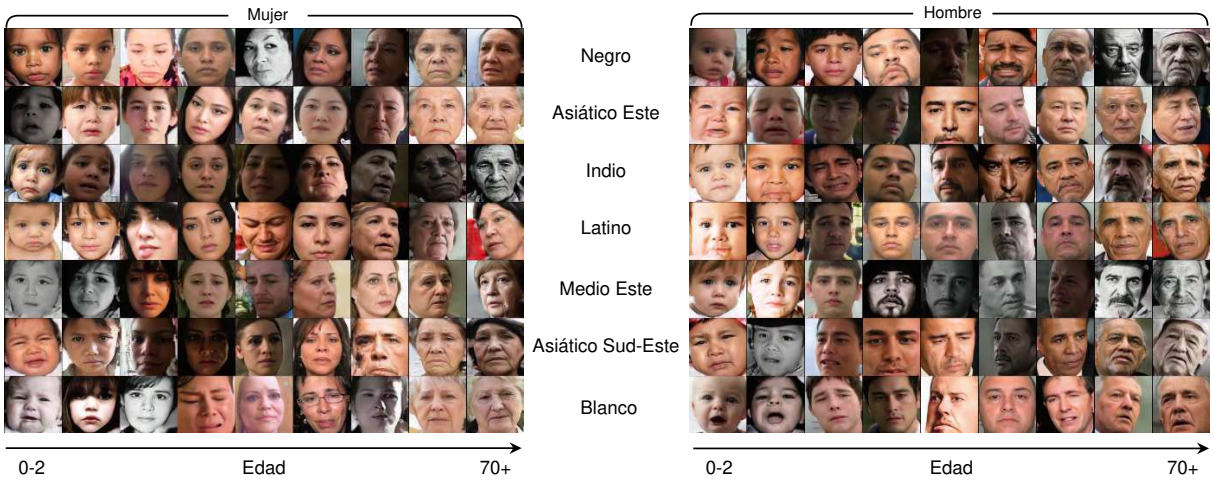


Figura A.24: Expresión "Tristeza" y raza "Blanco".

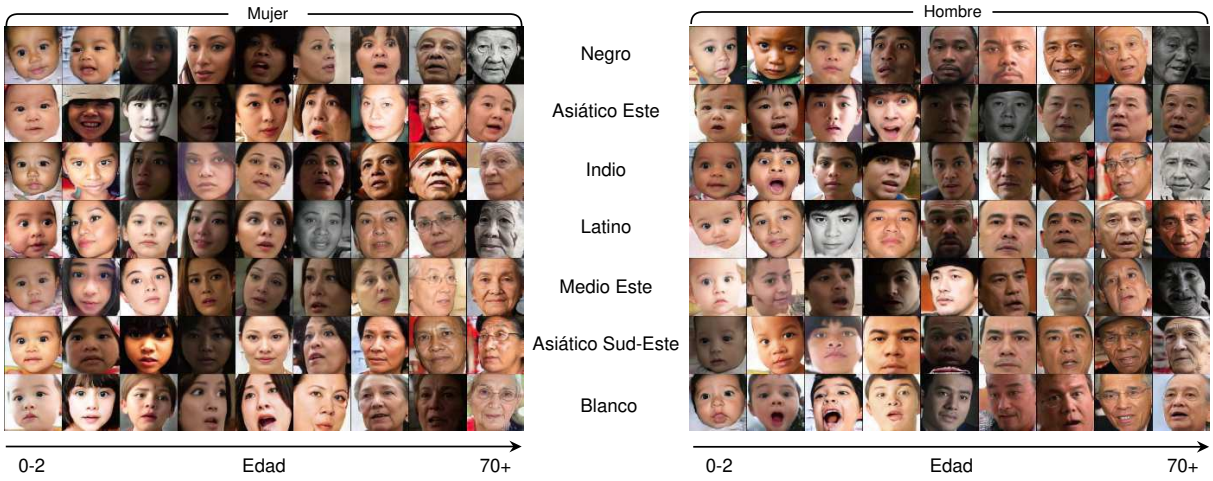


Figura A.25: Expresión "Sorpresa" y raza "Asiático".

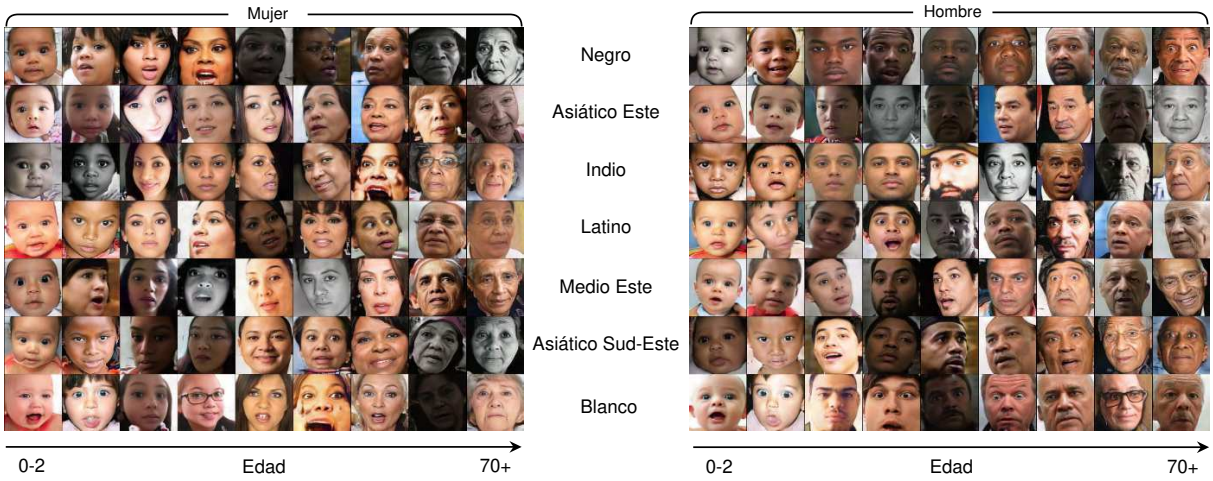


Figura A.26: Expresión "Sorpresa" y raza "Negro".

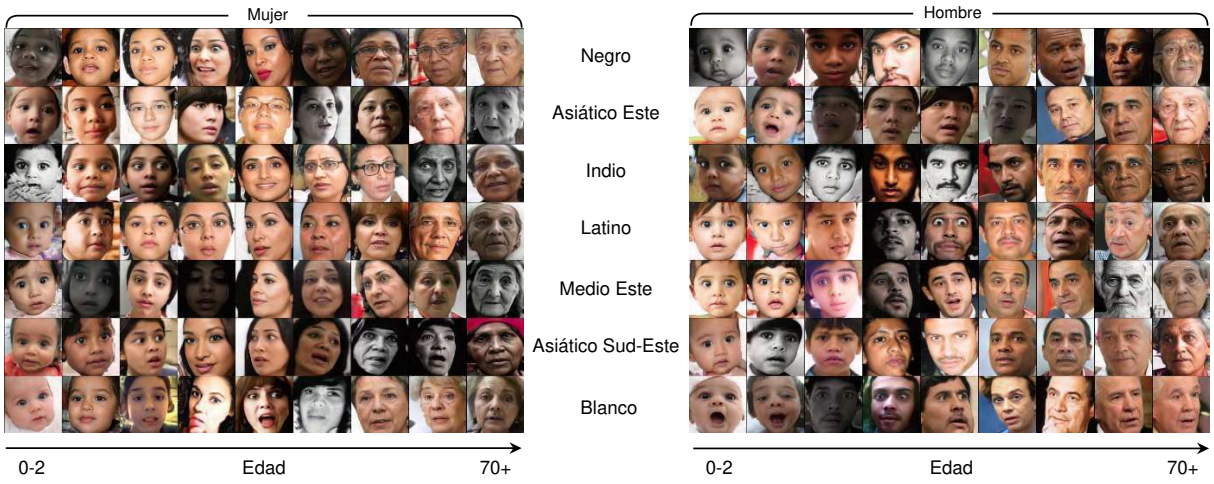


Figura A.27: Expresión “Sorpresa” y raza “Indio”.

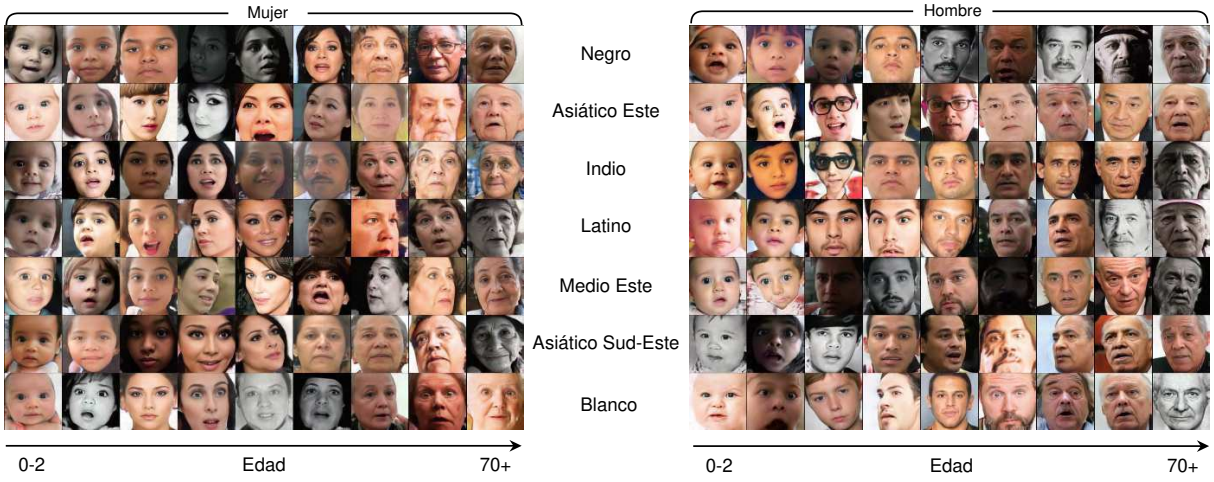


Figura A.28: Expresión “Sorpresa” y raza “Blanco”.